

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему
**ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ КАРТОЧНУЮ
ИГРУ «БЛЕКДЖЕК»**

БГУИР КП 6-05 06 12 01 020 ПЗ

Студент

А. Б. Кляус

Руководитель

С. В. Болтак

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Сравнительный анализ существующих решений	6
1.2 Постановка задачи	8
2 Проектирование программы	10
2.1 Проектирование структуры программы	10
2.2 Проектирование системы сетевого взаимодействия	11
3 Программная реализация	13
3.1 Описание разработанных модулей	13
3.2 Описание сетевого модуля	15
4 Тестирование	17
5 Руководство пользователя	19
Заключение	30
Список использованных источников	31
Приложение А (обязательное) Листинг код с комментариями	32
Приложение Б (обязательное) Схема алгоритма работы системы	82

ВВЕДЕНИЕ

В эпоху цифровизации традиционные настольные игры активно переходят в виртуальное пространство. Одним из востребованных представителей этого класса является блекджек – классическая карточная игра, сочетающая математические закономерности с высокодинамичным процессом. Разработка современного веб-приложения для блекджека представляет собой комплексную инженерную задачу на стыке проектирования распределенных систем, реализации сетевых протоколов реального времени и построения масштабируемых интерфейсов. Актуальность работы обусловлена высоким спросом на интерактивные платформы, способные обеспечивать мгновенный двусторонний обмен данными без задержек, свойственных классической http-модели.

Объектом исследования выступают технологии построения многопользовательских веб-систем реального времени на базе асинхронного подхода. Предметом исследования является процесс проектирования, программной реализации и тестирования игрового сервиса с использованием технологического стека, включающего node.js [1], express [2][3], typescript [4], socket.io [5][6], postgresql [7] и redis [8].

Цель проекта заключается в создании надежной программной системы, предоставляющей пользователям возможности безопасной авторизации, создания виртуальных комнат с гибкой настройкой правил, ведения игровых сессий в реальном времени и сохранения учетных данных в энергонезависимом хранилище.

Практическая значимость разработки определяется возможностью применения реализованных решений в качестве основы для построения более широкого класса многопользовательских игровых платформ. Выработанные подходы к организации серверной логики, управлению состоянием игровых сессий и обеспечению отказоустойчивости могут быть перенесены на смежные предметные области, требующие низкой задержки передачи данных и высокой степени согласованности распределенного состояния.

Для достижения поставленной цели необходимо решить ряд последовательных задач, начиная с проведения сравнительного анализа программных аналогов и формирования комплекса требований к веб-приложению. Далее требуется разработать модульную архитектуру системы, спроектировать протокол полнодуплексного взаимодействия на базе websocket [9] и продумать схему базы данных совместно с механизмами кэширования. Практическая реализация комплекса должна сопровождаться строгой типизацией данных, после чего необходимо провести полномасштабное тестирование готового продукта и разработать техническую документацию по его эксплуатации и развертыванию.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Приступая к проектированию архитектуры многопользовательской игровой веб-платформы, необходимо провести анализ существующих на рынке программных продуктов, предоставляющих функционал карточных игр в реальном времени. Рассмотрим основные аналоги разрабатываемой системы, представленные в различных сегментах индустрии.

1. Программные клиенты крупных коммерческих операторов, таких как *pokerstars* или *888casino*, представляют собой высоконагруженные системы, ориентированные на непрерывную многопользовательскую сессию. Данные решения обеспечивают поддержку миллионов одновременных подключений, предлагают широкий выбор игровых комнат и отличаются минимальной сетевой задержкой за счет использования бинарных протоколов связи. Основным недостатком является необходимость скачивания и установки специализированного программного обеспечения, что исключает возможность мгновенной игры непосредственно в браузере и существенно повышает порог входа для новой аудитории.

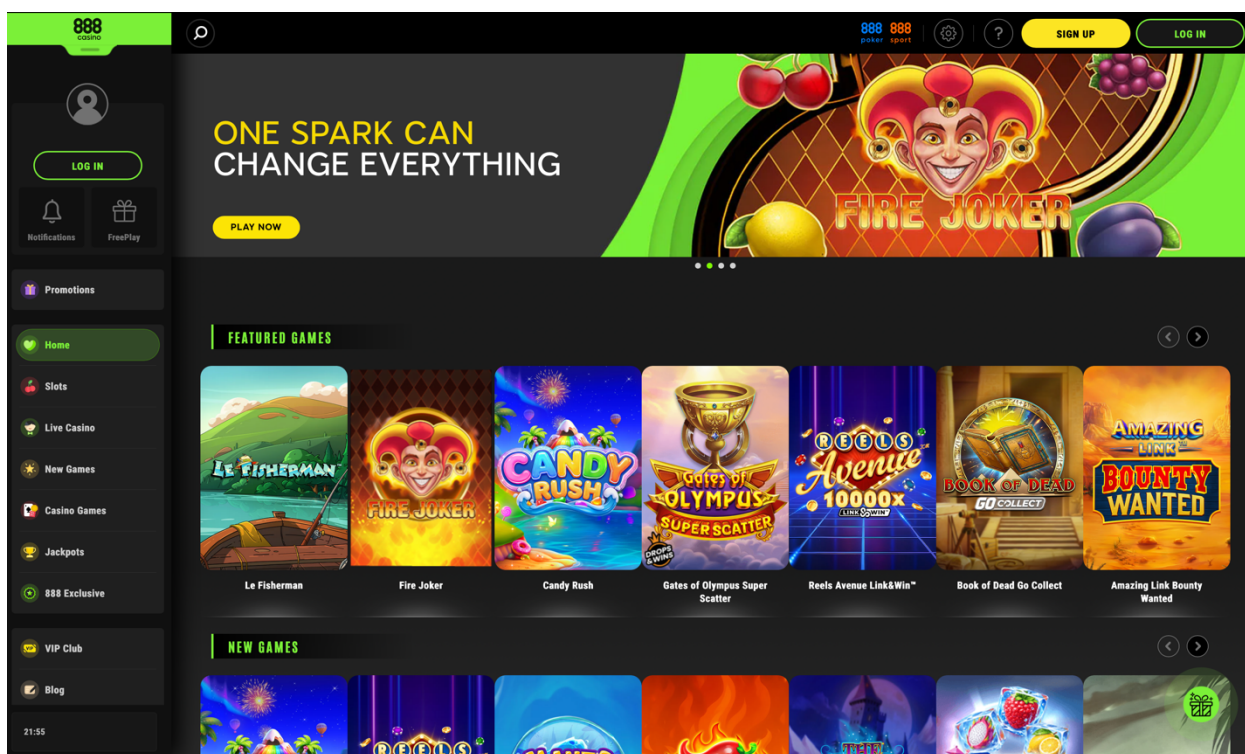


Рисунок 1.1 – 888casino

2. Платформы для интеграции браузерных онлайн-казино, разрабатываемые провайдерами уровня *evolution gaming* или *playtech*, специализируются на предоставлении доступа к карточным играм через веб-интерфейс. Такие системы используют сложные графические webgl-

интерфейсы или потоковое видео с живыми дилерами. Они характеризуются высокой степенью защиты данных и строгим контролем игрового процесса. Однако подобные решения обладают избыточными требованиями к аппаратному обеспечению и пропускной способности сети пользователя, а также представляют собой тяжеловесные коммерческие модули, не предполагающие создания изолированных частных комнат самими игроками.

3. Браузерные социальные казино и многопользовательские веб-симуляторы, например приложения формата *blackjackist* или *zynga poker*, ориентированы на массовую аудиторию и казуальный игровой процесс. Эти продукты легко запускаются в любом современном браузере без предварительной установки и обладают интуитивно понятной системой взаимодействия. К существенным недостаткам таких решений относятся слабая консистентность игрового состояния при нестабильном интернет-соединении, перегруженность интерфейса сторонними социальными функциями и частые отступления от классической математической модели игры в угоду удержанию внимания пользователей.

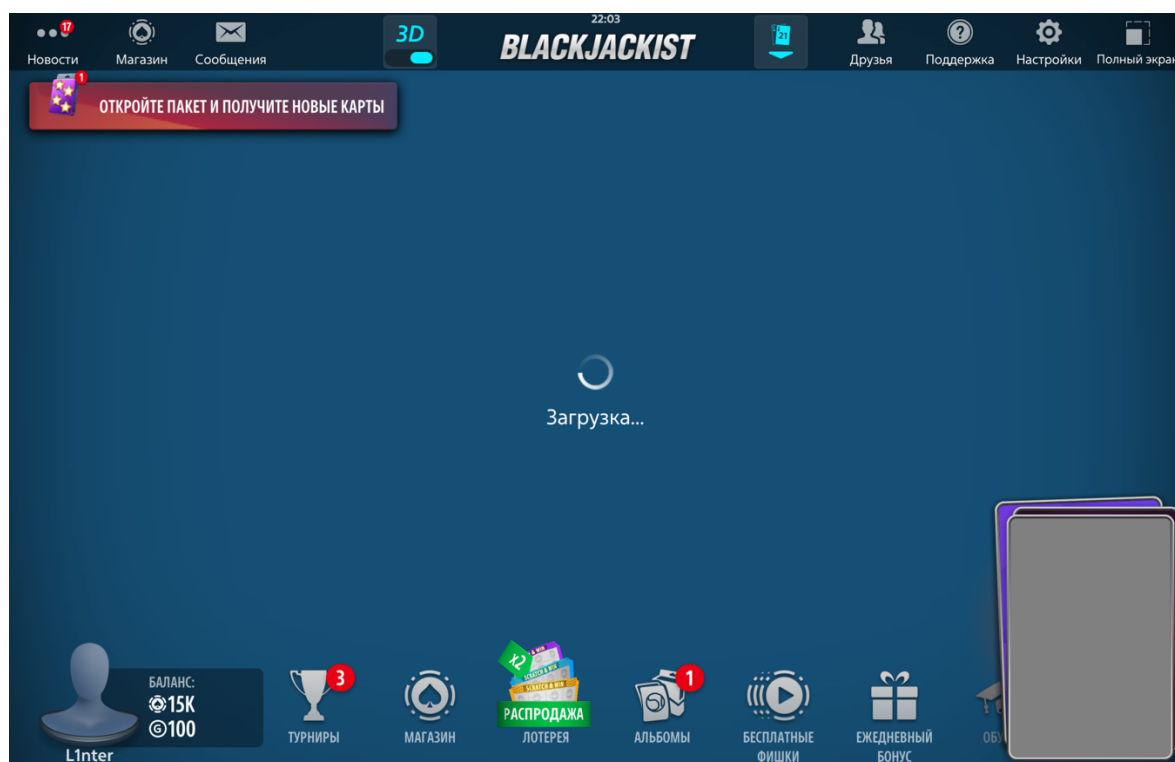


Рисунок 1.2 – *BlackJackist*

Таким образом, анализ существующих программных решений показывает, что на рынке преобладают либо тяжеловесные нативные клиенты, либо избыточно перегруженные коммерческие симуляторы. Разрабатываемое веб-приложение призвано занять нишу легковесных платформ, объединяющих строгую реализацию классических правил блекджека, возможность создания пользовательских комнат, низкую сетевую задержку и

абсолютную кроссплатформенность без необходимости установки стороннего программного обеспечения.

1.2 Постановка задачи

На основании аналитического обзора предметной области сформулировано техническое задание на разработку многопользовательского игрового веб-приложения blackjack. Система представляет собой клиент-серверный веб-комплекс с интерфейсом, доступным через современные интернет-браузеры без необходимости установки дополнительного программного обеспечения на стороне пользователя.

Функциональные требования к программному комплексу распределены по четырем основным логическим модулям. Модуль управления пользователями обеспечивает регистрацию новых учётных записей с обязательной валидацией уникальности псевдонима и адреса электронной почты, безопасное хранение паролей в виде хэш-значений [10], а также авторизацию пользователей. Для защиты от межсайтового скриптинга выдаваемые сессионные токены должны сохраняться исключительно в заголовках http-only cookie. Дополнительно данный модуль отвечает за обработку процедуры выхода из учетной записи с обязательным аннулированием активной сессии на стороне сервера.

Модуль управления игровыми комнатами позволяет авторизованным пользователям просматривать список активных столов и создавать новые комнаты с заданными параметрами, среди которых уникальное название, текстовое описание, ограничение максимального числа участников, количество колод в шузе, а также признак приватности и соответствующий пароль доступа. Кроме того, данный компонент организует вход пользователей в существующие комнаты, проверку доступов и корректную процедуру выхода из лобби с уведомлением других участников.

Модуль игрового движка отвечает за строгое соблюдение классических правил игры в блекджек и последовательное переключение игровых фаз, включая прием ставок, раздачу начальных карт, поочередное выполнение ходов пользователями, автоматический ход дилера до достижения семнадцати очков или выше, а также подведение итогов раунда с расчетом выплат. Движок реализует полную поддержку таких игровых действий, как взятие карты, фиксация текущего счета, удвоение ставки с получением одной карты, разделение пары одинаковых карт и отказ от игры с возвратом половины сделанной ставки. При расчете суммарного количества очков в руке алгоритм должен учитывать динамическое изменение ценности туза в зависимости от текущей вероятности перебора. Помимо перечисленных игровых действий, движок обязан корректно обрабатывать граничные ситуации: мгновенную победу при получении блекджека с первых двух карт, ничью при равенстве очков у игрока и дилера, а также автоматическое завершение хода в случае достижения счета в двадцать одно очко.

Модуль коммуникации дополняет игровой процесс и обеспечивает функционирование изолированного текстового чата внутри каждой игровой комнаты, позволяя участникам обмениваться сообщениями в режиме реального времени без сохранения истории переписки после закрытия лобби.

К разрабатываемому программному комплексу предъявляется ряд жестких нефункциональных и технических требований. В области производительности и латентности устанавливается предел времени на обработку сервером любого сетевого события и последующую трансляцию обновленного состояния всем клиентам в комнате, который не должен превышать 50 миллисекунд без учета задержек физических каналов связи. Столь жесткое ограничение обусловлено тем, что заметные задержки в передаче игровых событий нарушают синхронность восприятия игрового процесса участниками и негативно сказываются на общем качестве пользовательского опыта.

Консистентность данных достигается за счет транзакционного выполнения любого изменения состояния на стороне сервера с последующей централизованной рассылкой обновлений. Хранение и оперативное обновление промежуточных игровых состояний, текущих параметров комнат и активных сессий реализуется в оперативной базе данных redis. Долгосрочное хранение профилей пользователей, их учетных данных и основного финансового баланса организуется в реляционной базе данных postgresql. Подобное разделение между быстрым энергозависимым хранилищем и надежной реляционной базой данных позволяет одновременно обеспечить высокую скорость доступа к оперативным данным и гарантированную сохранность критически важной пользовательской информации.

Надёжность и отказоустойчивость системы гарантируют, что при кратковременном обрыве соединения длительностью до 10 секунд клиентский скрипт осуществит автоматическое переподключение к серверу, запросит актуальный снимок состояния игровой комнаты и полностью восстановит пользовательский интерфейс без потери данных. Механизм восстановления сессии реализуется путем хранения идентификатора комнаты и токена пользователя в защищенном локальном хранилище браузера, что позволяет клиенту однозначно идентифицировать себя при повторном подключении и получить от сервера полный снимок текущего состояния игры.

Требования к безопасности предписывают выполнение всей ключевой игровой логики, включая подсчет очков, генерацию и перемешивание колоды, принятие решений за дилера и валидацию ходов, исключительно на стороне сервера. Клиентская часть приложения выступает только в роли интерфейса визуализации и не имеет технической возможности напрямую влиять на внутренние переменные и состояние игрового движка. Такой подход исключает целый класс уязвимостей, связанных с подменой клиентских данных: любая попытка передать на сервер недопустимое игровое действие или некорректное значение ставки будет отклонена на этапе серверной валидации до применения каких-либо изменений к состоянию комнаты.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1 Проектирование структуры программы

Архитектура разрабатываемой программной системы построена на принципах модульного событийно-ориентированного проектирования и структурно разделена на два фундаментальных контура: серверный компонент в среде выполнения *node.js* и реактивный клиентский компонент, исполняемый в браузере. Подобное физическое и логическое разделение позволяет минимизировать сетевые издержки, обеспечить высокую масштабируемость и четко разграничить зоны ответственности между интерфейсом визуализации и защищенным ядром бизнес-логики.

Внутренняя структура серверного приложения организована в соответствии со слоистой архитектурной парадигмой. Первый базовый уровень представляет собой слой маршрутизации и обработки входящих *http*-запросов, реализованный на базе высокопроизводительного фреймворка *express* [2][3]. Он отвечает за первичную инициализацию веб-сервера, конфигурацию промежуточного программного обеспечения для безопасного разбора сессионных *cookie* и *json*-пакетов. Кроме того, этот уровень обслуживает маршруты авторизации и осуществляет динамический рендеринг веб-страниц с использованием шаблонизатора *handlebars* [11].

Второй уровень представлен слоем высокоскоростного двунаправленного взаимодействия, функционирующим под управлением библиотеки *socket.io* поверх стандарта *websocket*[9]. Данный компонент централизованно управляет жизненным циклом постоянных сетевых соединений. На этапе авторизационного рукопожатия система извлекает сессионные токены, верифицирует их и привязывает сокеты к профилям игроков. Далее этот слой распределяет активные подключения по изолированным виртуальным комнатам, организуя мгновенную широковеб-трансляцию асинхронных игровых событий всем участникам конкретного стола.

Центральное место в архитектуре занимает слой изолированной бизнес-логики, вычислительным ядром которого служит специализированный математический движок карточной игры. Этот модуль инкапсулирует в себе защищенные алгоритмы генерации и тасования виртуальных колод, математические функции вычисления достоинства комбинаций с учетом переменной ценности тузов, а также строгий конечный автомат состояний комнаты. Конечный автомат гарантирует безопасную и детерминированную смену игровых фаз: прием начальных ставок, раздачу карт, обработку пользовательских ходов, автоматические действия дилера и финальный расчет выплат.

За надежное сохранение информации и бесперебойное управление контекстом отвечает слой доступа к данным, функционально разделенный на два независимых репозитория. Долгосрочное персистентное хранение учетных записей, хэшированных паролей и актуальных финансовых балансов

пользователей делегировано реляционной базе данных *postgresql* [7]. Параллельно с этим хранение высокодинамичного состояния активных столов и промежуточных игровых решений перенесено в *in-memory* хранилище *redis* [8], что позволяет полностью исключить ресурсоемкие дисковые операции в процессе активной игры.

Клиентская часть программного комплекса спроектирована как автономное реактивное приложение, управляемое входящим потоком сетевых данных. Ее внутренние модули отвечают за поддержание стабильного соединения, автоматическое переподключение и обработку серверных событий. Специализированные контроллеры осуществляют плавную мутацию *dom*-дерева веб-страницы для визуализации раздачи карт, обновления балансов и вывода итогов раунда. Синхронно с этим клиентские обработчики перехватывают управляющие действия пользователя, трансформируя выбор фишек, игровые ходы и сообщения чата в строго типизированные запросы.

Глобальное взаимодействие всех компонентов организуется посредством параллельного использования двух независимых каналов коммуникации. Первичный *http*-протокол применяется исключительно для загрузки базового интерфейса и безопасного проведения процедуры авторизации. Вторичный канал, опирающийся на постоянные сетевые сокеты, монопольно обслуживает непрерывный игровой процесс. В этой структуре серверная часть выступает в роли единого координатора, который верифицирует поступающие команды, фиксирует изменения в оперативной памяти и производит итоговую транзакционную запись результатов в дисковое хранилище, гарантируя абсолютную консистентность всей системы.

2.2 Проектирование системы сетевого взаимодействия

Сетевое взаимодействие является наиболее критичным узлом системы реального времени. Традиционная модель *http*, основанная на парадигме одиночных запросов и ответов, непригодна для фазы активного игрового процесса, поскольку сервер не обладает механизмом самостоятельной инициации отправки данных клиенту в момент, когда другой участник берет карту или дилер завершает раунд. Для решения этой проблемы спроектирована событийно-ориентированная архитектура на базе протокола *websocket* [9] с использованием библиотеки *socket.io* [5].

Взаимодействие начинается с этапа установления связи, также известного как *handshake*. При переходе пользователя на страницу игровой комнаты клиентская библиотека *socket.io* отправляет первичный *http*-запрос, содержащий заголовок *upgrade: websocket*. Сервер извлекает из него сессионный токен, выполняет его валидацию через связку хранилищ *redis* и *postgresql* и, в случае успеха, подтверждает переключение протокола. Установленной сессии сокета присваивается уникальный идентификатор, записываемый в свойство *socket.id*, а верифицированные данные пользователя надежно привязываются к объекту *socket.data* для быстрого доступа при последующих операциях.

Исходящий поток данных от клиента к серверу строго регламентирован и обрабатывается через фиксированный набор событий (*client-to-server events*). Процесс подключения инициируется событием *room_join*, в рамках которого передается параметр *roomid*, а для принудительной синхронизации интерфейса предусмотрен запрос *get_room_state*, возвращающий актуальный снимок стола. Непосредственное управление игровым процессом осуществляется посредством трансляции команд: *place_bet* для фиксации ставки в соответствующей фазе, *hit* для запроса дополнительной карты и *stand* для завершения набора карт текущей рукой. Кроме того, реализована поддержка дополнительных стратегических действий, таких как событие *double* для удвоения ставки с автоматическим получением ровно одной карты, *split* для разделения парных карт на две независимые руки и *surrender* для добровольной сдачи с возвратом пятидесяти процентов от суммы ставки. Отправка текстовых сообщений во внутриигровой чат обслуживается отдельным событием *chat_message*.

Входящий поток данных от сервера к клиенту (*server-to-client events*) инкапсулирует команды асинхронного управления интерфейсом. Сервер транслирует событие *room_state*, содержащее полный *json*-снимок стола при первичном подключении, после чего рассылает сигнал *game_started*, информирующий о начале нового раунда и старте раздачи карт. В процессе активной игры клиенты получают атомарные обновления через событие *player_action*, фиксирующее изменение состояния руки после хода оппонента, а также *turn_changed* для уведомления о переходе права действия к следующему участнику. Поэтапная трансляция процесса набора карт дилером осуществляется через событие *dealer_play*. По завершении игрового цикла сервер формирует детализированный отчет *round_result* для всех участников и инициирует событие *betting_phase* для перевода комнаты обратно в стадию сбора ставок. Вспомогательный функционал протокола обеспечивает ретрансляцию текстовых данных через событие *chat_message*, рассылку сервисных уведомлений *player_joined* и *player_left* о перемещениях участников, а также отправку события *error*, содержащего код и описание ошибки при попытках совершения недопустимых действий.

Спроектированный протокол сводит к минимуму объем передаваемых данных, транслируя преимущественно изолированные дельта-обновления состояния вместо пересылки всего контекста комнаты. Это гарантирует высокую отзывчивость пользовательского интерфейса и бесперебойность игрового процесса даже при значительных колебаниях качества интернет-соединения.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Файловая структура проекта представлена на рисунке 3.1.

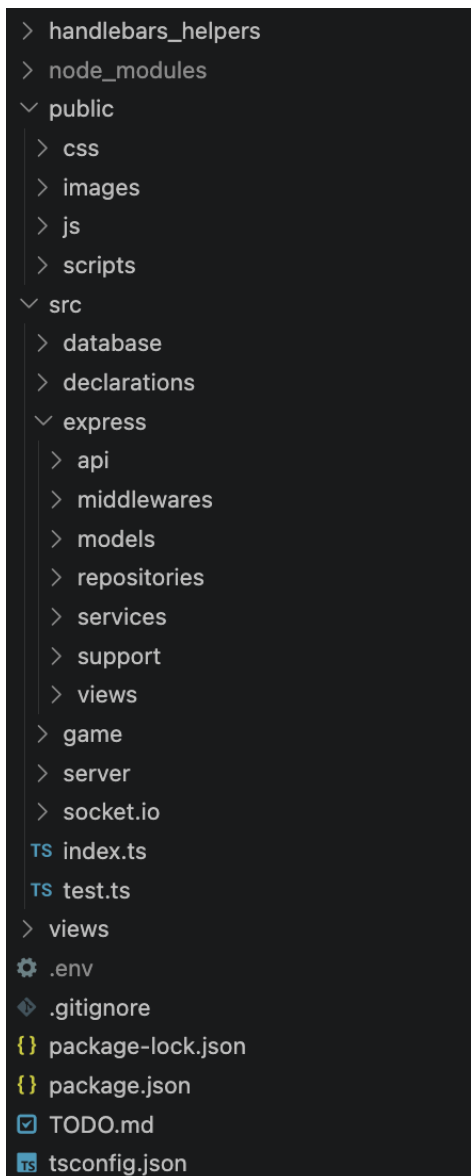


Рисунок 3.1 – Файловая структура проекта

3.1 Описание разработанных модулей

Разработанное веб-приложение построено на основе модульной архитектуры, обеспечивающей строгое разделение ответственности между компонентами клиентской и серверной частей. Программное средство состоит из нескольких взаимосвязанных логических блоков, каждый из которых инкапсулирует определенный набор функций. Подобная организация исходного кода упрощает отладку, профилирование и последующее масштабирование системы, позволяя модифицировать отдельные подсистемы без риска нарушения целостности всего комплекса.

Модуль доступа к данным отвечает за операции взаимодействия с базами данных и предоставляет унифицированную абстракцию над различными механизмами хранения информации. Серверная часть комплекса использует гибридную инфраструктуру. Долгосрочное и надежное хранение реализовано через специализированный репозиторий на базе реляционной субд *postgresql*, который управляет учетными записями, хэшированными паролями и актуальными финансовыми балансами игроков. Оперативное хранение возложено на репозитории резидентной системы *redis*. Модуль управления сессиями отвечает за кэширование авторизационных токенов, формируемых криптографической утилитой генерации, а компонент управления комнатами контролирует метаданные активных столов и списки участников, обеспечивая сверхбыстрый доступ к изменяемому контексту без задержек дискового ввода-вывода.

Модуль бизнес-логики инкапсулирует ключевую функциональность приложения и содержит алгоритмы, реализующие строгие математические правила карточной игры. Центральным элементом этого слоя выступает программный движок блекджека, полностью абстрагированный от сетевых протоколов. В его задачи входит генерация стандартизированного карточного шуза заданного объема, криптографически надежное перемешивание карт на основе алгоритма фишера-йейтса [12] и функция оценки текущего состояния рук. Алгоритм подсчета очков производит динамическое вычисление номиналов, автоматически понижая ценность тузов с одиннадцати до одного очка при угрозе перебора. Движок непрерывно управляет конечным автоматом состояний виртуального стола, регламентируя переходы между фазами приема ставок и активной игры, а также выполняет итоговый расчет выигрышей с применением классических коэффициентов выплат и синхронизирует результаты с модулем хранения.

Модуль сетевого взаимодействия и управления лобби координирует распределение участников и гарантирует мгновенную доставку игровых событий. Он объединяет клиентские скрипты управления интерфейсом лобби и серверные контроллеры, построенные поверх библиотеки *socket.io*. Данный компонент позволяет пользователям запрашивать актуальный список доступных столов, создавать новые приватные или публичные комнаты с индивидуальными лимитами числа участников и количества колод, а также безопасно подключаться к открытым партиям посредством верификации паролей. Для обеспечения безопасности этот же модуль реализует механизм защиты от множественных подключений: при входе пользователя в систему с нового устройства инициируется принудительное отключение старых сессий с автоматическим удалением устаревших сокетов из активных игровых комнат.

Модуль маршрутизации и пользовательского интерфейса обеспечивает визуальное представление информации и первичную обработку *http*-запросов. На стороне сервера маршрутизация выстроена с использованием веб-фреймворка *express* [3], обслуживающего *rest api* конечные точки авторизации, регистрации и управления профилем. Динамическое

формирование страниц реализовано посредством шаблонизатора *handlebars*. Специализированный программный обработчик контекста анализирует входящие запросы на предмет наличия валидного сессионного токена в файлах *cookie* и выполняет условный рендеринг. При успешной проверке пользователю отображаются защищенные элементы навигации и открывается доступ к игровому лобби, тогда как неавторизованным клиентам предоставляются исключительно формы базового доступа. На стороне браузера реактивные скрипты перехватывают ответы сервера и осуществляют динамическую перестройку структуры объектной модели документа без необходимости полной перезагрузки веб-страницы.

3.2 Описание сетевого модуля

Сетевой модуль выступает центральным коммуникационным ядром разрабатываемого программного комплекса, обеспечивая полнодуплексный обмен данными в реальном времени. Его логика физически разделена на два компонента: серверные контроллеры, реализованные в файле *socket.io.ts*, и клиентский реактивный скрипт *game.js*. Процесс взаимодействия начинается с инициализации соединения на стороне браузера, где вызов функции подключения осуществляется с явным указанием параметра *transports: ["websocket"]*. Это принудительно устанавливает стандарт *websocket* в качестве единственного транспортного механизма, что позволяет полностью исключить избыточные резервные *http*-опросы и минимизировать сетевые задержки при передаче игровых пакетов.

Архитектура клиентского приложения выстроена на реактивной обработке входящих событий от сервера. При первичном подключении или принудительном запросе синхронизации клиент получает событие *room_state*, содержащее сериализованный снимок текущего стола. Обработчик этого события кэширует информацию в локальную переменную и запускает метод комплексного рендеринга. Данный алгоритм анализирует текущую фазу игры и при необходимости скрывает вторую карту дилера, заменяя её псевдокодом, после чего последовательно перестраивает структуру документа. Происходит отрисовка карт дилера и всех участников, обновление счетчика оставшихся в колоде карт и синхронизация финансовых балансов пользователей в боковой панели интерфейса.

Динамика активного игрового процесса обеспечивается обработкой атомарных событий. Сигнал *player_action* транслируется сервером после каждого успешного хода любого из участников. Пакет данных содержит идентификатор игрока, обновленный массив его раздач и индекс активной руки, что позволяет клиентскому скрипту выполнять точечный рендеринг конкретного слота без полной перерисовки стола. Активная рука визуально выделяется стилистическим классом, а текстовые блоки отображают текущую сумму очков и статус раздачи. Передача права хода обслуживается событием *turn_changed*, которое обновляет глобальный идентификатор активного участника и вызывает функцию переоценки доступности элементов

управления. Метод строго проверяет контекст стола: кнопки специфических действий, таких как удвоение ставки или разделение парных карт, активируются исключительно при соблюдении правил блекджека (наличие ровно двух карт, совпадение рангов при сплите) и совпадении идентификатора ходящего с локальным пользователем.

Завершение раунда координируется через обработку события `round_result`. При его получении клиентская часть блокирует интерфейс управления ходами, полностью раскрывает закрытые карты дилера и формирует детализированную сводку в модальном окне. Алгоритм дифференцирует исходы раунда (натуральный блекджек, победа, ничья или поражение), подставляя соответствующие визуальные индикаторы, и применяет итоговые изменения к балансам игроков на столе. Окно результатов демонстрируется пользователям в течение шести секунд, после чего автоматически скрывается системным таймером, подготавливая интерфейс к новой фазе сбора ставок. Параллельно с визуальным отображением итогов клиентский скрипт направляет на сервер подтверждение получения результатов, что позволяет серверной части точно отслеживать готовность всех участников и своевременно инициировать переход к следующему раунду. В случае если баланс игрока по итогам раунда опускается ниже минимально допустимого порога ставки, интерфейс дополнительно отображает соответствующее уведомление и блокирует возможность участия в следующем раунде до пополнения счета.

Для корректного визуального представления раздач в модуле реализована вспомогательная утилита форматирования. Функция перехватывает строковые серверные идентификаторы карт (например, «ah» или «td») и преобразует их в html-разметку с использованием соответствующих юникод-символов мастей. При этом для червовых и бубновых карт алгоритм динамически присваивает инлайн-класс `card-red`, окрашивающий текст в красный цвет, что обеспечивает интуитивно понятное и аутентичное восприятие виртуальной колоды. Подобный подход к кодированию карт позволяет полностью абстрагировать визуальный слой от внутреннего представления данных: серверу достаточно передавать компактные двухсимвольные идентификаторы, тогда как вся логика преобразования в читаемую разметку сосредоточена на клиентской стороне. Это существенно сокращает объем передаваемых по сети данных и упрощает поддержку кодовой базы, поскольку любые изменения в визуальном оформлении карт не затрагивают серверную логику. Помимо форматирования мастей, утилита также обрабатывает специальные случаи отображения: закрытые карты дилера рендерятся с использованием отдельного `placeholder`-элемента, визуально обозначающего рубашку, что сохраняет целостность игровой механики сокрытия информации на уровне интерфейса.

4 ТЕСТИРОВАНИЕ

Тестирование разработанного многопользовательского веб-приложения проводилось с целью комплексного подтверждения корректности функционирования серверной бизнес-логики, строгого соблюдения математических правил карточной дисциплины, проверки стабильности двунаправленного сетевого взаимодействия и оценки устойчивости платформы к попыткам несанкционированной манипуляции данными со стороны клиентского интерфейса.

В рамках проверки модуля управления пользователями оценивалась корректность обработки запросов на регистрацию и авторизацию. При передаче валидных учетных данных сервер успешно осуществляет криптографическое хэширование пароля, создает новую запись в реляционной базе данных postgresql и возвращает http-статус успешной обработки вместе с уникальным сессионным токеном, который помещается в защищенные файлы cookie. В случае попытки регистрации с использованием уже существующего в системе псевдонима или адреса электронной почты контроллер корректно прерывает транзакцию, возвращая статус ошибки с соответствующим информационным сообщением, что исключает дублирование профилей.

Тестирование подсистемы игровых комнат подтвердило правильность алгоритмов распределения игроков. При создании приватного стола с заданными параметрами, такими как пользовательское название, пароль, количество колод и ограничение на число участников, система безошибочно формирует сериализованные структуры в кэш-памяти redis и возвращает создателю уникальный строковый идентификатор комнаты. Попытки несанкционированного входа в закрытую комнату с передачей некорректного пароля через api успешно блокируются на этапе проверки метаданных стола, после чего клиенту отказывается в доступе без инициализации сетевого сокета.

Основополагающим этапом стала верификация изолированного игрового движка, которая включала как оценку пользовательских сценариев, так и модульное тестирование чистых математических функций на базе встроенных средств среды node.js. Особое внимание уделялось алгоритму динамического подсчета очков. Система безошибочно обрабатывает ситуации с «мягкой» рукой: например, при стартовой раздаче туза и пятерки суммарный счет корректно фиксируется как шестнадцать очков. При последующем запросе дополнительной карты, приводящем к потенциальному перебору, алгоритм автоматически трансформирует ценность туза из одиннадцати очков в одно, сохраняя статус руки активным. В ситуациях, когда добор карты приводит к неизбежному превышению лимита в двадцать одно очко, конечный автомат мгновенно переводит руку в статус перебора и принудительно передает право хода следующему участнику.

Проверка дополнительных игровых стратегий показала полное соответствие классическим правилам. При активации удвоения ставки сервер валидирует наличие достаточного баланса в оперативной сессии игрока,

корректно списывает требуемую сумму, выдает строго одну дополнительную карту и завершает набор для текущей руки. Логика контроля очередности ходов исключает возможность возникновения состояния гонки: любые попытки клиента отправить команду на взятие карты или остановку вне своей очереди немедленно отклоняются сервером с отправкой ответного события об ошибке, так как идентификатор ходящего жестко зафиксирован в состоянии виртуальной комнаты. Проверка автоматического алгоритма дилера подтвердила, что виртуальный крупье последовательно добирает карты из колоды вплоть до достижения суммы в семнадцать или более очков, после чего гарантированно останавливается для финального сравнения результатов.

Помимо функциональных проверок, проведено тестирование механизмов обработки внештатных ситуаций, в частности — устойчивости системы к внезапным обрывам сетевого соединения. При симуляции потери сети клиентом серверный модуль корректно перехватывал событие отключения. Система автоматически завершала ход отключившегося пользователя, передавая право действия следующему участнику, что полностью исключало блокировку игрового процесса. Дополнительно тесты подтвердили консистентность данных: итоговые изменения балансов, рассчитанные в оперативном кэше Redis, безошибочно синхронизировались с реляционным хранилищем PostgreSQL, гарантируя сохранность финансовых показателей при любых сценариях прерывания сессии.

Завершающим этапом стала проверка безопасности программного комплекса. Целенаправленные попытки модификации локальных переменных клиентского интерфейса, таких как текущий финансовый баланс или право хода, с использованием инструментов разработчика в браузере приводили исключительно к визуальным искажениям на устройстве злоумышленника. При попытке совершить фактическое игровое действие с подмененными параметрами серверная часть немедленно пресекала транзакцию, опираясь исключительно на криптографически защищенный токен сессии и доверенное состояние стола в оперативной памяти сервера. Дополнительно была проверена устойчивость системы к повторному воспроизведению перехваченных сетевых пакетов: попытка переотправки ранее записанного события игрового действия отклонялась сервером ввиду несоответствия текущей фазы игры и версии состояния комнаты, зафиксированной в момент первоначальной отправки. Совокупность проведенных испытаний подтверждает, что разработанная архитектура с централизованным хранением доверенного состояния на стороне сервера и строгой валидацией каждого входящего события обеспечивает высокую степень защищенности игровой платформы от широкого спектра клиентских атак и непреднамеренных ошибок пользовательского окружения.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Настоящее руководство предназначено для ознакомления пользователей с графическим интерфейсом, логикой навигации и механизмами обратной связи многопользовательского игрового веб-приложения. Взаимодействие с программным комплексом начинается с запуска браузера и перехода на главную страницу системы.

При первичном посещении ресурса пользователю отображается базовое состояние главного меню. Интерфейс навигационной панели динамически адаптируется под текущий статус сессии. Для посетителей, не прошедших процедуру идентификации, главное меню содержит исключительно навигационные ссылки для перехода к формам регистрации и авторизации, в то время как доступ к игровому функционалу лобби остается программно скрытым.

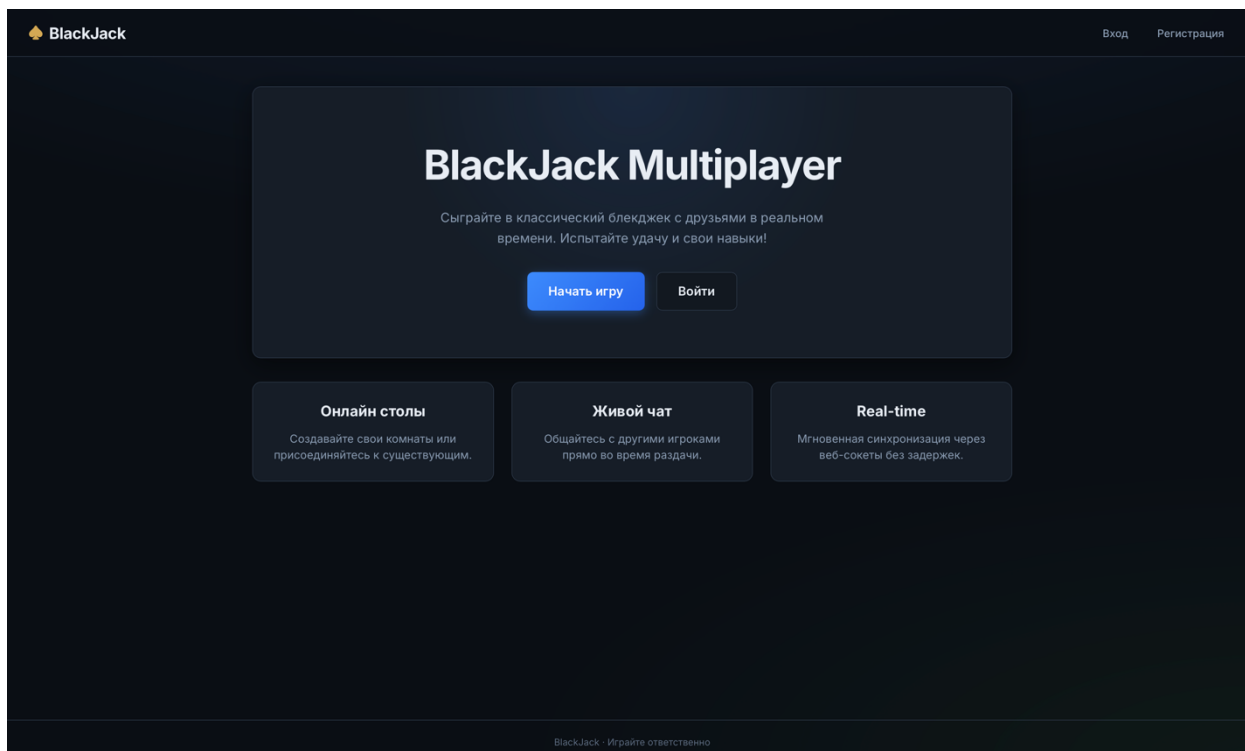


Рисунок 5.1 – Интерфейс главного меню для неавторизованного пользователя

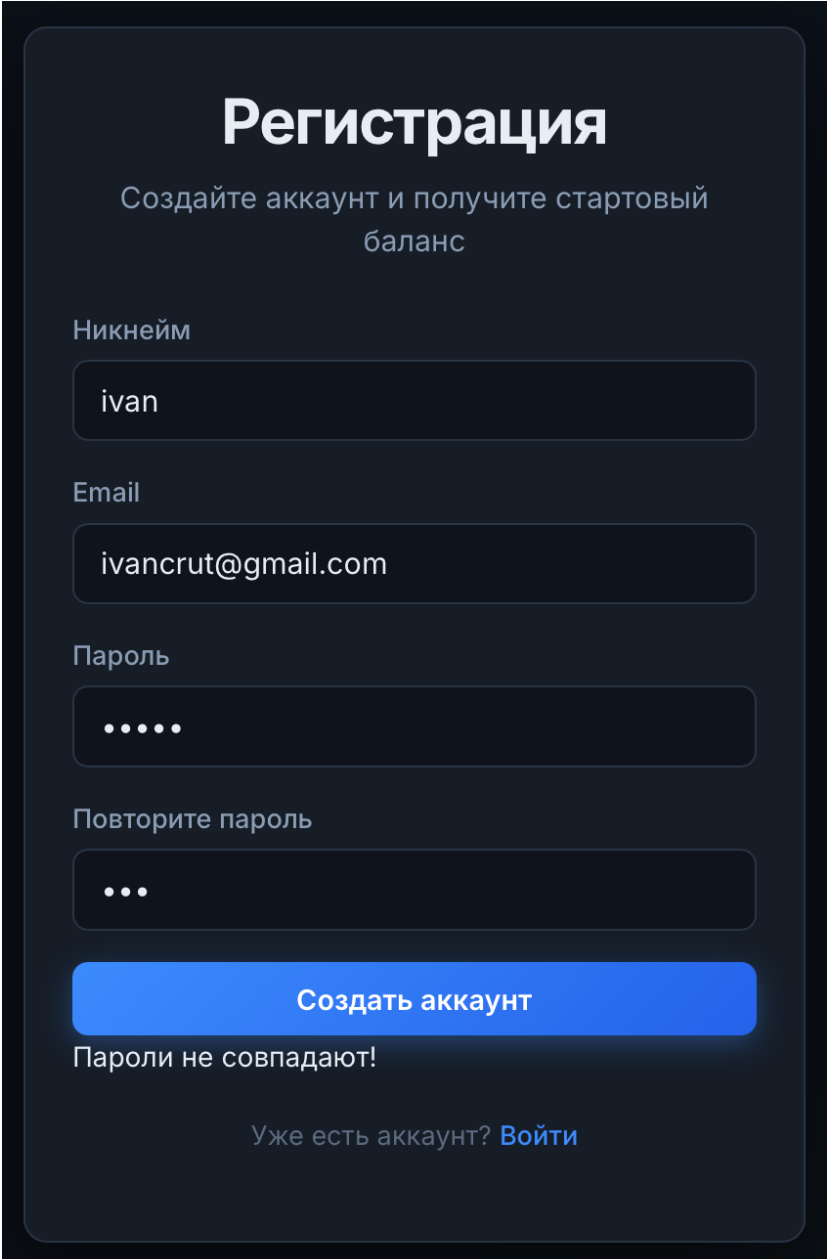
```
GET / HTTP/1.1
HTTP/1.1 302 Found (text/html)
GET /main HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.2 – Запрос на получение основной страницы

Для создания новой учетной записи необходимо заполнить форму регистрации, указав уникальный псевдоним, адрес электронной почты и дважды введя пароль. При заполнении формы система обеспечивает моментальную обратную связь. Если пользователь вводит несовпадающие пароли, локальный скрипт немедленно выводит текстовое предупреждение под полями ввода, предотвращая отправку данных.

```
GET /authorization/register HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.3 – Запрос на получение меню регистрации



The image shows a registration form titled "Регистрация" (Registration) with the subtitle "Создайте аккаунт и получите стартовый баланс" (Create an account and get a starting balance). The form contains four input fields: "Никнейм" (Nickname) with the value "ivan", "Email" with the value "ivancrut@gmail.com", "Пароль" (Password) with five dots, and "Повторите пароль" (Repeat password) with three dots. Below the password fields is a blue button labeled "Создать аккаунт" (Create account). Underneath the button, the text "Пароли не совпадают!" (Passwords do not match!) is displayed. At the bottom, there is a link that says "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рисунок 5.4 – Индикация ошибки несовпадения паролей на форме регистрации

В случае возникновения исключений на стороне сервера, таких как попытка регистрации уже существующего имени пользователя, система выводит соответствующее сообщение об ошибке внутри самой формы, сохраняя корректно заполненные поля для быстрого исправления.

Рисунок 5.5 – Индикация серверной ошибки на форме регистрации

Доступ к существующему профилю осуществляется через форму авторизации. При вводе неверного пароля или несуществующего логина система аналогичным образом отображает локальное предупреждение об ошибке валидации, не требуя полной перезагрузки страницы.

```
GET /authorization/login HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.6 – Запрос на получение меню входа

Рисунок 5.7 – Индикация ошибки валидации на форме авторизации

```
POST /api/authorization/login HTTP/1.1 , JSON (application/json)
HTTP/1.1 401 Unauthorized , JSON (application/json)
```

Рисунок 5.8 – Попытка зайти с невалидными данными

```
Member: identifier
  [Path with value: /identifier:ivan]
  [Member with value: identifier:ivan]
  String value: ivan
  Key: identifier
  [Path: /identifier]
Member: password
  [Path with value: /password:1234]
  [Member with value: password:1234]
  String value: 1234
  Key: password
  [Path: /password]
```

Рисунок 5.9 – Содержание тела запроса

```
POST /api/authorization/login HTTP/1.1 , JSON (application/json)
HTTP/1.1 202 Accepted , JSON (application/json)
GET /setcookie?sessionToken=95a6c1a65d36ee962eb48a18dd3e9674 HTTP/1.1
HTTP/1.1 302 Found (text/html)
GET /main HTTP/1.1
```

Рисунок 5.10 – Следующие запросы при валидных данных

В случае, когда пользователь вводит валидные данные, в ответ он получает сообщение с выделенным ему токеном. После чего, он перенаправляется на специально выделенный адрес, который нужен для установки токена в куки. После чего пользователь перенаправляется на основную страницу.

После успешного входа в систему навигационная панель автоматически перестраивается. Гостевые элементы скрываются, и пользователю становятся доступны основная ссылка для перехода в игровое лобби, персональный профиль, а также кнопка для безопасного завершения сеанса и выхода из учетной записи.

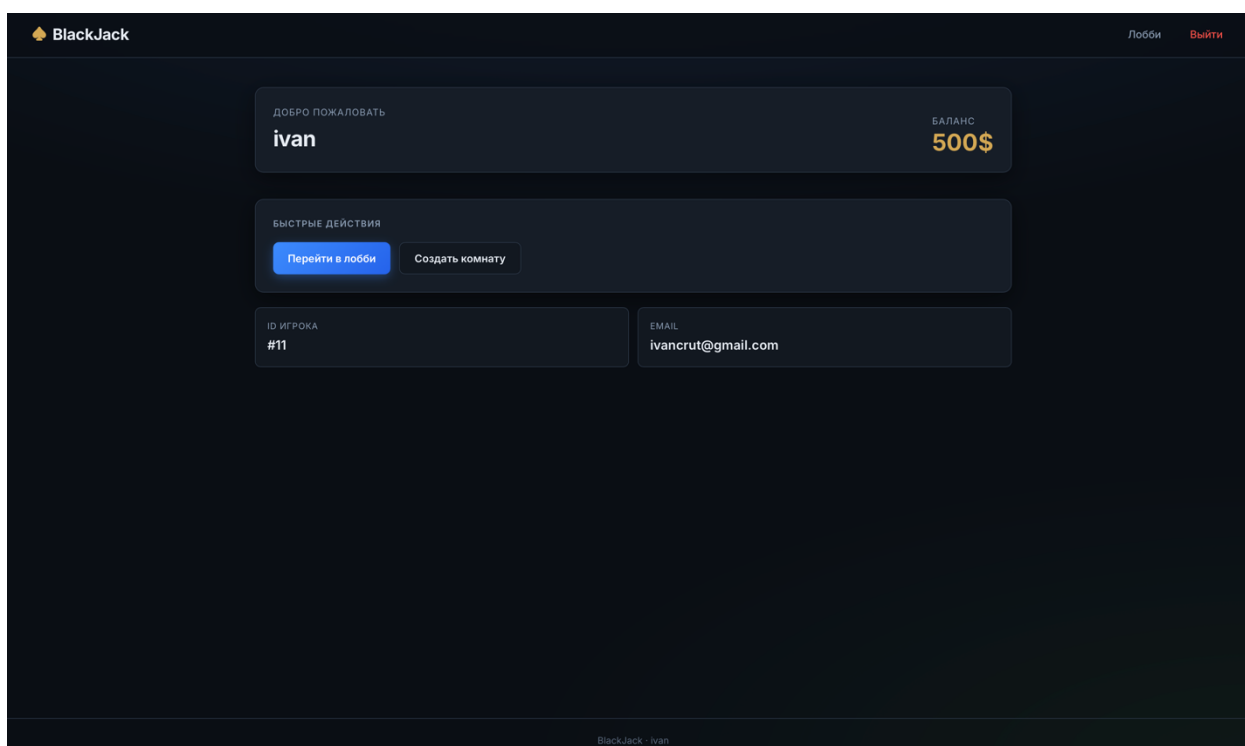


Рисунок 5.11 – Интерфейс главного меню для авторизованного пользователя

Для защиты от случайных действий в систему внедрен механизм обязательного подтверждения. При попытке завершить сеанс через главное меню браузер генерирует диалоговое окно с запросом подтверждения выхода из аккаунта.

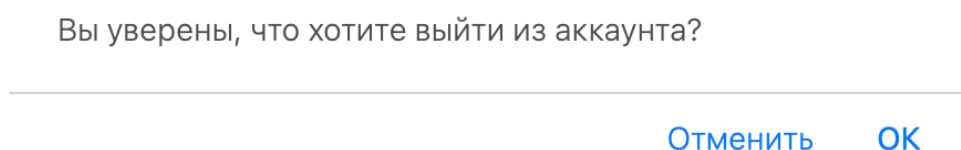


Рисунок 5.12 – Диалоговое окно подтверждения выхода из аккаунта

Переход в раздел лобби открывает перед пользователем интерфейс просмотра и поиска доступных игровых столов. На экране отображается список комнат с указанием их названия, описания, текущего количества игроков и количества используемых колод.

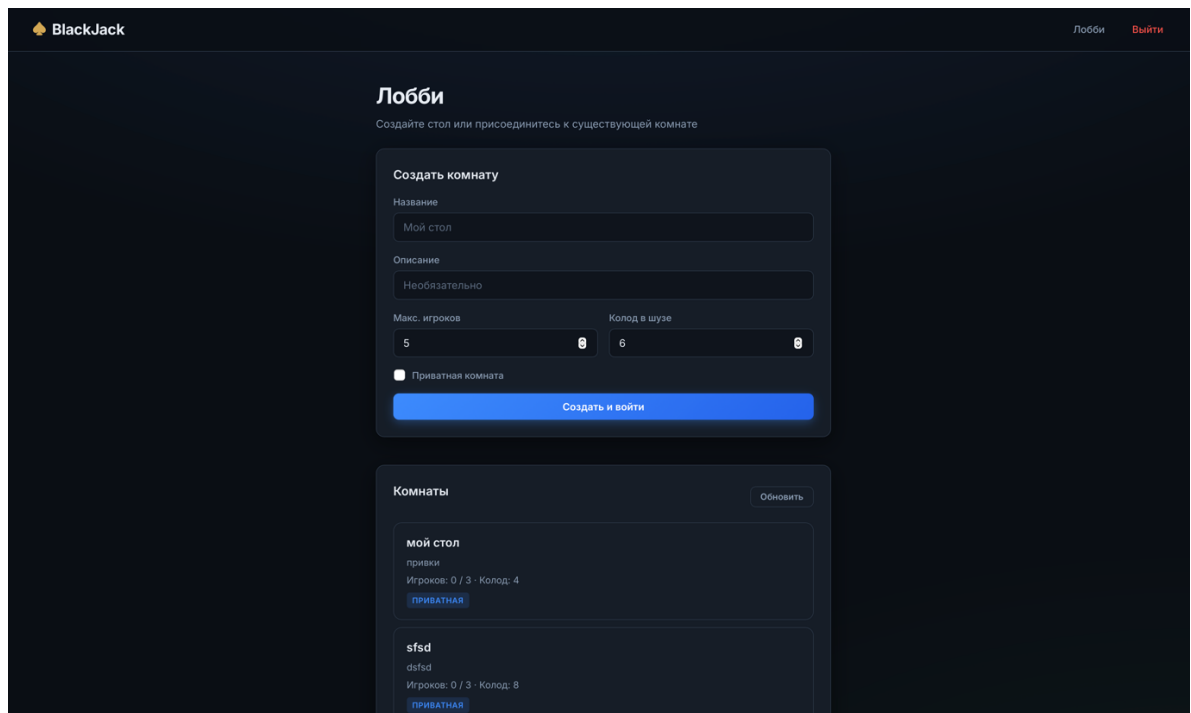


Рисунок 5.13 – Интерфейс игрового лобби со списком доступных комнат

```
GET /lobby/rooms HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.14 – Запрос на получение меню комнат

После загрузки всех скриптов страницы, автоматически отправляется запрос на получение всех активных комнат.

```
GET /api/lobby/rooms HTTP/1.1
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.15 – Дополнительный запрос на получение всех активных комнат

В интерфейсе лобби также присутствует панель конфигурации нового стола. При указании недопустимых параметров в форме создания комнаты выводится текстовая плашка с описанием ошибки, информируя пользователя о необходимости скорректировать лимиты или название.

The screenshot shows a web form titled "Создать комнату" (Create Room). It has two text input fields: "Название" (Name) with the value "qwe" and "Описание" (Description) with the value "qwe". Below these is a numeric input field for "Макс. игроков" (Max. players) with the value "100", which has a red error message above it: "Значение должно быть меньше или равно 7". Next to it is a numeric input field for "Колод в шузе" (Decks in shuff) with the value "6". There is a checkbox labeled "Приватная комната" (Private room) which is currently unchecked. At the bottom is a large blue button labeled "Создать и войти" (Create and login).

Рисунок 5.16 – Индикация ошибки при создании новой игровой комнаты

Вход в существующие приватные комнаты защищен паролем. При попытке подключения к такому столу система вызывает всплывающее диалоговое окно для ввода секретной комбинации символов, предотвращая несанкционированный доступ посторонних участников.

```
POST /api/lobby/rooms HTTP/1.1 , JSON (application/json)
HTTP/1.1 201 Created , JSON (application/json)
```

Рисунок 5.17 – Запрос на создание комнаты

```
JavaScript Object Notation: application/json
▼ Object
  > Member: name
  > Member: description
  > Member: maxPlayersCount
  > Member: deckCount
  > Member: isPrivate
  > Member: password
```

Рисунок 5.18 – Тело запроса на создание комнаты

```
JavaScript Object Notation: application/json
▼ Object
  > Member: message
  > Member: roomId
```

Рисунок 5.19 – Тело ответа с айди созданной комнаты

В теле запроса на создание комнаты находятся все данные, введенные пользователем для создания конкретной комнаты. В ответ на что пользователь получает ответ, содержащий айди созданной комнаты. После чего, пользователь перенаправляется на её страницу.

```
GET /lobby/room/d29d9807-ef30-47c8-bb53-01c92cb3315a HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.20 – запрос на получение страницы созданной комнаты

После успешного получения страницы комнаты, устанавливается WebSocket соединение с применением socket.io.

```
GET /socket.io/?EI0=4&transport=websocket HTTP/1.1
HTTP/1.1 101 Switching Protocols
```

Рисунок 5.21 – запрос на создание комнаты

Введите пароль комнаты:

Отменить **OK**

Рисунок 5.22 – Всплывающее окно для ввода пароля от приватной комнаты

Интерфейс самой игровой комнаты разделен на зону дилера, где отображаются карты и текущий счет, индивидуальные слоты игроков с их балансом и активными руками, а также панель текстового чата для общения участников.

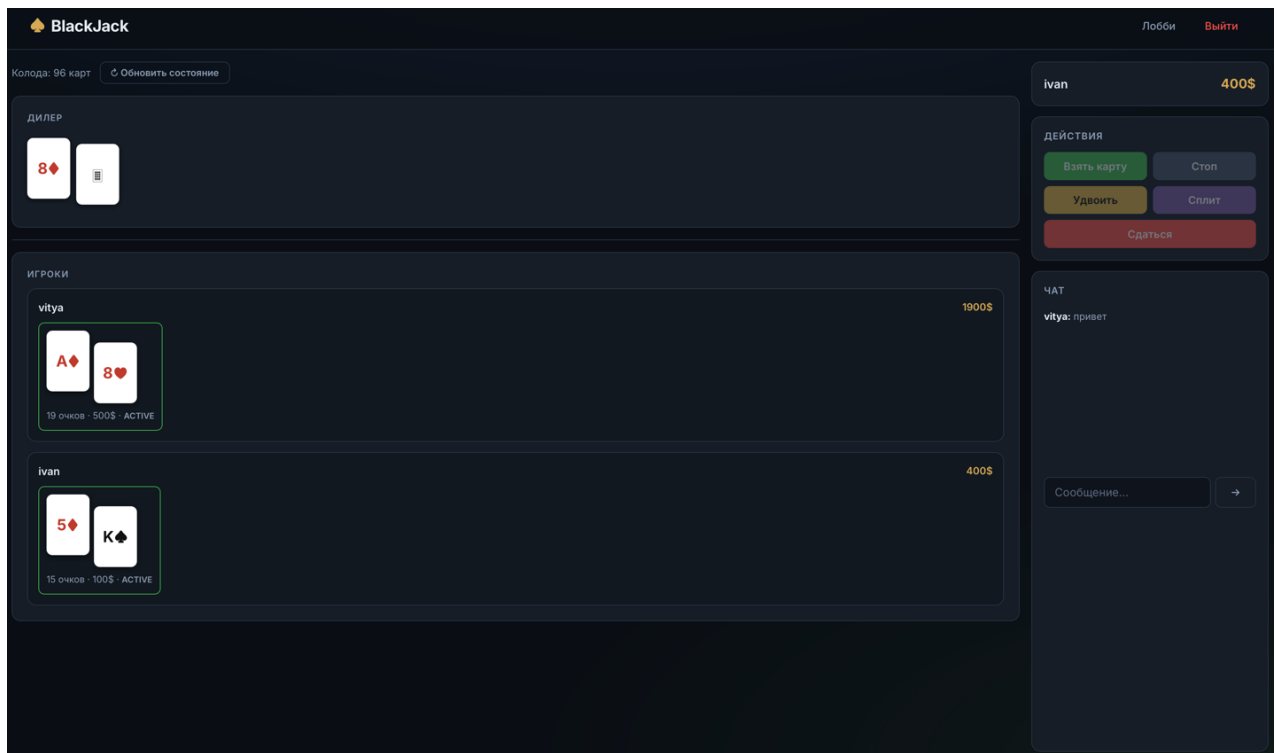


Рисунок 5.23 – Общий интерфейс активной игровой комнаты

Во время фазы приема ставок пользователю становится доступна специальная панель взаимодействия. Игрок может использовать интерактивные фишки для быстрого ввода суммы или вписать значение вручную в соответствующее поле.

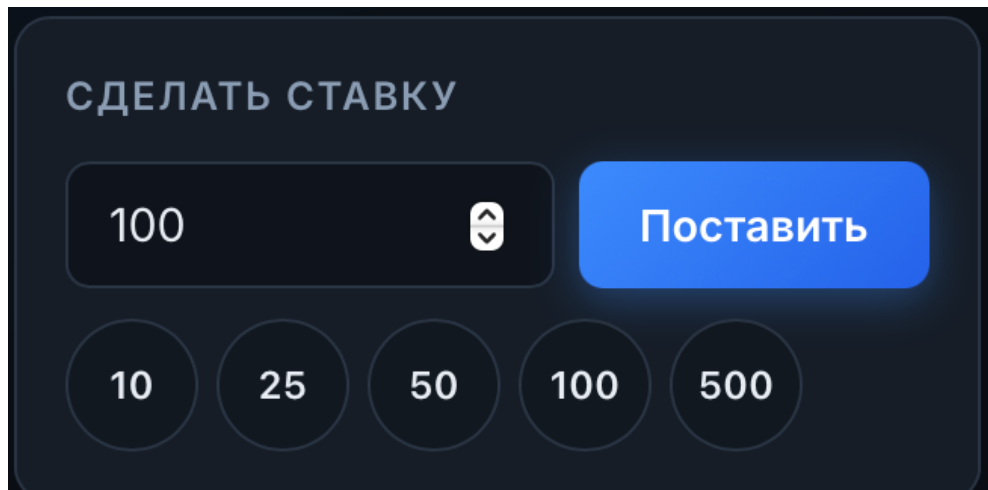
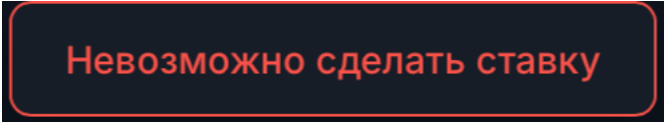


Рисунок 5.24 – Панель интерфейса для выбора и подтверждения ставки

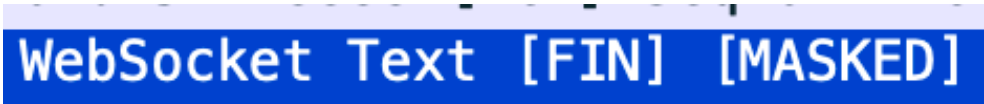
Система строго контролирует процесс совершения ставок. Если пользователь указывает нулевую или отрицательную сумму, на экран высвечивается красная плашка о невозможности совершить ставку.



Невозможно сделать ставку

Рисунок 5.25 – Предупреждающее уведомление о некорректной сумме ставки

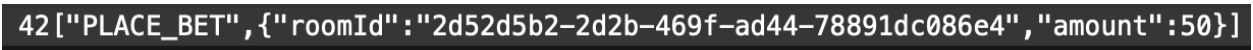
Теперь, после подключение к вебсокету, внутри комнаты клиент и сервер начинают обмениваться сообщениями на установленном соединении. Выглядит это следующим образом:



WebSocket Text [FIN] [MASKED]

Рисунок 5.26 – WebSocket сообщение

Содержание таких сообщений всегда похожее, а именно объект с названием совершаемого события и само содержание в формате JSON.

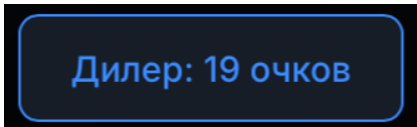


```
42 ["PLACE_BET", {"roomId": "2d52d5b2-2d2b-469f-ad44-78891dc086e4", "amount": 50}]
```

Рисунок 5.27 – Содержание сообщений

Для обеспечения непрерывной обратной связи в процессе активной партии реализован механизм динамических всплывающих уведомлений. Информационные плашки временно появляются в боковой части экрана и автоматически исчезают, не перекрывая обзор виртуального стола.

Синие плашки предназначены для информирования о штатных системных событиях. Они появляются при подключении нового участника к столу, перетасовке колоды или объявлении итогового счета дилера.



Дилер: 19 очков

Рисунок 5.28 – Информационное уведомление о внутриигровом событии

Внутри игровой комнаты также активно применяются диалоговые окна для подтверждения критических решений. Если игрок выбирает стратегическое действие досрочной сдачи во время раздачи, система дополнительно информирует его о неминуемой потере половины от сделанной ставки, запрашивая финальное подтверждение.

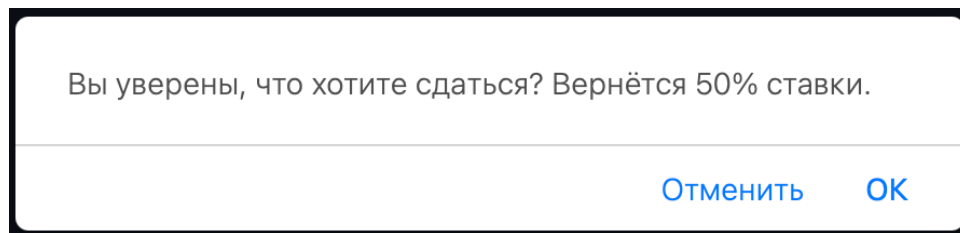


Рисунок 5.29 – Модальное окно подтверждения досрочной сдачи

Аналогичный механизм предупреждения срабатывает при попытке пользователя покинуть активную игровую комнату с помощью кнопки возврата или навигационного меню в заголовке страницы. Это защищает игрока от случайной потери текущего прогресса в раунде.

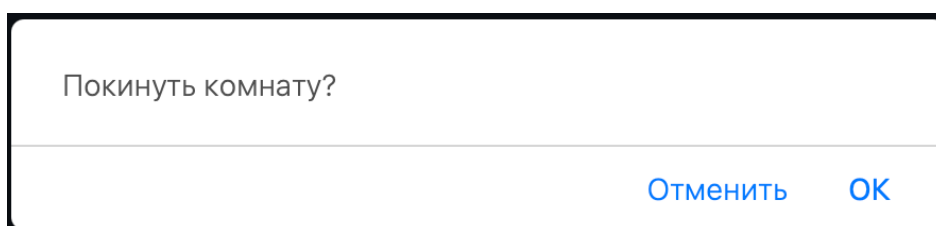


Рисунок 5.30 – Модальное окно подтверждения выхода из игровой комнаты

По завершении всех ходов в текущем раунде система поверх стола генерирует детализированное модальное окно с финансовыми результатами. В нем построчно отображаются исходы для каждого участника, суммы изменений баланса и общие итоги партии. Данное окно демонстрируется пользователям ровно шесть секунд и автоматически скрывается перед инициализацией новой фазы приема ставок.

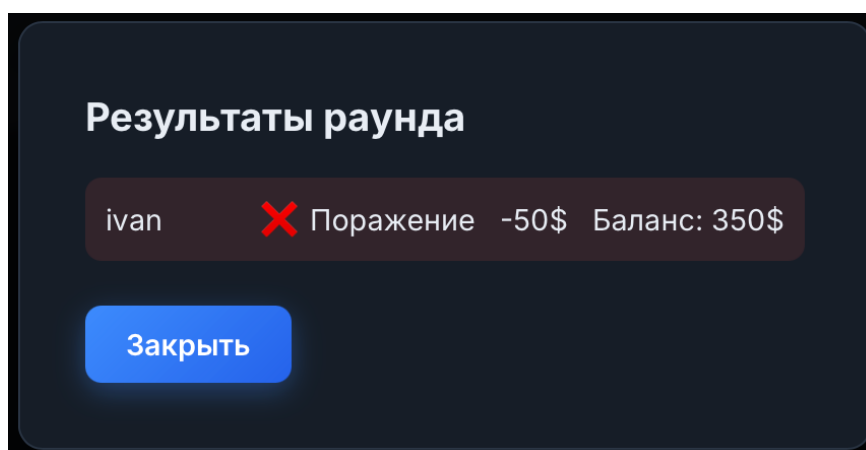


Рисунок 5.31 – Детализированное окно с финансовыми результатами раунда

```
[ "ROUND_RESULT", { "results": [ { "userId": 1, "name": "vitya", "result": "lose", "pa
```

Рисунок 5.32 – Содержание сообщения с результатом раунда

ЗАКЛЮЧЕНИЕ

В процессе выполнения данного курсового проекта была успешно спроектирована, реализована и протестирована масштабируемая многопользовательская распределённая программная система реального времени «*blackjack*». Актуальность проведенной работы обусловлена растущим спросом на высоконагруженные интерактивные веб-приложения, требующие мгновенного отклика, отказоустойчивости и надежной синхронизации состояний между множеством независимых клиентов.

В ходе аналитического этапа был проведён детальный анализ предметной области и выявлены ключевые проблемы *stateful*-серверов. Для обхода ограничений классического протокола *http* была выбрана событийно-ориентированная асинхронная парадигма на базе протокола *websocket*. Использование современного технологического стека, включающего среду выполнения *node.js*, фреймворк *express* и библиотеку *socket.io* поверх строго типизированного языка *typescript*, позволило создать надёжное серверное ядро, способное эффективно обрабатывать высококонкурентные потоки и обслуживать запросы с минимальной задержкой.

В рамках проектирования разработана детальная структурная схема слоистой архитектуры и спроектирован комплексный протокол сокет-событий. Он обеспечивает полнодуплексный обмен дельта-обновлениями состояний, существенно минимизируя сетевой трафик. Особое внимание уделено разделению ответственности баз данных: интеграция субд *postgresql* гарантировала долгосрочную персистентность профилей и безопасное изменение баланса, тогда как in-memory хранилище *redis* обеспечило высокопроизводительное кэширование динамических игровых сессий.

В систему внедрена надежная аутентификация на базе *http-only cookie*, защищающая от *xss*-атак, а также криптографическое хэширование паролей. Клиентский интерфейс построен по принципу «тонкого клиента» и управляется потоком серверных событий. Вся критическая игровая логика и математика карточных раздач изолированы на сервере, что фундаментально исключает несанкционированную модификацию данных или использование уязвимостей со стороны клиента.

Комплексное функциональное и нагрузочное тестирование подтвердило абсолютную корректность работы игрового движка и стабильность сессий при высоких нагрузках. Таким образом, поставленные задачи были полностью решены, а цель по созданию безопасной и высокотехнологичной игровой платформы *blackjack* успешно достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Node.js v22 API Reference [Электронный ресурс]. – Режим доступа: <https://nodejs.org/docs/latest-v22.x/api/>.
- [2] Express 5.x API Reference [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/api.html>.
- [3] Express.js. Routing Guide [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/guide/routing.html>.
- [4] The TypeScript Handbook [Электронный ресурс] / Microsoft Corporation. – Режим доступа: <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [5] Socket.IO v4 Documentation [Электронный ресурс]. – Режим доступа: <https://socket.io/docs/v4/>.
- [6] The Socket.IO Protocol Specification [Электронный ресурс]. – Режим доступа: <https://socket.io/docs/v4/socket-io-protocol/>.
- [7] PostgreSQL 17 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/index.html>.
- [8] Redis Documentation: Develop with Redis [Электронный ресурс]. – Режим доступа: <https://redis.io/docs/latest/develop/>.
- [9] The WebSocket Protocol: RFC 6455 [Электронный ресурс] / I. Fette, A. Melnikov. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc6455>.
- [10] Password Storage Cheat Sheet [Электронный ресурс] / OWASP Foundation. – Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.
- [11] Handlebars Language Guide [Электронный ресурс]. – Режим доступа: <https://handlebarsjs.com/guide/>.
- [12] Fisher–Yates shuffle [Электронный ресурс]. – Режим доступа: https://dev.to/tanvir_azad/fisher-yates-shuffle-the-right-way-to-randomize-an-array-4d2p.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

```
// Импортируем тип UserSession — используется только для типизации параметра хелпера
import type UserSession from "../src/express/models/userSession.dto.js";
/**
 * Регистрирует Handlebars-хелпер "isAuth".
 *
 * Хелпер позволяет в шаблонах условно показывать блоки в зависимости от
 * того, авторизован ли пользователь:
 *
 * {{#isAuth user}}
 *   Привет, {{user.name}}! ← выводится если сессия есть
 * {{else}}
 *   Войдите в аккаунт      ← выводится если сессии нет
 * {{/isAuth}}
 *
 * @param hbsInstance — экземпляр Handlebars (передается при регистрации)
 */
export const isAuth = (hbsInstance: any) => {
  hbsInstance.registerHelper("isAuth", function (this: any, userSession: UserSession, options:
any) {
    if (userSession) {
      // Пользователь авторизован — рендерим основной блок (fn)
      return options.fn(this);
    } else {
      // Пользователь не авторизован — рендерим блок {{else}} (inverse)
      return options.inverse(this);
    }
  });
};

=====
FILE: handlebars_helpers/registerHelpers.ts
=====
// Импортируем хелпер isAuth из соседнего модуля
import { isAuth } from "../isAuth.js";
/**
 * Точка входа для регистрации всех Handlebars-хелперов.
 * Вызывается один раз при инициализации приложения (src/server/express.ts).
 *
 * @param hbsInstance — экземпляр Handlebars
 */
export default (hbsInstance: any) => {
  isAuth(hbsInstance);
};

=====
FILE: public/js/game.js
=====
// -----
// Инициализация Socket.IO — принудительно используем WebSocket-транспорт
// (без long-polling), чтобы снизить задержки в игре
// -----
const socket = io({ transports: ["websocket"] });
// ID текущего пользователя — вставляется сервером в HTML как window.__MY_USER_ID__
const myUserId = Number(window.__MY_USER_ID__) || null;
// Глобальное состояние: чей сейчас ход, баланс, последнее состояние комнаты
let currentTurn = null;
let myBalance = null;
let lastRoomState = null;
// -----
// Вспомогательные функции
// -----
/**
 * Возвращает roomId из data-атрибута тега <body>.
 * Значение устанавливается сервером при рендере страницы.
 */
function getRoomId() {
  return document.body?.dataset?.roomId || null;
}
/**
 * Универсальная обёртка для отправки игровых событий на сервер.
 * Автоматически добавляет roomId к payload.
 *
 * @param event — название события (напр. "HIT", "STAND")
 * @param payload — дополнительные данные события
 * @returns false если emit не удался (нет комнаты или нет соединения)
```

```

*/
function emitGameEvent(event, payload = {}) {
  const roomId = getRoomId();
  if (!roomId) {
    showNotification("Комната не определена", "error");
    return false;
  }
  if (!socket.connected) {
    showNotification("Нет соединения с сервером", "error");
    return false;
  }
  socket.emit(event, { roomId, ...payload });
  return true;
}
// -----
// Обработчики событий сокета – системные (connect / disconnect)
// -----
socket.on("connect", () => {
  console.log("[socket] connected:", socket.id);
  const roomId = getRoomId();
  // Сразу после подключения входим в комнату (повторное подключение после разрыва)
  if (roomId) socket.emit("ROOM_JOIN", { roomId });
});
socket.on("disconnect", () => {
  console.log("[socket] disconnected");
  showNotification("Соединение потеряно", "error");
});
// -----
// Обработчики событий сокета – игровые
// -----
/**
 * ROOM_STATE – полное состояние комнаты.
 * Приходит при входе в комнату и при ручном обновлении (GET_ROOM_STATE).
 */
socket.on("ROOM_STATE", (data) => {
  console.log("[ROOM_STATE]", data);
  lastRoomState = data;
  renderFullState(data);
});
/**
 * GAME_STARTED – игра началась: все ставки сделаны, карты розданы.
 * Обновляем состояние и подсвечиваем активного игрока.
 */
socket.on("GAME_STARTED", (data) => {
  console.log("[GAME_STARTED]", data);
  showNotification("Игра началась!", "success");
  renderFullState(data);
  updateControls(data.currentPlayerId);
  highlightCurrentPlayer(data.currentPlayerId);
});
/**
 * PLAYER_ACTION – один из игроков совершил действие (hit/stand/double/split/surrender).
 * Обновляем его руки и баланс, а также кнопки управления для текущего пользователя.
 */
socket.on("PLAYER_ACTION", (data) => {
  console.log("[PLAYER_ACTION]", data);
  renderPlayerHands(data.userId, data.hands, data.currentHandIndex);
  // Обновляем баланс на экране, если он пришёл (double/split/surrender)
  if (data.balance != null) {
    setPlayerBalance(data.userId, data.balance);
  }
  // Если это наше действие – синхронизируем локальный кеш и пересчитываем кнопки
  if (isMe(data.userId) && lastRoomState) {
    const me = lastRoomState.players.find(p => isMe(p.userId));
    if (me) {
      me.hands = data.hands;
      me.currentHandIndex = data.currentHandIndex;
      updateActionButton(lastRoomState);
    }
  }
});
/**
 * TURN_CHANGED – ход перешёл к другому игроку или к другой руке после split.
 * Обновляем localState и UI-кнопки.
 */
socket.on("TURN_CHANGED", ({ userId, currentHandIndex }) => {
  console.log("[TURN_CHANGED] userId:", userId, "currentHandIndex:", currentHandIndex);
  currentTurn = userId;
  if (lastRoomState) {
    lastRoomState.currentPlayerId = userId;
  }
});

```

```

        // Если передана конкретная рука (после split), обновляем её рендер
        if (currentHandIndex !== null) {
            const player = lastRoomState.players.find(p => p.userId === userId);
            if (player) {
                player.currentHandIndex = currentHandIndex;
                renderPlayerHands(userId, player.hands, currentHandIndex);
            }
        }
        updateActionButtons(lastRoomState);
    }
    highlightCurrentPlayer(userId);
});
/**
 * DEALER_PLAY — дилер собирает карты до 17+.
 * Показываем все карты дилера и уведомление с его суммой.
 */
socket.on("DEALER_PLAY", ({ cards, score }) => {
    console.log("[DEALER_PLAY]", cards, score);
    renderDealer(cards, score, true); // true = показать все карты (не скрывать вторую)
    showNotification(`Дилер: ${score} очков`, "info");
});
/**
 * ROUND_RESULT — итоги раунда: выигрыши/проигрыши всех игроков.
 * Блокируем кнопки и показываем модальное окно с результатами.
 */
socket.on("ROUND_RESULT", (data) => {
    console.log("[ROUND_RESULT]", data);
    renderDealer(data.dealer.cards, data.dealer.score, true);
    renderRoundResults(data.results);
    syncBalancesFromResults(data.results);
    setControlsDisabled(true); // отключаем кнопки до следующей фазы ставок
});
/**
 * BETTING_PHASE — начинается новый раунд, все делают ставки.
 * Показываем панель ставок и скрываем игровые кнопки.
 */
socket.on("BETTING_PHASE", ({ deckShuffled }) => {
    console.log("[BETTING_PHASE]");
    if (deckShuffled) showNotification("Колода перетасована!", "info");
    showBettingUI();
    setControlsDisabled(true);
    updateBetButtonState(lastRoomState);
    highlightCurrentPlayer(null); // снимаем подсветку с текущего игрока
});
/**
 * DECK_SHUFFLED — колода была перемешана (осталось мало карт).
 */
socket.on("DECK_SHUFFLED", ({ remainingCards }) => {
    showNotification(`Колода перетасована (${remainingCards} карт)`, "info");
});
/**
 * PLAYER_JOINED — новый игрок подключился к комнате.
 */
socket.on("PLAYER_JOINED", ({ userId, name }) => {
    showNotification(`${name} подключился`, "info");
    addPlayerToList(userId, name);
});
/**
 * PLAYER_LEFT — игрок покинул комнату.
 */
socket.on("PLAYER_LEFT", ({ userId }) => {
    showNotification(`Игрок #${userId} покинул комнату`, "warning");
    removePlayerFromList(userId);
});
/**
 * CHAT_MESSAGE — входящее сообщение в чате.
 */
socket.on("CHAT_MESSAGE", ({ userId, name, text }) => {
    appendChatMessage(name, text, userId === myUserId);
});
/**
 * SESSION_INVALID — сессия истекла или недействительна.
 * Принудительно перенаправляем на страницу входа.
 */
socket.on("SESSION_INVALID", () => {
    alert("Сессия истекла. Вы будете перенаправлены на страницу входа.");
    window.location.href = "/authorization";
});
/**
 * FORCE_DISCONNECT — сервер принудительно разрывает соединение

```

```

* (например, при входе с другого устройства).
*/
socket.on("FORCE_DISCONNECT", ({ reason }) => {
    alert(reason);
    window.location.href = "/authorization";
});
/**
* ERROR — ошибка игровой логики (нельзя сделать ход и т.п.).
*/
socket.on("ERROR", ({ code, message }) => {
    console.error("[ERROR]", code, message);
    showNotification(message, "error");
});
/**
* BET_CONFIRMED — сервер подтвердил ставку текущего пользователя.
* Обновляем баланс и скрываем панель ставок.
*/
socket.on("BET_CONFIRMED", ({ balance }) => {
    showNotification(`Ставка принята! Баланс: ${balance}`, "success");
    updateBalance(balance);
    if (myUserId !== null) setPlayerBalance(myUserId, balance);
    hideBettingUI();
    showNotification("Ожидание других игроков...", "info");
});
/**
* PLAYER_BALANCE — обновление баланса конкретного игрока
* (рассылается после double/split/surrender).
*/
socket.on("PLAYER_BALANCE", ({ userId, balance }) => {
    setPlayerBalance(userId, balance);
});
// -----
// Запрос обновления состояния комнаты вручную
// -----
/**
* Отправляет GET_ROOM_STATE на сервер и блокирует кнопку на время запроса.
* Таймаут 8 сек страхует от зависания кнопки, если сервер не ответил.
*/
function requestRoomState() {
    const roomId = getRoomId();
    if (!roomId) {
        showNotification("Комната не определена", "error");
        return;
    }
    if (!socket.connected) {
        showNotification("Нет соединения с сервером", "error");
        return;
    }
    const btn = document.getElementById("btn-refresh");
    if (btn) btn.disabled = true;
    socket.emit("GET_ROOM_STATE", { roomId }, (res) => {
        if (btn) btn.disabled = false;
        if (res?.ok) {
            showNotification("Состояние обновлено", "success");
        } else if (res?.message) {
            showNotification(res.message, "error");
        }
    });
    // Страхочный таймаут: разблокируем кнопку если ask не пришёл за 8 секунд
    setTimeout(() => {
        if (btn?.disabled) {
            btn.disabled = false;
            showNotification("Таймаут ответа сервера", "warning");
        }
    }, 8000);
}
// -----
// Чат
// -----
/**
* Отправляет текст из поля ввода чата на сервер и очищает поле.
*/
function sendChatMessage() {
    const input = document.getElementById("chat-input");
    const text = input?.value?.trim();
    if (!text) return;
    if (!emitGameEvent("CHAT_MESSAGE", { text })) return;
    input.value = "";
}
// -----

```

```

// Инициализация кнопок управления игрой
// -----
/**
 * Привязывает обработчики к кнопкам игры.
 * Вызывается после загрузки DOM.
 */
function initGameButtons() {
  // Основные игровые действия — просто отправляем соответствующее событие
  document.getElementById("btn-hit")?.addEventListener("click", () => {
    emitGameEvent("HIT");
  });
  document.getElementById("btn-stand")?.addEventListener("click", () => {
    emitGameEvent("STAND");
  });
  document.getElementById("btn-double")?.addEventListener("click", () => {
    emitGameEvent("DOUBLE");
  });
  document.getElementById("btn-split")?.addEventListener("click", () => {
    emitGameEvent("SPLIT");
  });
  // Surrender с подтверждением — возвращается только 50% ставки
  document.getElementById("btn-surrender")?.addEventListener("click", () => {
    if (confirm("Вы уверены, что хотите сдаться? Вернётся 50% ставки.")) {
      emitGameEvent("SURRENDER");
    }
  });
  // Кнопка "Поставить" — проверяем, что ставки ещё нет, и берём сумму из input
  document.getElementById("btn-bet")?.addEventListener("click", () => {
    if (!canPlaceBet()) {
      showNotification("Вы уже сделали ставку", "warning");
      return;
    }
    const input = document.getElementById("bet-input");
    const amount = Number(input?.value ?? 0);
    if (amount <= 0) {
      showNotification("Введите корректную ставку", "error");
      return;
    }
    emitGameEvent("PLACE_BET", { amount });
  });
  // Клик по фишке (chip) устанавливает её номинал в поле ввода ставки
  document.querySelectorAll(".chip").forEach(chip => {
    chip.addEventListener("click", () => {
      const input = document.getElementById("bet-input");
      if (input) input.value = chip.dataset.value;
    });
  });
  // Чат: кнопка и Enter
  document.getElementById("btn-send-msg")?.addEventListener("click", sendChatMessage);
  document.getElementById("chat-input")?.addEventListener("keydown", (e) => {
    if (e.key === "Enter") sendChatMessage();
  });
  // Кнопка ручного обновления состояния
  document.getElementById("btn-refresh")?.addEventListener("click", requestRoomState);
  // Закрытие модального окна с результатами раунда
  document.getElementById("result-modal-close")?.addEventListener("click", () => {
    const modal = document.getElementById("result-modal");
    if (modal) modal.style.display = "none";
  });
}
// Запускаем инициализацию кнопок после загрузки DOM
if (document.readyState === "loading") {
  document.addEventListener("DOMContentLoaded", initGameButtons);
} else {
  initGameButtons();
}
// -----
// Функции рендеринга состояния
// -----
/**
 * Полный рендер состояния комнаты (вызывается при ROOM_STATE / GAME_STARTED).
 * Обновляет все UI-элементы: дилера, игроков, кнопки, панель ставок.
 */
function renderFullState(state) {
  lastRoomState = state;
  currentTurn = state.currentPlayerId;
  // Показываем карты дилера открытыми только в фазе ставок или если все уже открыты
  const revealDealer = state.status === "BETTING"
    || state.dealer.cards.every(c => c !== "??");
  renderDealer(state.dealer.cards, state.dealer.score, revealDealer);
}

```

```

renderAllPlayers(state.players);
syncBalancesFromPlayers(state.players);
updateDeckCount(state.deckRemaining);
if (state.status === "BETTING") {
    // Фаза ставок: показываем панель ставок, игровые кнопки отключены
    showBettingUI();
    setControlsDisabled(true);
    updateBetButtonState(state);
    highlightCurrentPlayer(null);
} else {
    // Игровая фаза: скрываем ставки, активируем нужные кнопки
    hideBettingUI();
    updateActionButtons(state);
    highlightCurrentPlayer(state.currentPlayerId);
}
}
/**
 * Рендерит карты и счёт дилера.
 *
 * @param cards    - массив карт в компактном формате ["AH", "??", ...]
 * @param score    - числовой счёт руки
 * @param reveal   - если false, вторая карта дилера скрыта (рубашкой)
 */
function renderDealer(cards, score, reveal) {
    const el = document.getElementById("dealer-cards");
    if (!el) return;
    el.innerHTML = cards
        .map(c => `<div class="card">${reveal ? formatCard(c) : (c === "??" ? "🃏" :
formatCard(c))}</div>`)
        .join("");
    const scoreEl = document.getElementById("dealer-score");
    if (scoreEl) scoreEl.textContent = reveal && score > 0 ? `Дилер: ${score}` : "";
}
/**
 * Полностью перерисовывает список всех игроков в комнате.
 *
 * @param players - массив PlayerState из состояния комнаты
 */
function renderAllPlayers(players) {
    const container = document.getElementById("players-container");
    if (!container) return;
    container.innerHTML = "";
    players.forEach(p => {
        const div = document.createElement("div");
        div.className = "player-slot";
        div.id = `player-${p.userId}`;
        div.innerHTML = `
            <div class="player-name">${escapeHtml(p.name)} <span class="player-
balance">${p.balance}</span></div>
            <div class="player-hands" id="hands-${p.userId}"></div>
        `;
        container.appendChild(div);
        renderPlayerHands(p.userId, p.hands, p.currentHandIndex);
    });
}
/**
 * Рендерит руки конкретного игрока.
 * Активная рука выделяется классом "hand--active".
 *
 * @param userId      - ID игрока
 * @param hands       - массив его рук (HandState)
 * @param currentHandIndex - индекс активной руки (нужен при split)
 */
function renderPlayerHands(userId, hands, currentHandIndex) {
    const container = document.getElementById(`hands-${userId}`);
    if (!container) return;
    container.innerHTML = hands.map((hand, i) => `
        <div class="hand ${i === currentHandIndex ? "hand--active" : ""} hand--
${hand.status.toLowerCase()} ">
            <div class="hand-cards">${hand.cards.map(c => `<span
class="card">${formatCard(c)}</span>`).join("")}</div>
            <div class="hand-info">
                ${hand.score} очков · ${hand.bet}$ · <span class="hand-
status">${hand.status}</span>
            </div>
        </div>
    `).join("");
}
/**
 * Показывает модальное окно с итогами раунда.

```

```

* Автоматически закрывается через 6 секунд.
*
* @param results - массив RoundResultEntry
*/
function renderRoundResults(results) {
    const modal = document.getElementById("result-modal");
    const body = document.getElementById("result-body");
    if (!modal || !body) return;
    body.innerHTML = results.map(r => `
        <div class="result-row result-row--${r.result}">
            <span>${escapeHtml(r.name)}</span>
            <span>${
                r.result === "blackjack" ? "🃏 Блекджек!" :
                r.result === "win" ? "✅ Победа" :
                r.result === "push" ? "👉 Ничья" : "❌ Поражение"
            }</span>
            <span>${(r.netProfit ?? 0) > 0 ? "+" : ""}${r.netProfit ?? 0}</span>
            <span>Баланс: ${r.newBalance}</span>
        </div>
    `).join("");
    modal.style.display = "flex";
    setTimeout(() => { modal.style.display = "none"; }, 6000);
}
// -----
// Логика ставок
// -----
/**
 * Проверяет, сделал ли текущий пользователь ставку в текущем раунде.
 */
function hasPlacedBet(state = lastRoomState) {
    if (!state || state.status !== "BETTING") return false;
    const me = state.players?.find((p) => isMe(p.userId));
    if (!me || !Array.isArray(me.hands)) return false;
    return me.hands.some((hand) => Number(hand?.bet) > 0);
}
/**
 * Можно ли сделать ставку: только в фазу BETTING и если ещё не ставил.
 */
function canPlaceBet(state = lastRoomState) {
    if (!state || state.status !== "BETTING") return false;
    return !hasPlacedBet(state);
}
/**
 * Обновляет состояние кнопки "Поставить" и поля ввода:
 * блокирует их, если ставка уже сделана.
 */
function updateBetButtonState(state = lastRoomState) {
    const betButton = document.getElementById("btn-bet");
    const betInput = document.getElementById("bet-input");
    const chips = document.querySelectorAll(".chip");
    const disabled = !canPlaceBet(state);
    if (betButton) {
        betButton.disabled = disabled;
        betButton.textContent = disabled ? "Ставка уже сделана" : "Поставить";
    }
    if (betInput) betInput.disabled = disabled;
    chips.forEach((chip) => { chip.disabled = disabled; });
}
// -----
// Логика кнопок игрового действия
// -----
/**
 * Клиентская проверка возможности split:
 * 2 карты с одинаковым рангом (первым символом кода карты).
 */
function canSplit(hand) {
    return hand.cards.length === 2 && hand.cards[0]?.[0] === hand.cards[1]?.[0];
}
/**
 * Обновляет активность кнопок HIT / STAND / DOUBLE / SPLIT / SURRENDER
 * в зависимости от того, чей сейчас ход и какова активная рука.
 */
function updateActionButtons(state) {
    const isMyTurn = state.currentPlayerId !== null && Number(state.currentPlayerId) === myUserId;
    const me = state.players.find(p => isMe(p.userId));
    const hand = me?.hands[me.currentHandIndex ?? 0];
    // Кнопки активны только если наш ход и рука ещё не завершена
    const active = isMyTurn && hand?.status === "ACTIVE";
    const hit = document.getElementById("btn-hit");

```

```

const stand = document.getElementById("btn-stand");
const dbl = document.getElementById("btn-double");
const split = document.getElementById("btn-split");
const surrender = document.getElementById("btn-surrender");
if (hit) hit.disabled = !active;
if (stand) stand.disabled = !active;
if (dbl) dbl.disabled = !active || !hand || hand.cards.length !== 2; // double
только на 2 картах
if (split) split.disabled = !active || !hand || !canSplit(hand); // split
только при одинаковых рангах
if (surrender) surrender.disabled = !active || !hand || hand.cards.length !== 2; // surrender
только на 2 картах
}
/**
 * Обёртка: обновляет кнопки при смене хода, используя lastRoomState.
 */
function updateControls(currentPlayerId) {
  if (lastRoomState) {
    updateActionButtons({ ...lastRoomState, currentPlayerId });
  }
}
/**
 * Включает или отключает все игровые кнопки разом.
 * Используется при переходе между фазами.
 */
function setControlsDisabled(disabled) {
  ["btn-hit", "btn-stand", "btn-double", "btn-split", "btn-surrender"].forEach(id => {
    const btn = document.getElementById(id);
    if (btn) btn.disabled = disabled;
  });
}
// -----
// UI-утилиты
// -----
/**
 * Подсвечивает слот активного игрока классом "player-slot--active".
 * Передай null чтобы снять подсветку со всех.
 */
function highlightCurrentPlayer(userId) {
  document.querySelectorAll(".player-slot").forEach(el => el.classList.remove("player-slot--active"));
  document.getElementById(`player-${userId}`)?.classList.add("player-slot--active");
}
/** Показывает панель ставок */
function showBettingUI() { document.getElementById("betting-panel")?.classList.remove("hidden"); }
/** Скрывает панель ставок */
function hideBettingUI() { document.getElementById("betting-panel")?.classList.add("hidden"); }
/**
 * Проверяет, принадлежит ли userId текущему пользователю.
 */
function isMe(userId) {
  return myUserId !== null && Number(userId) === myUserId;
}
/**
 * Обновляет отображение баланса в шапке страницы (#my-balance).
 */
function updateBalance(balance) {
  if (balance == null || Number.isNaN(Number(balance))) return;
  myBalance = Number(balance);
  const el = document.getElementById("my-balance");
  if (el) el.textContent = `${myBalance}$`;
}
/**
 * Обновляет баланс в карточке конкретного игрока.
 * Если это наш баланс — также обновляет шапку.
 */
function setPlayerBalance(userId, balance) {
  if (balance == null || Number.isNaN(Number(balance))) return;
  const slot = document.getElementById(`player-${userId}`);
  const span = slot?.querySelector(".player-balance");
  if (span) span.textContent = `${Number(balance)}$`;
  if (isMe(userId)) updateBalance(balance);
}
/**
 * Синхронизирует балансы всех игроков из массива PlayerState.
 */
function syncBalancesFromPlayers(players) {
  if (!Array.isArray(players)) return;
  for (const p of players) {

```

```

        setPlayerBalance(p.userId, p.balance);
    }
}
/**
 * Синхронизирует балансы после раунда из массива RoundResultEntry.
 */
function syncBalancesFromResults(results) {
    if (!Array.isArray(results)) return;
    for (const r of results) {
        if (r.newBalance !== null) setPlayerBalance(r.userId, r.newBalance);
    }
}
/**
 * Обновляет счётчик оставшихся карт в колоде.
 */
function updateDeckCount(remaining) {
    const el = document.getElementById("deck-remaining");
    if (el) el.textContent = `Колода: ${remaining} карт`;
}
/**
 * Добавляет нового игрока в список (без полного перерендера).
 * Не добавляет дубликаты.
 */
function addPlayerToList(userId, name) {
    const container = document.getElementById("players-container");
    if (!container || document.getElementById(`player-${userId}`)) return;
    const div = document.createElement("div");
    div.className = "player-slot";
    div.id = `player-${userId}`;
    div.innerHTML = `<div class="player-name">${escapeHtml(name)}</div><div class="player-hands"
id="hands-${userId}"></div>`;
    container.appendChild(div);
}
/**
 * Удаляет слот игрока из DOM.
 */
function removePlayerFromList(userId) {
    document.getElementById(`player-${userId}`)?.remove();
}
/**
 * Добавляет сообщение в чат и прокручивает к низу.
 *
 * @param name — имя отправителя
 * @param text — текст сообщения
 * @param isMine — выравнивание по правому краю если это наше сообщение
 */
function appendChatMessage(name, text, isMine) {
    const container = document.getElementById("chat-messages");
    if (!container) return;
    const div = document.createElement("div");
    div.className = `chat-msg ${isMine ? "chat-msg--mine" : ""}`;
    div.innerHTML = `<b>${escapeHtml(name)}</b> ${escapeHtml(text)}`;
    container.appendChild(div);
    container.scrollTop = container.scrollHeight;
}
/**
 * Показывает всплывающее уведомление на 3.5 секунды.
 *
 * @param message — текст уведомления
 * @param type — тип: "info" | "success" | "warning" | "error"
 */
function showNotification(message, type = "info") {
    const el = document.getElementById("notification");
    if (!el) return;
    el.textContent = message;
    el.className = `notification notification--${type} notification--visible`;
    clearTimeout(el._timer);
    el._timer = setTimeout(() => el.classList.remove("notification--visible"), 3500);
}
/**
 * Форматирует код карты в HTML-строку с мастью и рангом.
 *
 * "???" → рубашка карты 🃏
 * "AH" → <span class="card-red">A♥</span>
 */
function formatCard(card) {
    if (card === "???") return "🃏";
    const suitMap = { H: "♥", D: "♦", C: "♣", S: "♠" };
    const rank = card[0] === "T" ? "10" : card[0]; // T = Ten (10)
    const suitKey = card[1];

```

```

    const suit    = suitMap[suitKey] ?? suitKey;
    const red     = suitKey === "H" || suitKey === "D"; // червы и бубны – красные
    return `<span class="${red ? "card-red" : ""}">${rank}${suit}</span>`;
}
/**
 * Экранирует HTML-спецсимволы для безопасной вставки пользовательского текста в DOM.
 */
function escapeHtml(str) {
    return String(str).replace(/([<>"']/g, m => (
        { "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;" }[m]
    ));
}
}
=====
FILE: public/scripts/lobbyRoomsScript.js
=====
// -----
// Элементы интерфейса лобби
// -----
const updateBtn      = document.getElementById("update");
const roomsList      = document.getElementById("rooms-list");
const roomsEmpty     = document.getElementById("rooms-empty");
const createForm      = document.getElementById("create-room-form");
const createError     = document.getElementById("create-room-error");
const createBtn       = document.getElementById("create-room-btn");
const isPrivateCheckbox = document.getElementById("is-private");
const passwordField   = document.getElementById("password-field");
// -----
// Утилиты отображения ошибок
// -----
/** Показывает сообщение об ошибке в элементе el */
function showError(el, message) {
    el.textContent = message;
    el.classList.remove("hidden");
}
/** Скрывает сообщение об ошибке */
function hideError(el) {
    el.textContent = "";
    el.classList.add("hidden");
}
}
// -----
// Логика поля пароля
// -----
/**
 * Показывает/скрывает поле пароля в зависимости от чекбокса "Приватная".
 * Также управляет атрибутом required, чтобы форма не сабмитилась без пароля.
 */
function togglePasswordField() {
    const isPrivate = isPrivateCheckbox.checked ?? false;
    passwordField?.classList.toggle("hidden", !isPrivate);
    const passwordInput = passwordField?.querySelector('input[name="password"]');
    if (passwordInput) passwordInput.required = isPrivate;
}
isPrivateCheckbox?.addEventListener("change", togglePasswordField);
togglePasswordField(); // установить состояние при загрузке страницы
// -----
// Действия с комнатой
// -----
/**
 * Вход в комнату. Если приватная – запрашивает пароль через prompt.
 *
 * @param roomObj – объект RoomMeta из API
 */
async function joinRoom(roomObj) {
    let password = null;
    if (roomObj.isPrivate) {
        password = prompt("Введите пароль комнаты:");
        if (!password) return; // пользователь отменил диалог
    }
    const response = await fetch(`/api/lobby/room/${roomObj.roomId}`, {
        method: "PUT",
        headers: { "content-type": "application/json" },
        body: JSON.stringify({ password }),
    });
    if (!response.ok) {
        alert("Не удалось войти в комнату");
        return;
    }
    const data = await response.json();
    if (data?.message !== "all good") {
        alert(data?.message ?? "Не удалось войти в комнату");
    }
}

```

```

        return;
    }
    window.location.href = `/lobby/room/${roomObj.roomId}`;
}
/**
 * Удаляет пустую комнату после подтверждения.
 *
 * @param roomObj — объект RoomMeta из API
 */
async function deleteRoom(roomObj) {
    if (!confirm(`Удалить пустую комнату "${roomObj.name}"?`)) return;
    const response = await fetch(`/api/lobby/room/${roomObj.roomId}`, {
        method: "DELETE",
        headers: { "content-type": "application/json" },
    });
    if (!response.ok) {
        const data = await response.json().catch(() => ({}));
        alert(data?.message ?? "Не удалось удалить комнату");
        return;
    }
    const data = await response.json();
    if (data?.message !== "all good") {
        alert(data?.message ?? "Не удалось удалить комнату");
        return;
    }
    await showRooms(); // перерисовываем список
}
/**
 * Создаёт DOM-элемент карточки комнаты.
 * Кнопка "Удалить" появляется только если комната пустая.
 *
 * @param roomObj — объект RoomMeta
 * @returns <li> элемент с данными комнаты
 */
function makeRoom(roomObj) {
    const roomEl = document.createElement("li");
    roomEl.className = "room-card";
    const isEmpty = Number(roomObj.currentPlayersCount) === 0;
    roomEl.innerHTML = `
        <p class="room-card__name">${escapeHtml(roomObj.name)}</p>
        ${roomObj.description ? `<p class="room-
card__meta">${escapeHtml(roomObj.description)}</p>` : ""}
        <p class="room-card__meta">
            Игроков: ${roomObj.currentPlayersCount} / ${roomObj.maxPlayersCount}
            · Колод: ${roomObj.deckCount ?? 6}
        </p>
        ${roomObj.isPrivate ? `<span class="room-card__private">Приватная</span>` : ""}
        ${isEmpty ? `<div class="room-card__actions"><button type="button" class="btn btn--ghost
btn--sm room-delete-btn">Удалить</button></div>` : ""}
    `;
    // Клик по карточке — вход в комнату
    roomEl.addEventListener("click", () => joinRoom(roomObj));
    // Клик по "Удалить" — не всплывает, чтобы не срабатывал joinRoom
    if (isEmpty) {
        const deleteBtn = roomEl.querySelector(".room-delete-btn");
        deleteBtn?.addEventListener("click", (event) => {
            event.stopPropagation();
            deleteRoom(roomObj);
        });
    }
    return roomEl;
}
/**
 * Безопасное экранирование HTML (вариант через textContent).
 */
function escapeHtml(text) {
    const div = document.createElement("div");
    div.textContent = text ?? "";
    return div.innerHTML;
}
/**
 * Загружает список комнат с сервера и отображает их.
 * Если комнат нет — показывает заглушку #rooms-empty.
 */
async function showRooms() {
    const response = await fetch("/api/lobby/rooms", {
        method: "GET",
        headers: { "content-type": "application/json" },
    });
    if (!response.ok) {

```

```

        console.error("server error!");
        return;
    }
    const data = await response.json();
    const rooms = Array.isArray(data) ? data : [];
    if (rooms.length === 0) {
        roomsList.replaceChildren();
        roomsEmpty?.classList.remove("hidden");
        return;
    }
    roomsEmpty?.classList.add("hidden");
    roomsList.replaceChildren(...rooms.map(makeRoom));
}
// -----
// Форма создания комнаты
// -----
createForm?.addEventListener("submit", async (e) => {
    e.preventDefault();
    hideError(createError);
    createBtn.disabled = true; // защита от двойного сабмита
    const formData = new FormData(createForm);
    const isPrivate = formData.get("isPrivate") === "on";
    const payload = {
        name: formData.get("name"),
        description: formData.get("description") || "",
        maxPlayersCount: Number(formData.get("maxPlayersCount")),
        deckCount: Number(formData.get("deckCount")),
        isPrivate,
        password: isPrivate ? formData.get("password") : null,
    };
    try {
        const response = await fetch("/api/lobby/rooms", {
            method: "POST",
            headers: { "content-type": "application/json" },
            body: JSON.stringify(payload),
        });
        const data = await response.json();
        if (!response.ok || data?.message !== "all good" || !data?.roomId) {
            showError(createError, data?.message ?? "Не удалось создать комнату");
            return;
        }
        // Успех — переходим в созданную комнату
        window.location.href = `/lobby/room/${data.roomId}`;
    } catch {
        showError(createError, "Ошибка сети");
    } finally {
        createBtn.disabled = false; // всегда разблокируем кнопку
    }
});
// Кнопка обновления списка и первоначальная загрузка
updateBtn?.addEventListener("click", showRooms);
showRooms();
=====
FILE: public/scripts/logout.js
=====
// IIFE — изолируем переменные, чтобы не загрязнять глобальный scope
(function () {
    const btn = document.getElementById("logout");
    // Защита от двойной привязки (если скрипт загружается несколько раз)
    if (!btn || btn.dataset.logoutBound === "1") return;
    btn.dataset.logoutBound = "1";
    btn.addEventListener("click", async (event) => {
        event.preventDefault();
        if (!confirm("Вы уверены, что хотите выйти из аккаунта?")) return;
        try {
            const response = await fetch("/api/authorization/logout", {
                method: "POST",
                headers: { "content-type": "application/json" },
            });
            const responseJson = await response.json();
            if (responseJson["message"] === "all good") {
                window.location.href = "/main";
                return;
            }
            console.log("Ошибка при попытке выхода из аккаунта!");
        } catch (error) {
            console.log("error occurred: ", error);
        }
    });
})();
})();

```

```

=====
FILE: public/scripts/roomHeaderScript.js
=====
// Шапка страницы комнаты – перехватываем клики по навигации,
// чтобы перед уходом из комнаты вызвать API leave и освободить слот
const header = document.getElementById("header");
if (!header) {
  console.warn("[roomHeaderScript] header not found");
} else {
  header.addEventListener("click", async (event) => {
    // Ищем ближайшую кнопку или ссылку (event delegation)
    const targetEl = event.target.closest("button, a");
    if (!targetEl || targetEl.id === "logout") return; // logout обрабатывается отдельно
    const href = targetEl.getAttribute("href");
    if (!href) return;
    event.preventDefault();
    if (!confirm("Покинуть комнату?")) return;
    try {
      // Уведомляем сервер о выходе из комнаты
      const response = await fetch("/api/lobby/room/leave", {
        method: "PUT",
        headers: { "content-type": "application/json" },
      });
      const data = await response.json();
      if (!response.ok || data?.message !== "all good") {
        alert(data?.message ?? "Не удалось выйти из комнаты");
        return;
      }
      // После успешного выхода переходим по ссылке
      window.location.href = href;
    } catch (error) {
      console.error("[roomHeaderScript]", error);
      alert("Ошибка при выходе из комнаты");
    }
  });
}

=====
FILE: public/scripts/roomScript.js
=====
// TODO: устаревший файл – хардкод адреса, не используется в production
const socket = io("http://127.0.0.1:8888");
=====
FILE: public/scripts/script.js
=====
// TODO: устаревший тестовый файл – WebSocket использует неправильный протокол
// (socket.send принимает строку, не объект), не используется в проекте
const socket = new WebSocket(`ws://${window.location.host}`);
socket.send({ method: "BET", amount: "200", roomId: "4" });
=====
FILE: public/scripts/userLoginScript.js
=====
const loginForm = document.getElementById("login-form");
const loginStatus = document.getElementById("login-status");
loginForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const nicknameInput = document.getElementById("nickname-input");
  const passwordInput = document.getElementById("password-input");
  const loginData = {
    identifier: nicknameInput.value, // email или никнейм
    password: passwordInput.value,
  };
  fetch("/api/authorization/login", {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify(loginData),
  })
    .then(response => response.json())
    .then(data => {
      console.log(data);
      if (data["message"] === "all good") {
        // Сервер вернул sessionToken – устанавливаем cookie через отдельный роут
        window.location.href = `/setcookie?sessionToken=${data["sessionToken"]}`;
        return;
      }
      // Показываем сообщение об ошибке
      loginStatus.textContent = data["message"];
    })
    .catch(error => { console.log(error); });
});
=====

```

```

FILE: public/scripts/userRegistrationScript.js
=====
const registerForm = document.getElementById("register-form");
const formStatus = document.getElementById("form-status");
registerForm.addEventListener("submit", async (event) => {
    event.preventDefault();
    const nickname = document.getElementById("nickname-input");
    const email = document.getElementById("email-input");
    const password = document.getElementById("password-input");
    const passwordRepeat = document.getElementById("password-repeat-input");
    // Клиентская валидация совпадения паролей
    if (password.value !== passwordRepeat.value) {
        formStatus.textContent = "Пароли не совпадают!";
        return;
    }
    const sendValues = { name: nickname.value, email: email.value, password: password.value };
    try {
        const response = await fetch("/api/authorization/register", {
            method: "POST",
            headers: { "content-type": "application/json" },
            body: JSON.stringify(sendValues),
        });
        const responseJson = await response.json();
        if (responseJson["message"] === "all good") {
            // Регистрация успешна — устанавливаем cookie через setCookie-плагин
            window.location.href = `/setCookie?sessionToken=${responseJson["sessionToken"]}`;
        }
        formStatus.textContent = responseJson["message"];
    } catch (error) {
        formStatus.textContent = "Ошибка подключения к серверу!";
        console.log("error occurred: ", error);
    }
});
=====
FILE: src/database/database.ts
=====
import { Pool } from "pg";
/**
 * Пул подключений к PostgreSQL.
 * Параметры берутся из переменных окружения (.env).
 * Пул переиспользует существующие соединения для снижения задержек.
 */
const pool = new Pool({
    user: process.env.DB_USER,
    host: process.env.DB_HOST,
    database: process.env.DB_NAME,
    port: Number(process.env.DB_PORT),
});
export default pool;
=====
FILE: src/database/redis.ts
=====
import { createClient } from "redis";
/**
 * Клиент Redis (по умолчанию localhost:6379).
 * Используется для хранения сессий, состояния игровых комнат и очередности ходов.
 *
 * Подключение выполняется явно в src/index.ts (await redisClient.connect()).
 */
const client = createClient();
client.on("error", (err) => {
    console.log(`redis error: ${err}`);
});
client.on("ready", () => {
    console.log("connected to redis!");
});
export default client;
=====
FILE: src/declarations/declarations.d.ts
=====
// Декларация для модуля express-hbs, у которого нет встроенных TypeScript-типов
declare module 'express-hbs';
// Расширяем глобальное пространство имён для кастомных валидаторов
declare global {
    namespace ValidationSpace {
        interface Validate {}
    }
}
=====
FILE: src/declarations/request.userSession.d.ts
=====

```

```

=====
import type UserSession from "../express/models/userSession.dto.ts";
/**
 * Расширяем стандартный Request Express'a полем userSession.
 * Устанавливается в AuthorizationMiddleware.checkUserByCookie.
 * null — если пользователь не авторизован.
 */
declare global {
  namespace Express {
    interface Request {
      userSession?: UserSession | null;
    }
  }
}

=====
FILE: src/express/api/controllers/authorization.api.controller.ts
=====
import type { Request, Response, NextFunction } from "express";
import userSessionService from "../../services/userSession.service.js";
import userService from "../../services/user.service.js";
import roomService from "../../services/room.service.js";
/**
 * Контроллер REST API авторизации.
 * Маршруты: /api/authorization/login | /register | /logout
 */
class AuthorizationApiController {
  /**
   * Вспомогательный метод: выкидывает все WebSocket-сессии пользователя
   * и удаляет его из текущей комнаты в Redis.
   *
   * Нужен при повторном входе с другого устройства и при logout.
   *
   * @param userId — ID пользователя
   * @param reason — причина отключения (показывается клиенту)
   */
  private kickFromRoom = async (userId: number, reason: string = "Выполнен вход с другого устройства") => {
    try {
      const { socketio } = await import("../server/server.js");
      const sockets = await socketio.fetchSockets();
      // Ищем все сокеты этого пользователя и отправляем им FORCE_DISCONNECT
      for (const s of sockets) {
        if ((s.data as any)?.userSession?.userId === userId) {
          s.emit("FORCE_DISCONNECT", { reason });
          s.disconnect();
        }
      }
    } catch {
      // socket может быть не инициализирован на момент вызова (первый старт)
    }
    // Убираем пользователя из Redis-комнаты в любом случае
    await roomService.removeUserFromCurrentRoom(userId);
  };
  /**
   * POST /api/authorization/login
   * Аутентифицирует пользователя по email/никнейму и паролю.
   * При успехе возвращает sessionToken для установки cookie.
   */
  login = async (req: Request, res: Response, next: NextFunction) => {
    if (!req.body) { res.json({ message: "Empty req.body" }); return; }
    const { identifier, password }: Record<string, string> = req.body;
    if (!identifier || !password) {
      res.status(400).json({ message: "Пустые значения для identifier или password" });
      return;
    }
    const userSession = await userService.authenticate(identifier, password);
    if (!userSession) {
      res.status(401).json({ message: "Неверный логин или пароль" });
      return;
    }
    // Сбрасываем предыдущую сессию (если пользователь уже залогинен с другого места)
    await this.kickFromRoom(userSession.userId);
    const token = await userSessionService.setSession(userSession);
    if (!token) {
      res.status(500).json({ message: "Ошибка при создании сессии" });
      return;
    }
    res.status(202).json({ message: "all good", sessionToken: token });
  };
  /**

```

```

* POST /api/authorization/register
* Регистрирует нового пользователя и сразу создаёт сессию.
*/
register = async (req: Request, res: Response, next: NextFunction) => {
  if (!req.body) { res.json({ message: "Empty req.body" }); return; }
  const { name, email, password } = req.body;
  if (!name || !email || !password) {
    res.status(400).json({ message: "Пустые значения для name, email или password" });
    return;
  }
  const userSession = await userService.addUser(name, email, password);
  if (!userSession) {
    res.status(400).json({ message: "Пользователь с такими данными уже существует" });
    return;
  }
  const token = await userSessionService.setSession(userSession);
  if (!token) {
    res.status(500).json({ message: "Ошибка при создании сессии" });
    return;
  }
  res.status(202).json({ message: "all good", sessionToken: token });
};
/**
* POST /api/authorization/logout
* Завершает сессию, выкидывает пользователя из комнаты.
*/
logout = async (req: Request, res: Response, next: NextFunction) => {
  const sessionToken = req.cookies["sessionToken"];
  if (!sessionToken) {
    res.status(400).json({ message: "Нет активной сессии" });
    return;
  }
  if (req.userSession?.userId) {
    await this.kickFromRoom(req.userSession.userId, "Вы вышли из аккаунта");
  }
  const result = await userSessionService.removeSessionByToken(sessionToken);
  if (!result) {
    res.status(400).json({ message: "Сессия не найдена" });
    return;
  }
  res.status(200).json({ message: "all good" });
};
}
export default new AuthorizationApiController();
=====
FILE: src/express/api/controllers/room.api.controller.ts
=====
import type { Request, Response, NextFunction } from "express";
import roomService from "../../services/room.service.js";
import engine from "../../game/BlackjackEngine.js";
import roomRepo from "../../repositories/room.redis.repository.js";
import type { RoomMeta } from "../../models/room.meta.dto.js";
/**
* Контроллер REST API для управления комнатами.
* Маршруты: /api/lobby/rooms | /api/lobby/room/:roomId | /api/lobby/room/leave
*/
class RoomsApiController {
  /**
  * POST /api/lobby/rooms
  * Создаёт новую комнату для авторизованного пользователя.
  */
  createRoom = async (req: Request, res: Response, next: NextFunction) => {
    if (!req.userSession) {
      res.status(401).json({ message: "not logged in" });
      return;
    }
    const body = req.body ?? {};
    const result = await roomService.createRoomForUser(req.userSession.userId, {
      name: body.name,
      description: body.description,
      maxPlayersCount: body.maxPlayersCount,
      isPrivate: body.isPrivate,
      password: body.password,
      deckCount: body.deckCount,
    });
    if ("error" in result) {
      res.status(400).json({ message: result.error });
      return;
    }
    res.status(201).json({ message: "all good", roomId: result.roomId });
  };
}

```

```

};
/**
 * GET /api/lobby/rooms
 * Возвращает список всех активных комнат (мета-данные без игрового состояния).
 */
getAllRooms = async (req: Request, res: Response, next: NextFunction) => {
  const data: RoomMeta[] | null = await roomService.getAllRoomsMeta();
  if (!data) {
    res.json([]);
  } else {
    res.json(data);
  }
};
/**
 * PUT /api/lobby/room/:roomId
 * Добавляет пользователя в комнату (вход в комнату).
 * Для частных комнат нужен пароль в теле запроса.
 */
enterRoom = async (req: Request, res: Response, next: NextFunction) => {
  if (!req.userSession) {
    res.json({ message: "not logged" });
    return;
  }
  if (!req.body) {
    res.json({ message: "emptyBody" });
    return;
  }
  const roomId: string = req.params["roomId"] as string;
  const password: string|null = req.body?.password ?? null;
  if (!roomId) {
    res.json({ message: "invalid input" });
    return;
  }
  const userId = req.userSession.userId;
  const result = await roomService.addUserToRoom(roomId, userId, password);
  if (!result) {
    res.json({ message: "error when attempt to join!" });
    return;
  }
  res.json({ message: "all good" });
};
/**
 * Вспомогательный метод: уведомляет всех в комнате об уходе игрока
 * и обновляет состояние через Socket.IO.
 */
private notifyRoomUserLeft = async (roomId: string, userId: number) => {
  try {
    const { socketio } = await import("../server/server.js");
    socketio.to(roomId).emit("PLAYER_LEFT", { userId });
    // Отписываем все сокеты этого пользователя от комнаты
    const sockets = await socketio.fetchSockets();
    for (const socket of sockets) {
      if ((socket.data as any)?.userSession?.userId === userId) {
        socket.leave(roomId);
        (socket.data as any).currentRoomId = null;
      }
    }
    // Обновляем состояние комнаты для оставшихся игроков
    const game = await roomRepo.getRoomGame(roomId);
    if (!game) return;
    const state = await engine.buildRoomState(roomId);
    socketio.to(roomId).emit("ROOM_STATE", state);
  } catch (err) {
    console.error("[room leave notify error]", err);
  }
};
/**
 * PUT /api/lobby/room/leave
 * Выход текущего пользователя из его комнаты.
 */
leaveRoom = async (req: Request, res: Response, next: NextFunction) => {
  if (!req.userSession) {
    res.json({ message: "not logged" });
    return;
  }
  const userId = req.userSession.userId;
  const roomId = await roomService.removeUserFromCurrentRoom(userId);
  if (!roomId) {
    res.json({ message: "you out from any room!" });
    return;
  }
};

```

```

    }
    await this.notifyRoomUserLeft(roomId, userId);
    res.json({ message: "all good", roomId });
  };
  /**
   * DELETE /api/lobby/room/:roomId
   * Удаляет комнату, только если она пустая.
   */
  deleteRoom = async (req: Request, res: Response, next: NextFunction) => {
    if (!req.userSession) {
      res.status(401).json({ message: "not logged in" });
      return;
    }
    const roomId = req.params["roomId"] as string;
    if (!roomId) {
      res.status(400).json({ message: "invalid input" });
      return;
    }
    const deleted = await roomService.deleteEmptyRoom(roomId);
    if (!deleted) {
      res.status(409).json({ message: "room is not empty or does not exist" });
      return;
    }
    res.json({ message: "all good" });
  };
}
export default new RoomsApiController();
=====
FILE: src/express/api/routers/authorization.api.router.ts
=====
import { Router } from "express";
import AuthorizationApiController from "../controllers/authorization.api.controller.js";
/**
 * Роутер авторизационного API.
 * Базовый путь: /api/authorization (задаётся в express.ts)
 */
class AuthorizationApiRouter {
  router: Router = Router();
  constructor() {
    this.initialRoutes();
  }
  initialRoutes() {
    this.router.post("/login", AuthorizationApiController.login);
    this.router.post("/register", AuthorizationApiController.register);
    this.router.post("/logout", AuthorizationApiController.logout);
  }
}
export default new AuthorizationApiRouter().router;
=====
FILE: src/express/api/routers/rooms.api.router.ts
=====
import { Router } from "express";
import RoomsApiController from "../controllers/room.api.controller.js";
/**
 * Роутер API комнат.
 * Базовый путь: /api/lobby (задаётся в express.ts)
 * Защищён middleware checkForAuthAPI (только для авторизованных).
 */
class RoomsApiRouter {
  router: Router = Router();
  constructor() {
    this.initialRoutes();
  }
  initialRoutes() {
    this.router.post("/rooms", RoomsApiController.createRoom); // создать комнату
    this.router.put("/room/leave", RoomsApiController.leaveRoom); // выйти из комнаты
    this.router.get("/rooms", RoomsApiController.getAllRooms); // список всех
    КОМНАТ
    this.router.put("/room/:roomId", RoomsApiController.enterRoom); // войти в комнату
    this.router.delete("/room/:roomId", RoomsApiController.deleteRoom); // удалить комнату
  }
}
export default new RoomsApiRouter().router;
=====
FILE: src/express/middlewares/authentication.middleware.ts
=====
import CryptoGenerator from "../support/cryptoGenerator.js";
import type { Request, Response, NextFunction } from "express";
import UserSessionService from "../services/userSession.service.js";
/**

```

```

* Middleware аутентификации.
* Читает sessionToken из cookie и кладёт сессию в req.userSession.
* Не блокирует запрос – просто заполняет контекст (null если не авторизован).
*/
class AuthenticationMiddleware {
  checkUserByCookie = async (req: Request, res: Response, next: NextFunction) => {
    const sessionToken = req.cookies["sessionToken"];
    if (!sessionToken) {
      req.userSession = null;
      return next();
    }
    const userSession = await UserSessionService.getUserSessionByToken(sessionToken);
    if (!userSession) {
      req.userSession = null;
      return next();
    }
    req.userSession = userSession;
    return next();
  };
}
export default new AuthenticationMiddleware();
=====
FILE: src/express/middlewares/checkForAuthAPI.middleware.ts
=====
import type { Request, Response, NextFunction } from "express";
/**
 * Guard-middleware для API-роутов.
 * Возвращает 401 если пользователь не авторизован.
 * Использует req.userSession, установленный AuthenticationMiddleware.
 */
class CheckForAuthAPIMiddleware {
  checkUser = async (req: Request, res: Response, next: NextFunction) => {
    if (!req.userSession) {
      res.status(401).json({ message: "you need to login first!" });
      return;
    }
    next();
  };
}
export default new CheckForAuthAPIMiddleware();
=====
FILE: src/express/middlewares/checkForAuthentication.middleware.ts
=====
import type { Request, Response, NextFunction } from "express";
/**
 * Guard-middleware для View-роутов (SSR).
 * Перенаправляет неавторизованных на страницу регистрации.
 */
class CheckForAuthenticationMiddleware {
  checkUser = async (req: Request, res: Response, next: NextFunction) => {
    if (!req.userSession) {
      res.redirect("/authorization/register");
      return;
    }
    next();
  };
}
export default new CheckForAuthenticationMiddleware();
=====
FILE: src/express/middlewares/checkForUserInRoom.middleware.ts
=====
import type { Request, Response, NextFunction } from "express";
import roomService from "../services/room.service.js";
/**
 * Guard-middleware для страницы комнаты.
 * Проверяет, что текущий пользователь действительно находится в комнате :roomId.
 * Если нет – перенаправляет на /main.
 * Предотвращает прямой доступ к URL комнаты без предварительного входа через API.
 */
class CheckForUserInRoomMiddleware {
  checkUser = async (req: Request, res: Response, next: NextFunction) => {
    const roomId = req.params["roomId"] as string;
    if (!roomId || !req.userSession) {
      res.redirect("/main");
      return;
    }
    const users = await roomService.getAllUsersFromRoom(roomId);
    const includes = users.some(el => Number(el) === req.userSession?.userId);
    if (!includes) {

```

```

        res.redirect("/main");
        return;
    }
    next();
};
}
export default new CheckForUserInRoomMiddleware();
=====
FILE: src/express/middlewares/refreshCookie.middleware.ts
=====
import type { Request, Response, NextFunction } from "express";
import UserSessionService from "../services/userSession.service.js";
import { EXPIRATION_TIME } from "../support/expirationTime.helper.js";
/**
 * Middleware продления сессии.
 * При каждом запросе продлевает TTL сессии в Redis и обновляет maxAge cookie.
 * Если сессия истекла – очищает cookie.
 *
 * Это "скользящее" время жизни: пока пользователь активен, сессия не истекает.
 */
class RefreshSessionMiddleware {
    refreshSession = async (req: Request, res: Response, next: NextFunction) => {
        const sessionToken = req.cookies["sessionToken"];
        if (!sessionToken) {
            return next();
        }
        const userSession = await UserSessionService.refreshSessionByToken(sessionToken);
        if (!userSession) {
            // Сессия истекла или не найдена – удаляем невалидный cookie
            res.clearCookie("sessionToken", { path: "/" });
            return next();
        }
        // Обновляем maxAge cookie (скользящее окно)
        res.cookie("sessionToken", sessionToken, {
            maxAge: EXPIRATION_TIME.cookie,
            path: "/",
            httpOnly: true,
        });
        return next();
    };
};
}
export default new RefreshSessionMiddleware();
=====
FILE: src/express/models/hand.dto.ts
=====
/**
 * Статусы руки игрока в ходе раунда.
 *
 * ACTIVE – рука ещё в игре, игрок может делать ходы
 * STOOD – игрок остановился (stand)
 * DOUBLE-HIT – игрок удвоил ставку и получил одну карту
 * SURRENDER – игрок сдался (возврат 50% ставки)
 * BUST – сумма карт превысила 21
 * BLACKJACK – натуральный блекджек (туз + десятка при раздаче)
 */
export enum hand_status {
    ACTIVE = "ACTIVE",
    STOOD = "STOOD",
    DOUBLE = "DOUBLE-HIT",
    SURRENDER = "SURRENDER",
    BUST = "BUST",
    BLACKJACK = "BLACKJACK",
}
/**
 * Рука игрока в Redis.
 * cards – компактный формат: ["AH", "KS"] (ранг + масть)
 */
export interface Hand {
    bet: number;
    cards: string[];
    status: hand_status;
}
=====
FILE: src/express/models/player.dto.ts
=====
import type { Hand } from "../hand.dto.js";
/**
 * Данные игрока (хранятся в Redis per-player).
 * currentIndex – при split указывает на активную руку.
 */

```

```

export interface Player {
    currentHandIndex: 0;
    hands: Hand[];
}
=====
FILE: src/express/models/room.dto.ts
=====
/**
 * Статус игровой комнаты.
 * PLAYING — идёт раунд, BETTING — фаза ставок между раундами.
 */
export enum room_status {
    PLAYING = "PLAYING",
    BETTING = "BETTING",
}
/**
 * Игровое состояние комнаты (хранится в Redis hash room:game:<roomId>).
 *
 * dealer — карты дилера в компактном формате ["AH", "KS"]
 * deck — оставшиеся карты в колоде (массив в Redis как JSON-строка)
 * reshufflePending — флаг: перемешать колоду перед следующим startGame
 * playerOrder — порядок ходов (массив userId) — устанавливается при startGame
 */
export interface Room {
    roomId: string;
    dealer: string[];
    status: room_status;
    currentPlayer: number | null;
    deck: string[];
    reshufflePending: boolean;
    playerOrder: number[];
}
=====
FILE: src/express/models/room.meta.dto.ts
=====
/**
 * Мета-данные комнаты (хранятся в Redis hash room:meta:<roomId>).
 * Показываются в лобби без раскрытия игрового состояния.
 *
 * deckCount — количество колод, используемых в игре (1-8)
 * password — пустая строка для публичных комнат
 */
export interface RoomMeta {
    roomId: string;
    name: string;
    description: string;
    maxPlayersCount: number;
    currentPlayersCount: number;
    isPrivate: boolean;
    password: string;
    deckCount: number;
}
=====
FILE: src/express/models/user.dto.ts
=====
/**
 * Данные пользователя из PostgreSQL (таблица users).
 * hpassword — хранимый пароль (в текущей реализации без хэширования, см. user.service.ts).
 */
export default interface User {
    name: string;
    email: string;
    userId: number;
    balance: number;
    hpassword: string;
}
=====
FILE: src/express/models/userSession.dto.ts
=====
/**
 * Данные сессии пользователя (хранятся в Redis hash user:sess:<token>).
 * Не содержит пароль — это безопасный снимок профиля для сессионного слоя.
 */
export default interface UserSession {
    name: string;
    email: string;
    userId: number;
    balance: number;
}
=====

```

```

FILE: src/express/repositories/room.redis.repository.ts
=====
import RedisClient from "../../database/redis.js";
import type { RoomMeta } from "../../models/room.meta.dto.js";
import type { Room } from "../../models/room.dto.js";
import { roomStatus } from "../../models/room.dto.js";
import type { Hand } from "../../models/hand.dto.js";
// -----
// Схема ключей Redis
// -----
/**
 * rooms:active - Set всех активных roomId
 * room:meta:<roomId> - Hash мета-данных комнаты
 * room:game:<roomId> - Hash игрового состояния комнаты
 * room:users:<roomId> - Set userId игроков в комнате
 * room:player:<roomId>:user:<userId> - Hash состояния конкретного игрока
 * user:id:room:<userId> - String: текущий roomId пользователя
 */
const PREFIXES = {
  roomsActive: "rooms:active",
  roomMeta: (roomId: string) => `room:meta:${roomId}`,
  roomUser: (roomId: string, userId: number) => `room:player:${roomId}:user:${userId}`,
  roomUsers: (roomId: string) => `room:users:${roomId}`,
  roomGame: (roomId: string) => `room:game:${roomId}`,
  userIdRoom: (userId: number) => `user:id:room:${userId}`,
};
class RoomRedisRepository {
  /** Возвращает количество игроков в комнате */
  getPlayersCount = async (roomId: string) => {
    return RedisClient.sCard(PREFIXES.roomUsers(roomId));
  };
  /** Возвращает массив всех активных roomId или null если комнат нет */
  getActiveRoomIds = async () => {
    const rooms = await RedisClient.sMembers(PREFIXES.roomsActive);
    return rooms.length ? rooms : null;
  };
  /**
   * Возвращает мета-данные всех активных комнат за один пайплайн (MULTI).
   * Одновременно запрашивает мета-хэши и счётчики игроков для каждой комнаты.
   */
  getRoomMetas = async () => {
    const rooms = await this.getActiveRoomIds();
    if (!rooms) return null;
    const multi = RedisClient.multi();
    rooms.forEach(id => multi.hGetAll(PREFIXES.roomMeta(id)));
    rooms.forEach(id => multi.sCard(PREFIXES.roomUsers(id)));
    const raw = await multi.exec();
    const roomCount = rooms.length;
    const rawMetas = raw.slice(0, roomCount) as unknown as Record<string, string>[];
    const userCounts = raw.slice(roomCount);
    const metas: RoomMeta[] = [];
    rooms.forEach((id, i) => {
      const m = rawMetas[i];
      if (!m || Object.keys(m).length === 0) return; // пустой хэш = комната удалена
      metas.push({
        roomId: id,
        name: m["name"] || "",
        description: m["description"] || "",
        maxPlayersCount: Number(m["max_players_count"]),
        isPrivate: m["is_private"] === "true",
        password: m["password"] || "",
        currentPlayersCount: Number(userCounts[i]),
        deckCount: Number(m["deck_count"]) || 6,
      });
    });
    return metas;
  };
  /** Возвращает мета-данные одной комнаты (мета + счётчик игроков) */
  getRoomMeta = async (roomId: string): Promise<RoomMeta | null> => {
    const multi = RedisClient.multi();
    multi.hGetAll(PREFIXES.roomMeta(roomId));
    multi.sCard(PREFIXES.roomUsers(roomId));
    const raw = await multi.exec();
    const obj = raw[0] as unknown as Record<string, string>;
    if (!obj || Object.keys(obj).length === 0) return null;
    return {
      roomId,
      name: obj["name"] || "",
      description: obj["description"] || "",
      maxPlayersCount: Number(obj["max_players_count"]),
    };
  };
}

```

```

        isPrivate:          obj["is_private"] === "true",
        password:          obj["password"] || "",
        currentPlayersCount: Number(raw[1]),
        deckCount:         Number(obj["deck_count"]) || 6,
    };
};
/**
 * Удаляет все комнаты из Redis (вызывается при старте сервера).
 * Также очищает осиротевшие ключи user:id:room:*.
 * Возвращает количество удалённых комнат.
 */
deleteAllRooms = async (): Promise<number> => {
    const activeRoomIds = await this.getActiveRoomIds();
    const roomMetaKeys = await RedisClient.keys(`${PREFIXES.roomMeta("")}*`);
    const roomIdsFromMeta = roomMetaKeys.map(key => key.replace(PREFIXES.roomMeta(""),
    ""));
    const roomIds = Array.from(new Set([...(activeRoomIds ?? []),
    ...roomIdsFromMeta]));
    if (roomIds.length === 0) {
        await RedisClient.del(PREFIXES.roomsActive);
        const orphanedUserRooms = await RedisClient.keys("user:id:room:*");
        if (orphanedUserRooms.length) await RedisClient.del(orphanedUserRooms);
        return 0;
    }
    await Promise.all(roomIds.map(roomId => this.deleteRoom(roomId)));
    // Дополнительная зачистка осиротевших ключей
    const leftovers = await Promise.all([
        RedisClient.keys(`${PREFIXES.roomMeta("")}*`),
        RedisClient.keys(`${PREFIXES.roomGame("")}*`),
        RedisClient.keys(`${PREFIXES.roomUsers("")}*`),
        RedisClient.keys("room:player:*"),
        RedisClient.keys("user:id:room:*"),
    ]);
    const allLeftovers = [PREFIXES.roomsActive, ...leftovers.flat()];
    const toDelete = allLeftovers.filter(k => k);
    if (toDelete.length) await RedisClient.del(toDelete);
    return roomIds.length;
};
/** Возвращает текущий roomId пользователя или null */
getUserCurrentRoomId = async (userId: number) => {
    return await RedisClient.get(PREFIXES.userIdRoom(userId));
};
/** Возвращает Set всех userId игроков в комнате */
getAllUsersFromRoom = async (roomId: string) => {
    return await RedisClient.sMembers(PREFIXES.roomUsers(roomId));
};
getRoomUsers = async (roomId: string) => {
    return await RedisClient.sMembers(PREFIXES.roomUsers(roomId));
};
/**
 * Создаёт комнату в Redis (атомарно через MULTI):
 * - мета-данные
 * - игровое состояние (начальное: BETTING, пустая колода)
 * - добавляет создателя в список игроков
 * - маппинг userId → roomId
 */
createRoom = async (roomId: string, userId: number, roomMeta: RoomMeta) => {
    const multi = RedisClient.multi();
    multi.hSet(PREFIXES.roomMeta(roomId), {
        name: roomMeta.name,
        description: roomMeta.description,
        max_players_count: String(roomMeta.maxPlayersCount),
        is_private: roomMeta.isPrivate ? "true" : "false",
        password: roomMeta.password,
        deck_count: String(roomMeta.deckCount ?? 6),
    });
    multi.sAdd(PREFIXES.roomUsers(roomId), String(userId));
    multi.hSet(PREFIXES.roomGame(roomId), {
        room_id: roomId,
        dealer: "[]",
        deck: "[]",
        current_player: "",
        status: room_status.BETTING,
        reshuffle_pending: "false",
        player_order: "[]",
    });
    multi.set(PREFIXES.userIdRoom(userId), roomId);
    multi.hSet(PREFIXES.roomUser(roomId, userId), {
        current_hand_index: "0",
        hands: "[]",
    });
};

```

```

    });
    multi.sAdd(PREFIXES.roomsActive, roomId);
    await multi.exec();
    return roomId;
};
/**
 * Полностью удаляет комнату вместе с данными всех игроков.
 */
deleteRoom = async (roomId: string) => {
    const multi = RedisClient.multi();
    const usersInRoom = await RedisClient.sMembers(PREFIXES.roomUsers(roomId));
    // Удаляем данные каждого игрока и маппинг userId → roomId
    usersInRoom.forEach(uid =>
        multi.del([PREFIXES.roomUser(roomId, Number(uid)), PREFIXES.userIdRoom(Number(uid))])
    );
    multi.del([PREFIXES.roomMeta(roomId), PREFIXES.roomGame(roomId),
PREFIXES.roomUsers(roomId)]);
    multi.sRem(PREFIXES.roomsActive, roomId);
    await multi.exec();
    return roomId;
};
/**
 * Удаляет комнату только если она пустая.
 * Использует WATCH для защиты от race condition.
 */
deleteRoomIfEmpty = async (roomId: string) => {
    const usersKey = PREFIXES.roomUsers(roomId);
    const initialCount = await RedisClient.sCard(usersKey);
    if (initialCount !== 0) return false;
    // WATCH гарантирует, что между проверкой и удалением никто не зашёл
    await RedisClient.watch(usersKey);
    const latestCount = await RedisClient.sCard(usersKey);
    if (latestCount !== 0) {
        await RedisClient.unwatch();
        return false;
    }
    const multi = RedisClient.multi();
    multi.del([PREFIXES.roomMeta(roomId), PREFIXES.roomGame(roomId), usersKey]);
    multi.sRem(PREFIXES.roomsActive, roomId);
    const result = await multi.exec();
    return Array.isArray(result);
};
/**
 * Добавляет игрока в комнату.
 * Если игрок был в другой комнате — удаляет его оттуда.
 */
addUserToRoom = async (roomId: string, userId: number) => {
    await RedisClient.watch(PREFIXES.roomMeta(roomId));
    const multi = RedisClient.multi();
    const prevRoom = await RedisClient.get(PREFIXES.userIdRoom(userId));
    // Выходим из предыдущей комнаты (если была)
    if (prevRoom) multi.sRem(PREFIXES.roomUsers(prevRoom), String(userId));
    multi.set(PREFIXES.userIdRoom(userId), roomId);
    multi.hSet(PREFIXES.roomUser(roomId, userId), { current_hand_index: "0", hands: "[]" });
    multi.sAdd(PREFIXES.roomUsers(roomId), String(userId));
    const result = await multi.exec();
    return result ? roomId : null;
};
/** Удаляет игрока из конкретной комнаты */
removeUserFromRoom = async (roomId: string, userId: number) => {
    const currentRoom = await RedisClient.get(PREFIXES.userIdRoom(userId));
    const multi = RedisClient.multi();
    multi.sRem(PREFIXES.roomUsers(roomId), String(userId));
    multi.del(PREFIXES.roomUser(roomId, userId));
    // Удаляем маппинг userId → roomId только если это текущая комната
    if (currentRoom === roomId) multi.del(PREFIXES.userIdRoom(userId));
    await multi.exec();
    return roomId;
};
/** Удаляет игрока из его текущей комнаты. Возвращает roomId или null */
removeUserFromCurrentRoom = async (userId: number) => {
    const roomId = await RedisClient.get(PREFIXES.userIdRoom(userId));
    if (!roomId) return null;
    await this.removeUserFromRoom(roomId, userId);
    return roomId;
};
/** Читает игровое состояние комнаты из Redis */
getRoomGame = async (roomId: string): Promise<Room | null> => {
    const raw = await RedisClient.hGetAll(PREFIXES.roomGame(roomId));
    if (!raw || !raw["room_id"]) return null;

```

```

        return {
            roomId:      raw["room_id"],
            dealer:      JSON.parse(raw["dealer"] || "[]"),
            deck:        JSON.parse(raw["deck"] || "[]"),
            status:      (raw["status"] as room_status) || room_status.BETTING,
            currentPlayer: raw["current_player"] ? Number(raw["current_player"]) : null,
            reshufflePending: raw["reshuffle_pending"] === "true",
            playerOrder:  JSON.parse(raw["player_order"] || "[]"),
        };
    };

    /** Обновляет отдельные поля игрового состояния комнаты */
    setRoomGameFields = async (roomId: string, fields: Record<string, string>) => {
        await RedisClient.hSet(PREFIXES.roomGame(roomId), fields);
    };

    /** Читает руки и текущий индекс руки конкретного игрока */
    getPlayerState = async (roomId: string, userId: number): Promise<{ hands: Hand[];
    currentHandIndex: number } | null> => {
        const raw = await RedisClient.hGetAll(PREFIXES.roomUser(roomId, userId));
        if (!raw || raw["hands"] === undefined) return null;
        return {
            hands:      JSON.parse(raw["hands"] || "[]"),
            currentHandIndex: Number(raw["current_hand_index"] || 0),
        };
    };

    /** Сохраняет руки и текущий индекс руки игрока */
    setPlayerState = async (roomId: string, userId: number, hands: Hand[], currentHandIndex:
    number) => {
        await RedisClient.hSet(PREFIXES.roomUser(roomId, userId), {
            current_hand_index: String(currentHandIndex),
            hands:      JSON.stringify(hands),
        });
    };

    /** Возвращает количество колод, настроенных для комнаты (по умолчанию 6) */
    getDeckCount = async (roomId: string): Promise<number> => {
        const val = await RedisClient.hGet(PREFIXES.roomMeta(roomId), "deck_count");
        return val ? Number(val) : 6;
    };

    /**
     * Загружает состояния всех игроков за один пайплайн.
     * Возвращает Map<userId, { hands, currentHandIndex }>.
     */
    getAllPlayersStates = async (roomId: string): Promise<Map<number, { hands: Hand[];
    currentHandIndex: number }>> => {
        const userIds = await RedisClient.sMembers(PREFIXES.roomUsers(roomId));
        const multi = RedisClient.multi();
        userIds.forEach(id => multi.hGetAll(PREFIXES.roomUser(roomId, Number(id))));
        const raws = await multi.exec() as unknown as Record<string, string>[];
        const result = new Map<number, { hands: Hand[]; currentHandIndex: number }>();
        userIds.forEach((id, i) => {
            const raw = raws[i];
            if (raw && raw["hands"] !== undefined) {
                result.set(Number(id), {
                    hands:      JSON.parse(raw["hands"] || "[]"),
                    currentHandIndex: Number(raw["current_hand_index"] || 0),
                });
            }
        });
        return result;
    };
}

export { PREFIXES };
export default new RoomRedisRepository();
=====
FILE: src/express/repositories/user.pg.repository.ts
=====
import pool from "../../database/database.js";
import type User from "../../models/user.dto.js";
/**
 * Репозиторий пользователей (PostgreSQL).
 * Таблица: users (user_id, email, name, hpassword, balance)
 */
class UserPGRepository {
    /** Находит пользователя по ID */
    getUserById = async (userId: number) => {
        const data = await pool.query(
            `SELECT user_id as "userId", email, name, hpassword, balance FROM users WHERE
            id=$1;`,
            [userId]
        );
        return data.rows.length > 0 ? (data.rows[0] as User) : null;
    };
}

```

```

    };
    /** Создаёт нового пользователя, возвращает присвоенный user_id */
    setUser = async (user: User) => {
        const answer = await pool.query(
            `INSERT INTO users(email, name, hpassword, balance) VALUES($1,$2,$3,$4) RETURNING
user_id`,
            [user.email, user.name, user.hpassword, user.balance]
        );
        return Number(answer.rows[0]["user_id"]);
    };
    /** Находит пользователя по email */
    getUserByEmail = async (email: string) => {
        const data = await pool.query(
            `SELECT user_id as "userId", email, name, hpassword, balance FROM users WHERE
email=$1;`,
            [email]
        );
        return data.rows.length > 0 ? (data.rows[0] as User) : null;
    };
    /** Находит пользователя по никнейму */
    getUserByName = async (name: string) => {
        const data = await pool.query(
            `SELECT user_id as "userId", email, name, hpassword, balance FROM users WHERE
name=$1;`,
            [name]
        );
        return data.rows.length > 0 ? (data.rows[0] as User) : null;
    };
    /**
     * Находит пользователя по никнейму или email (для логина).
     * Позволяет входить по любому из двух идентификаторов.
     */
    getUserByEmailOrName = async (nameOrEmail: string) => {
        const data = await pool.query(
            `SELECT user_id as "userId", email, name, hpassword, balance FROM users WHERE name=$1
OR email=$1 LIMIT 1;`,
            [nameOrEmail]
        );
        return data.rows.length > 0 ? (data.rows[0] as User) : null;
    };
    /** Проверяет, существует ли пользователь с таким email или никнеймом */
    isUserCreated = async (identifier: string) => {
        const data = await pool.query(
            `SELECT user_id as "userId", email, name, hpassword, balance FROM users WHERE name=$1
OR email=$1 LIMIT 1;`,
            [identifier]
        );
        return data.rows.length > 0;
    };
    /** Обновляет баланс пользователя в PostgreSQL */
    updateBalance = async (userId: number, newBalance: number) => {
        await pool.query(`UPDATE users SET balance=$1 WHERE user_id=$2;`, [newBalance, userId]);
    };
}
export default new UserPGRepository();
=====
FILE: src/express/repositories/userSession.redis.repository.ts
=====
import RedisClient from "../../database/redis.js";
import type UserSession from "../../models/userSession.dto.js";
/**
 * Схема ключей:
 * user:sess:<sessionToken> - Hash данных сессии
 * user:id:sess:<userId> - String: токен сессии пользователя (для поиска по userId)
 */
const PREFIXES = {
    userSession: (userId: any) => `user:id:sess:${userId}`,
    session: (sessionToken: any) => `user:sess:${sessionToken}`,
};
class UserSessionRedisRepository {
    /** Возвращает сессию по токenu или null если не найдена / истекла */
    getUserSessionByToken = async (sessionToken: string) => {
        const sessionTokenKey = PREFIXES.session(sessionToken);
        const obj = await RedisClient.hGetAll(sessionTokenKey);
        if (!obj["userId"]) return null;
        return {
            name: obj["name"] as string,
            email: obj["email"] as string,
            balance: Number(obj["balance"]),
            userId: Number(obj["userId"]),
        };
    };
}

```

```

        } as UserSession;
    };
    /**
     * Создаёт новую сессию в Redis.
     * Предварительно удаляет старую сессию пользователя (one session per user).
     *
     * @param session      - данные сессии
     * @param generatedToken - токен, сгенерированный в UserSessionService
     * @param expirationTime - TTL в секундах
     */
    setSession = async (session: UserSession, generatedToken: string, expirationTime: number):
    Promise<string> => {
        await this.deleteSessionByUserId(session.userId);
        const sessionTokenKey = PREFIXES.session(generatedToken);
        await RedisClient.hSet(sessionTokenKey, session as any);
        await RedisClient.expire(sessionTokenKey, expirationTime);
        // Обратный маппинг userId → token (чтобы можно было найти сессию по userId)
        const userSessionKey = PREFIXES.userSession(session.userId);
        await RedisClient.set(userSessionKey, generatedToken, { EX: expirationTime });
        return generatedToken;
    };
    /** Удаляет сессию по userId (оба ключа: hash и строка-маппинг) */
    deleteSessionByUserId = async (userId: number) => {
        const session = await this.getSessionByUserId(userId);
        if (!session) return null;
        const userSessionKey = PREFIXES.userSession(userId);
        const sessionTokenKey = PREFIXES.session(session);
        await RedisClient.del([userSessionKey, sessionTokenKey]);
        return session;
    };
    /** Удаляет сессию по токenu */
    deleteSessionBySessionToken = async (sessionToken: string) => {
        if (!sessionToken) return null;
        const sessionTokenKey = PREFIXES.session(sessionToken);
        const userId = await RedisClient.hGet(sessionTokenKey, "userId");
        if (!userId) return null;
        const userSessionKey = PREFIXES.userSession(userId);
        await RedisClient.del([userSessionKey, sessionTokenKey]);
        return sessionToken;
    };
    /** Возвращает токен сессии пользователя по его userId */
    getSessionByUserId = async (userId: number) => {
        const userSessionKey = PREFIXES.userSession(userId);
        return await RedisClient.get(userSessionKey);
    };
    /**
     * Продлевает TTL сессии (скользящее время жизни).
     * Обновляет expire для обоих ключей.
     */
    refreshSessionByToken = async (sessionToken: string, expirationTime: number) => {
        if (!sessionToken) return null;
        const sessionTokenKey = PREFIXES.session(sessionToken);
        const userSession = await this.getUserSessionByToken(sessionToken);
        if (!userSession) return null;
        await RedisClient.expire(sessionTokenKey, expirationTime);
        const userSessionKey = PREFIXES.userSession(userSession.userId);
        await RedisClient.expire(userSessionKey, expirationTime);
        return sessionToken;
    };
}
export default new UserSessionRedisRepository();
=====
FILE: src/express/services/room.service.ts
=====
import crypto          from "node:crypto";
import roomRedisRepository from "../repositories/room.redis.repository.js";
import type { RoomMeta } from "../models/room.meta.dto.js";
/** Параметры создания комнаты (из тела запроса) */
export type CreateRoomInput = {
    name:          string;
    description?:  string;
    maxPlayersCount?: number;
    isPrivate?:    boolean;
    password?:     string;
    deckCount?:    number;
};
/**
 * Сервис управления комнатами.
 * Бизнес-логика поверх RoomRedisRepository.
 */

```

```

class RoomService {
  getPlayersCount = async (roomId: string) => {
    return await roomRedisRepository.getPlayersCount(roomId);
  };
  getAllRoomsMeta = async () => {
    return await roomRedisRepository.getRoomMetas();
  };
  createRoom = async (roomId: string, userId: number, roomMeta: RoomMeta) => {
    return await roomRedisRepository.createRoom(roomId, userId, roomMeta);
  };
  /**
   * Полная валидация и создание комнаты для пользователя.
   * Генерирует UUID для roomId, валидирует все поля.
   *
   * @returns { roomId } при успехе или { error } при ошибке валидации
   */
  createRoomForUser = async (userId: number, input: CreateRoomInput): Promise<{ roomId: string
} | { error: string }> => {
    const name = input.name?.trim();
    if (!name) return { error: "Укажите название комнаты" };
    const maxPlayersCount = Number(input.maxPlayersCount ?? 5);
    if (!Number.isInteger(maxPlayersCount) || maxPlayersCount < 2 || maxPlayersCount > 7) {
      return { error: "Количество игроков: от 2 до 7" };
    }
    const deckCount = Number(input.deckCount ?? 6);
    if (!Number.isInteger(deckCount) || deckCount < 1 || deckCount > 8) {
      return { error: "Количество колод: от 1 до 8" };
    }
    const isPrivate = Boolean(input.isPrivate);
    const password = isPrivate ? (input.password?.trim() ?? "") : "";
    if (isPrivate && !password) {
      return { error: "Для приватной комнаты нужен пароль" };
    }
    // Если игрок уже в комнате — удаляем его оттуда перед созданием новой
    const currentRoom = await roomRedisRepository.getUserCurrentRoomId(userId);
    if (currentRoom) {
      await roomRedisRepository.removeUserFromRoom(currentRoom, userId);
    }
    const roomId = crypto.randomUUID();
    const roomMeta: RoomMeta = {
      roomId,
      name,
      description: input.description?.trim() ?? "",
      maxPlayersCount,
      currentPlayersCount: 0,
      isPrivate,
      password,
      deckCount,
    };
    await roomRedisRepository.createRoom(roomId, userId, roomMeta);
    return { roomId };
  };
  deleteRoom = async (roomId: string) => {
    return await roomRedisRepository.deleteRoom(roomId);
  };
  /** Удаляет комнату только если она пустая */
  deleteEmptyRoom = async (roomId: string) => {
    const roomMeta = await roomRedisRepository.getRoomMeta(roomId);
    if (!roomMeta) return false;
    return await roomRedisRepository.deleteRoomIfEmpty(roomId);
  };
  /**
   * Добавляет игрока в комнату с проверкой пароля и лимита.
   * Возвращает null при ошибке (неверный пароль, комната заполнена, не найдена).
   */
  addUserToRoom = async (roomId: string, userId: number, password: string | null = null) => {
    const meta = await roomRedisRepository.getRoomMeta(roomId);
    if (!meta) return null;
    // Проверка пароля для приватных комнат
    if (meta.isPrivate) {
      if (!meta.password || meta.password !== password) return null;
    }
    // Проверка лимита игроков
    if (meta.currentPlayersCount >= meta.maxPlayersCount) return null;
    return await roomRedisRepository.addUserToRoom(roomId, userId);
  };
  removeUserFromRoom = async (roomId: string, userId: number) => {
    return await roomRedisRepository.removeUserFromRoom(roomId, userId);
  };
  getRoomMeta = async (roomId: string) => {

```

```

        return await roomRedisRepository.getRoomMeta(roomId);
    };
    /** Удаляет игрока из его текущей комнаты. Возвращает roomId или null */
    removeUserFromCurrentRoom = async (userId: number) => {
        const currentRoom = await roomRedisRepository.getUserCurrentRoomId(userId);
        if (!currentRoom) return null;
        await roomRedisRepository.removeUserFromRoom(currentRoom, userId);
        return currentRoom;
    };
    getAllUsersFromRoom = async (roomId: string) => {
        return await roomRedisRepository.getAllUsersFromRoom(roomId);
    };
}
export default new RoomService();
=====
FILE: src/express/services/user.service.ts
=====
import UserPgRepository from "../repositories/user.pg.repository.js";
import type User          from "../models/user.dto.js";
import type UserSession from "../models/userSession.dto.js";
import userPgRepository from "../repositories/user.pg.repository.js";
/**
 * Простое сравнение строк (паролей).
 * TODO: заменить на bcrypt.compare для безопасного хранения паролей.
 */
function compare(str1: string, str2: string) {
    return str1 === str2;
}
// Временный userId до вставки в БД (реальный присваивает PostgreSQL)
const UNKNOWN_USER_ID = -1000;
class UserService {
    getUserById = async (userId: number) => {
        return await UserPgRepository.getUserById(userId);
    };
    setNewUser = async (user: User) => {
        return await UserPgRepository.setUser(user);
    };
    getUserByEmail = async (email: string) => {
        return await UserPgRepository.getUserByEmail(email);
    };
    /**
     * Аутентифицирует пользователя.
     * Находит по email или никнейму, сравнивает пароль.
     * Возвращает UserSession (без пароля) или null.
     */
    authenticate = async (identifier: string, password: string) => {
        const user = await UserPgRepository.getUserByEmailOrName(identifier);
        if (!user) return null;
        const isMatch = compare(password, user.hpassword);
        if (!isMatch) return null;
        // Возвращаем все поля кроме пароля
        const { hpassword, ...userSession } = user;
        return userSession as UserSession;
    };
    /**
     * Регистрирует нового пользователя.
     * Проверяет уникальность никнейма и email.
     * Стартовый баланс: 500.
     *
     * TODO: хэшировать пароль перед сохранением.
     */
    addUser = async (name: string, email: string, password: string) => {
        if (await userPgRepository.isUserCreated(name) || await
            userPgRepository.isUserCreated(email)) {
            return null; // дубликат
        }
        const user: User = {
            balance: 500,
            email,
            hpassword: password, // TODO: bcrypt.hash(password)
            name,
            userId: UNKNOWN_USER_ID,
        };
        const userId = await userPgRepository.setUser(user);
        return { balance: user.balance, email: user.email, name: user.name, userId };
    };
}
export default new UserService();
=====
FILE: src/express/services/userSession.service.ts

```

```

=====
import userSessionRedisRepository from "../repositories/userSession.redis.repository.js";
import type UserSession           from "../models/userSession.dto.js";
import cryptoGenerator            from "../support/cryptoGenerator.js";
import { EXPIRATION_TIME }       from "../support/expirationTime.helper.js";
/**
 * Сервис управления сессиями пользователей.
 * Абстрагирует работу с Redis от контроллеров.
 */
class UserSessionService {
  /**
   * Создаёт новую сессию: генерирует токен и сохраняет в Redis.
   * Старая сессия пользователя удаляется автоматически (one session per user).
   */
  setSession = async (userSession: UserSession) => {
    if (!await userSessionRedisRepository.getSessionByUserId(userSession.userId)) {
      await userSessionRedisRepository.deleteSessionByUserId(userSession.userId);
    }
    const generatedToken = cryptoGenerator.generateSessionToken(16);
    return await userSessionRedisRepository.setSession(userSession, generatedToken,
      EXPIRATION_TIME.redis);
  };
  getUserSessionByToken = async (sessionToken: string) => {
    return await userSessionRedisRepository.getUserSessionByToken(sessionToken);
  };
  /** Удаляет сессию (logout) */
  removeSessionByToken = async (sessionToken: string) => {
    return await userSessionRedisRepository.deleteSessionBySessionToken(sessionToken);
  };
  /** Продлевает TTL сессии (вызывается RefreshSessionMiddleware) */
  refreshSessionByToken = async (sessionToken: string) => {
    return await userSessionRedisRepository.refreshSessionByToken(sessionToken,
      EXPIRATION_TIME.redis);
  };
}
export default new UserSessionService();
=====
FILE: src/express/support/cryptoGenerator.ts
=====
import crypto from "node:crypto";
/**
 * Генератор криптографически стойких случайных токенов.
 * Используется для создания sessionToken при входе/регистрации.
 */
class CryptoGenerator {
  /**
   * Генерирует токен в виде hex-строки.
   * @param length — длина в байтах (итоговая строка будет в 2 раза длиннее)
   */
  generateSessionToken = (length: number) => {
    return crypto.randomBytes(length).toString("hex");
  };
}
export default new CryptoGenerator();
=====
FILE: src/express/support/expirationTime.helper.ts
=====
/**
 * Время жизни сессии.
 *
 * cookie — в миллисекундах (для res.cookie maxAge): 12 часов
 * redis  — в секундах (для Redis EXPIRE): 12 часов
 *
 * Оба значения должны быть синхронизированы.
 */
export const EXPIRATION_TIME = {
  cookie: 12 * 60 * 60 * 1000, // 43 200 000 мс = 12 часов
  redis:  12 * 60 * 60,        // 43 200 сек   = 12 часов
};
=====
FILE: src/express/support/setCookie.support.ts
=====
import type { Request, Response } from "express";
import { EXPIRATION_TIME } from "../expirationTime.helper.js";
/**
 * Обработчик GET /setcookie?sessionToken=<token>
 *
 * После успешного логина/регистрации клиент перенаправляется сюда
 * для установки HttpOnly cookie с токеном сессии, а затем уходит на /main.
 */

```

```

    * HttpOnly защищает токен от кражи через XSS.
    */
const setCookie = async (req: Request, res: Response) => {
    const sessionToken = req.query["sessionToken"];
    res.cookie("sessionToken", sessionToken, {
        maxAge:    EXPIRATION_TIME.cookie,
        path:      "/",
        httpOnly:  true,
    });
    res.redirect("/main");
};
export default setCookie;
=====
FILE: src/express/views/controllers/about.view.controller.ts
=====
// Закомментированный контроллер страницы "О нас" – не используется в текущей версии
// import type { Request, Response, NextFunction } from "express";
// class AboutViewController {
//     renderPage = async (req, res, next) => {
//         res.render("room", { layout: "layout", user }, (err, html) => {
//             console.log("Кто-то зашел в комнату!", user ? user.name : "");
//             res.send(html);
//         });
//     };
// }
// export default new AboutViewController();
=====
FILE: src/express/views/controllers/authorization.view.controller.ts
=====
import type { Request, Response, NextFunction } from "express";
/**
 * Контроллер SSR-страниц авторизации.
 * Использует Handlebars: шаблон "authorization" с разными partial и скриптами.
 */
class AuthorizationViewController {
    /** GET /authorization/login → рендерит форму входа */
    renderLoginPage = async (req: Request, res: Response, next: NextFunction) => {
        res.render("authorization", {
            layout: "layout",
            scripts: "loginScripts",
            partial: "loginView",
            user:    req.userSession,
        });
    };
    /** GET /authorization/register → рендерит форму регистрации */
    renderRegistrationPage = async (req: Request, res: Response, next: NextFunction) => {
        res.render("authorization", {
            layout: "layout",
            scripts: "registerScripts",
            partial: "registerView",
            user:    req.userSession,
        });
    };
}
export default new AuthorizationViewController();
=====
FILE: src/express/views/controllers/main.view.controller.ts
=====
import type { Request, Response, NextFunction } from "express";
import UserService        from "../../services/user.service.js";
import UserSessionService from "../../services/userSession.service.js";
/**
 * Контроллер главной страницы.
 * Передаёт данные сессии в шаблон (для отображения имени пользователя).
 */
class MainViewController {
    renderPage = async (req: Request, res: Response, next: NextFunction) => {
        console.log(req.userSession);
        res.render("main", {
            layout: "layout",
            user:    req.userSession,
        }, (err, html) => {
            console.log("error:", err);
            res.send(html);
        });
    };
}
export default new MainViewController();
=====
FILE: src/express/views/controllers/room.view.controller.ts

```

```

=====
import type { Request, Response, NextFunction } from "express";
import roomService from "../../services/room.service.js";
/**
 * Контроллер SSR-страниц лобби и комнаты.
 */
class RoomViewController {
  /** GET /lobby/rooms → список всех комнат */
  RenderRoomsPage = async (req: Request, res: Response, next: NextFunction) => {
    const data = await roomService.getAllRoomsMeta() || [];
    res.render("rooms", { layout: "layout", rooms: data, user: req.userSession });
  };
  /**
   * GET /lobby/room/:roomId → страница игровой комнаты.
   * Передаёт roomId и userId в шаблон для Socket.IO-инициализации на клиенте.
   */
  RenderRoomPage = async (req: Request, res: Response, next: NextFunction) => {
    const roomId = req.params["roomId"];
    res.render("room", {
      layout: "roomLayout",
      user: req.userSession,
      roomId,
      userId: req.userSession?.userId,
    });
  };
}
export default new RoomViewController();
=====
FILE: src/express/views/routes/authorization.view.router.ts
=====
import { Router } from "express";
import AuthorizationViewController from "../../controllers/authorization.view.controller.js";
/**
 * Роутер SSR-страниц авторизации.
 * Базовый путь: /authorization (задаётся в express.ts)
 */
class AuthorizationViewRouter {
  router: Router = Router();
  constructor() {
    this.initialRoutes();
  }
  initialRoutes() {
    this.router.get("/login", AuthorizationViewController.renderLoginPage);
    this.router.get("/register", AuthorizationViewController.renderRegistrationPage);
    this.router.get("/", (req, res) => {
      res.redirect("/login"); // /authorization → /authorization/login
    });
  }
}
export default new AuthorizationViewRouter().router;
=====
FILE: src/express/views/routes/main.view.router.ts
=====
import { Router } from "express";
import MainViewController from "../../controllers/main.view.controller.js";
/**
 * Роутер главной страницы.
 * Базовый путь: /main
 */
class MainViewRouter {
  router: Router = Router();
  constructor() {
    this.initialRoutes();
  }
  initialRoutes() {
    this.router.get("", MainViewController.renderPage);
  }
}
export default new MainViewRouter().router;
=====
FILE: src/express/views/routes/room.view.router.ts
=====
import { Router } from "express";
import RoomViewController from "../../controllers/room.view.controller.js";
import checkForUserInRoomMiddleware from "../../middlewares/checkForUserInRoom.middleware.js";
/**
 * Роутер SSR-страниц лобби.
 * Базовый путь: /lobby (защищён checkForAuthentication в express.ts)
 *
 * /lobby/rooms — список комнат (открыт для любого авторизованного)

```

```

* /lobby/room/:roomId - страница комнаты (дополнительная проверка: игрок в комнате)
*/
class RoomViewRouter {
  router: Router = Router();
  constructor() {
    this.initialRoutes();
  }
  initialRoutes() {
    this.router.get("/rooms", RoomViewController.RenderRoomsPage);
    this.router.get("/room/:roomId",
      checkForUserInRoomMiddleware.checkUser, // guard: пользователь должен быть в комнате
      RoomViewController.RenderRoomPage
    );
  }
}
export default new RoomViewRouter().router;
=====
FILE: src/game/BlackjackEngine.ts
=====
import RedisClient      from "../database/redis.js";
import roomRepo          from "../express/repositories/room.redis.repository.js";
import userSessionRepo   from "../express/repositories/userSession.redis.repository.js";
import userPGRepo        from "../express/repositories/user.pg.repository.js";
import { hand_status }   from "../express/models/hand.dto.js";
import { room_status }   from "../express/models/room.dto.js";
import type { Hand }      from "../express/models/hand.dto.js";
import type { PlayerState, HandType, RoomState, RoundResultEntry, RoundResult } from
"./socket.io/socket.types.js";
// -----
// Константы колоды
// -----
const RANKS = ["2", "3", "4", "5", "6", "7", "8", "9", "T", "J", "Q", "K", "A"] as const;
const SUITS = ["H", "D", "C", "S"] as const;
/** Порог остатка карт в колоде - при достижении устанавливается reshufflePending */
const RESHUFFLE_THRESHOLD = 52;
// -----
// Чистые функции (используются в тестах и внутри движка)
// -----
/**
 * Возвращает числовое значение карты.
 * Туз = 11 (может уменьшиться до 1 в countHand).
 * T, J, Q, K = 10.
 */
export function cardValue(card: string): number {
  if (!card) return 0;
  const r = card[0]!;
  if (r === "A") return 11;
  if (["T", "J", "Q", "K"].includes(r)) return 10;
  return parseInt(r, 10);
}
/** Возвращает ранг карты (первый символ кода: "A", "K", "7" и т.п.) */
export function cardRank(card: string): string { return card[0]!; }
/**
 * Подсчитывает сумму руки с учётом мягких/жёстких тузов.
 * Скрытые карты ("??") игнорируются.
 *
 * Алгоритм: сначала считаем все тузы как 11, затем уменьшаем по одному
 * пока сумма > 21 и есть "мягкие" тузы.
 */
export function countHand(cards: string[]): number {
  let total = 0, aces = 0;
  for (const c of cards) {
    if (!c || c === "??") continue;
    total += cardValue(c);
    if (c[0] === "A") aces++;
  }
  while (total > 21 && aces > 0) { total -= 10; aces--; }
  return total;
}
/**
 * Генерирует колоду из deckCount стандартных колод (52 карты каждая).
 * Карты в формате "<ранг><масть>": "AH", "KS", "TD" и т.д.
 */
export function generateDeck(deckCount: number): string[] {
  const deck: string[] = [];
  for (let n = 0; n < deckCount; n++)
    for (const s of SUITS)
      for (const r of RANKS)
        deck.push(r + s);
  return deck;
}

```

```

}
/**
 * Перемешивает колоду на месте (алгоритм Фишера-Йейтса).
 */
export function shuffleDeck(deck: string[]): void {
  for (let i = deck.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [deck[i], deck[j]] = [deck[j]!, deck[i]!];
  }
}
/** Возвращает true если в колоде меньше RESHUFFLE_THRESHOLD карт */
export function shouldReshuffle(deck: string[]): boolean {
  return deck.length < RESHUFFLE_THRESHOLD;
}
/**
 * Проверяет возможность split: 2 карты с одинаковым рангом.
 * Примечание: Т и J не могут быть split, хотя оба стоят 10.
 */
export function canSplit(hand: Hand): boolean {
  return hand.cards.length === 2 && cardRank(hand.cards[0]!) === cardRank(hand.cards[1]!);
}
/** Проверяет возможность double down: только при 2 картах на руке */
export function canDouble(hand: Hand): boolean {
  return hand.cards.length === 2;
}
/** Рука завершена если статус отличен от ACTIVE */
function isHandDone(hand: Hand): boolean {
  return hand.status !== hand_status.ACTIVE;
}
/** Натуральный блекджек: ровно 2 карты с суммой 21 */
export function isNaturalBlackjack(cards: string[]): boolean {
  return cards.length === 2 && countHand(cards) === 21;
}
/**
 * Определяет результат одной руки игрока против дилера.
 *
 * Порядок проверок:
 * 1. Surrender/Bust → lose (ставка потеряна)
 * 2. Блекджек игрока vs блекджек дилера → push (возврат ставки)
 * 3. Блекджек игрока → выплата 2.5x
 * 4. Блекджек дилера (у игрока нет) → lose
 * 5. Перебор дилера → win (2x)
 * 6. Сравнение очков
 *
 * @returns { result, payout } — результат и сумма выплаты (0 при проигрыше)
 */
function resolveHand(
  hand: Hand,
  dealerCards: string[],
  dealerScore: number,
  dealerBust: boolean,
): { result: RoundResult; payout: number } {
  const playerScore = countHand(hand.cards);
  const playerBJ = hand.status === hand_status.BLACKJACK || isNaturalBlackjack(hand.cards);
  const dealerBJ = isNaturalBlackjack(dealerCards);
  if (hand.status === hand_status.SURRENDER) return { result: "lose", payout: 0 };
  if (hand.status === hand_status.BUST) return { result: "lose", payout: 0 };
  if (playerBJ) {
    if (dealerBJ) return { result: "push", payout: hand.bet };
    return { result: "blackjack", payout: Math.floor(hand.bet * 2.5) };
  }
  if (dealerBJ) return { result: "lose", payout: 0 };
  if (dealerBust) return { result: "win", payout: hand.bet * 2 };
  if (playerScore > dealerScore) return { result: "win", payout: hand.bet * 2 };
  if (playerScore < dealerScore) return { result: "lose", payout: 0 };
  return { result: "push", payout: hand.bet };
}
/**
 * Сводит результаты нескольких рук (при split) в один итог для отображения.
 * Приоритет: blackjack > win > push > lose.
 */
function summarizePlayerResult(
  handResults: RoundResult[],
  totalPayout: number,
  totalWagered: number,
): RoundResult {
  if (handResults.includes("blackjack")) return "blackjack";
  const netProfit = totalPayout - totalWagered;
  if (netProfit > 0) return "win";
  if (totalPayout > 0 && netProfit === 0) return "push";
}

```

```

    if (handResults.every(r => r === "push")) return "push";
    if (handResults.some(r => r === "win")) return "win";
    return "lose";
}
/**
 * Конвертирует внутренний объект Hand в HandState для отправки клиенту.
 * Добавляет вычисленное поле score.
 */
function toHandState(hand: Hand): HandState {
    return {
        bet: hand.bet,
        cards: hand.cards,
        status: hand.status as string as HandState["status"],
        score: countHand(hand.cards),
    };
}
// -----
// Движок игры (stateful методы — работают с Redis)
// -----
class BlackjackEngine {
    /**
     * Пополняет колоду если она пуста.
     * Вызывается перед каждым извлечением карты (защита от крашей).
     */
    private refillDeck = async (roomId: string, deck: string[]): Promise<void> => {
        if (deck.length > 0) return;
        const deckCount = await roomRepo.getDeckCount(roomId);
        deck.push(...generateDeck(deckCount));
        shuffleDeck(deck);
        await roomRepo.setRoomGameFields(roomId, { reshuffle_pending: "false" });
    };
    /**
     * Извлекает верхнюю карту из колоды.
     * Если после извлечения остаётся мало карт — ставит флаг reshufflePending.
     */
    private popCard = async (roomId: string, deck: string[]): Promise<string> => {
        await this.refillDeck(roomId, deck);
        const card = deck.pop()!;
        if (shouldReshuffle(deck)) {
            await roomRepo.setRoomGameFields(roomId, { reshuffle_pending: "true" });
        }
        return card;
    };
    /**
     * Собирает полное состояние комнаты для отправки клиентам (ROOM_STATE).
     *
     * @param revealDealer — если false, скрывает вторую карту дилера ("??")
     */
    buildRoomState = async (roomId: string, revealDealer = false): Promise<RoomState> => {
        const game = await roomRepo.getRoomGame(roomId);
        if (!game) throw new Error("Room game not found");
        const allStates = await roomRepo.getAllPlayersStates(roomId);
        const userIds = await roomRepo.getAllUsersFromRoom(roomId);
        const players: PlayerState[] = [];
        for (const uid of userIds) {
            const userId = Number(uid);
            const session = await userSessionRepo.getSessionByUserId(userId);
            const ps = allStates.get(userId) ?? { hands: [], currentHandIndex: 0 };
            players.push({
                userId,
                // Получаем имя и баланс из Redis-сессии
                name: session ? (await
userSessionRepo.getUserSessionByToken(session)).name ?? uid : uid,
                balance: session ? (await
userSessionRepo.getUserSessionByToken(session)).balance ?? 0 : 0,
                currentHandIndex: ps.currentHandIndex,
                hands: ps.hands.map(toHandState),
            });
        }
        const dealerCards = game.dealer;
        const dealerScore = countHand(dealerCards);
        return {
            roomId,
            status: game.status as RoomState["status"],
            currentPlayerId: game.currentPlayer,
            dealer: {
                // Во время игры скрываем вторую карту дилера
                cards: revealDealer || game.status === room_status.BETTING
                    ? dealerCards
                    : dealerCards.map((c, i) => i === 1 ? "??" : c),
            },
        };
    };
}

```

```

        score: revealDealer ? dealerScore : (dealerCards[0] ? cardValue(dealerCards[0]) :
0),
        },
        players,
        deckRemaining: game.deck.length,
    };
};
/**
 * Принимает ставку игрока в фазу BETTING.
 * Списывает сумму из Redis-баланса.
 *
 * @returns { newBalance, allBetsPlaced } или null при ошибке
 */
placeBet = async (roomId: string, userId: number, amount: number): Promise<{ newBalance:
number; allBetsPlaced: boolean } | null> => {
    if (amount <= 0) return null;
    const game = await roomRepo.getRoomGame(roomId);
    if (!game || game.status !== room_status.BETTING) return null;
    const sessionToken = await userSessionRepo.getSessionByUserId(userId);
    if (!sessionToken) return null;
    const session = await userSessionRepo.getUserSessionByToken(sessionToken);
    if (!session || session.balance < amount) return null;
    const ps = await roomRepo.getPlayerState(roomId, userId) ?? { hands: [],
currentHandIndex: 0 };
    // Запрет повторной ставки
    const alreadyBet = ps.hands.some(hand => hand.bet > 0);
    if (alreadyBet) return null;
    ps.hands = [{ bet: amount, cards: [], status: hand_status.ACTIVE }];
    await roomRepo.setPlayerState(roomId, userId, ps.hands, 0);
    // Обновляем баланс в Redis-сессии
    const newBalance = session.balance - amount;
    await userSessionRepo.setSession({ ...session, balance: newBalance }, sessionToken, 60 *
60 * 24);
    // Проверяем, все ли игроки сделали ставки
    const allStates = await roomRepo.getAllPlayersStates(roomId);
    const allBetsPlaced = [...allStates.values()].every(s => s.hands.length > 0 &&
s.hands[0]!.bet > 0);
    return { newBalance, allBetsPlaced };
};
/**
 * Запускает раунд: раздаёт карты всем игрокам и дилеру (2 круга).
 * Определяет начального игрока. Если у дилера блекджек – все стоят.
 */
startGame = async (roomId: string): Promise<RoomState> => {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game) throw new Error("Room not found");
    let deck = game.deck;
    // Перемешиваем если нужно (reshufflePending или колода пуста)
    if (deck.length === 0 || game.reshufflePending) {
        const deckCount = await roomRepo.getDeckCount(roomId);
        deck = generateDeck(deckCount);
        shuffleDeck(deck);
    }
    const userIds = (await roomRepo.getAllUsersFromRoom(roomId)).map(Number);
    const playerOrder = [...userIds];
    const dealer: string[] = [];
    // Раздача: 2 карты каждому игроку по очереди, затем 2 дилеру
    const playerHands = new Map<number, Hand[]>();
    for (const uid of playerOrder) {
        const ps = await roomRepo.getPlayerState(roomId, uid);
        const bet = ps?.hands[0]?.bet ?? 0;
        playerHands.set(uid, [{ bet, cards: [], status: hand_status.ACTIVE }]);
    }
    for (let round = 0; round < 2; round++) {
        for (const uid of playerOrder) {
            const card = await this.popCard(roomId, deck);
            playerHands.get(uid)![0]!.cards.push(card);
        }
        dealer.push(await this.popCard(roomId, deck));
    }
    const dealerHasBlackjack = isNaturalBlackjack(dealer);
    for (const uid of playerOrder) {
        const hand = playerHands.get(uid)![0]!;
        if (isNaturalBlackjack(hand.cards)) {
            hand.status = hand_status.BLACKJACK; // блекджек у игрока
        } else if (dealerHasBlackjack) {
            hand.status = hand_status.STOOD; // все автоматически стоят при ВJ дилера
        }
        await roomRepo.setPlayerState(roomId, uid, [hand], 0);
    }
}

```

```

// Первый игрок с ACTIVE-рукой (или null если все завершены/BJ у дилера)
const firstActive = dealerHasBlackjack
  ? null
  : playerOrder.find(uid => playerHands.get(uid)![0]!.status === hand_status.ACTIVE) ??
null;

await roomRepo.setRoomGameFields(roomId, {
  deck: JSON.stringify(deck),
  dealer: JSON.stringify(dealer),
  status: room_status.PLAYING,
  current_player: firstActive !== null ? String(firstActive) : "",
  player_order: JSON.stringify(playerOrder),
  reshuffle_pending: "false",
});
return this.buildRoomState(roomId, false);
};
/**
 * HIT — игрок берёт карту.
 * Автоматически ставит BUST если сумма > 21.
 */
playerHit = async (roomId: string, userId: number): Promise<{ hands: HandState[];
currentHandIndex: number } | null> => {
  const game = await roomRepo.getRoomGame(roomId);
  if (!game || game.status !== room_status.PLAYING || game.currentPlayer !== userId) return
null;

  const ps = await roomRepo.getPlayerState(roomId, userId);
  if (!ps) return null;
  const hand = ps.hands[ps.currentHandIndex];
  if (!hand || isHandDone(hand)) return null;
  const deck = game.deck;
  const card = await this.popCard(roomId, deck);
  hand.cards.push(card);
  await roomRepo.setRoomGameFields(roomId, { deck: JSON.stringify(deck) });
  if (countHand(hand.cards) > 21) hand.status = hand_status.BUST;
  await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
  return { hands: ps.hands.map(toHandState), currentHandIndex: ps.currentHandIndex };
};
/**
 * STAND — игрок останавливается (не берёт карт).
 */
playerStand = async (roomId: string, userId: number): Promise<{ hands: HandState[];
currentHandIndex: number } | null> => {
  const game = await roomRepo.getRoomGame(roomId);
  if (!game || game.status !== room_status.PLAYING || game.currentPlayer !== userId) return
null;

  const ps = await roomRepo.getPlayerState(roomId, userId);
  if (!ps) return null;
  const hand = ps.hands[ps.currentHandIndex];
  if (!hand || isHandDone(hand)) return null;
  hand.status = hand_status.STOOD;
  await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
  return { hands: ps.hands.map(toHandState), currentHandIndex: ps.currentHandIndex };
};
/**
 * DOUBLE DOWN — удваивает ставку, берёт ровно одну карту, завершает ход.
 * Списывает дополнительную ставку из баланса.
 */
playerDouble = async (roomId: string, userId: number): Promise<{ hands: HandState[];
currentHandIndex: number; balanceDeducted: number } | null> => {
  const game = await roomRepo.getRoomGame(roomId);
  if (!game || game.status !== room_status.PLAYING || game.currentPlayer !== userId) return
null;

  const ps = await roomRepo.getPlayerState(roomId, userId);
  if (!ps) return null;
  const hand = ps.hands[ps.currentHandIndex];
  if (!hand || !canDouble(hand)) return null;
  const sessionToken = await userSessionRepo.getSessionByUserId(userId);
  if (!sessionToken) return null;
  const session = await userSessionRepo.getUserSessionByToken(sessionToken);
  if (!session || session.balance < hand.bet) return null;
  // Списываем ещё hand.bet (ставка удваивается)
  const newBalance = session.balance - hand.bet;
  await userSessionRepo.setSession({ ...session, balance: newBalance }, sessionToken, 60 *
60 * 24);
  hand.bet *= 2;
  const deck = game.deck;
  const card = await this.popCard(roomId, deck);
  hand.cards.push(card);
  await roomRepo.setRoomGameFields(roomId, { deck: JSON.stringify(deck) });
  // DOUBLE-HIT при любом результате (даже bust)
  hand.status = countHand(hand.cards) > 21 ? hand_status.BUST : hand_status.DOUBLE;

```

```

    await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
    return {
      hands: ps.hands.map(toHandState),
      currentHandIndex: ps.currentHandIndex,
      balanceDeducted: hand.bet / 2, // сколько списали дополнительно
    };
  };
  /**
   * SPLIT — разделяет два одинаковых по рангу карты на две руки.
   * Каждая рука получает по одной новой карте.
   * Списывает ставку (равную исходной) для второй руки.
   */
  playerSplit = async (roomId: string, userId: number): Promise<{ hands: HandState[];
  currentHandIndex: number; balanceDeducted: number } | null> => {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game || game.status !== room_status.PLAYING || game.currentPlayer !== userId) return
    null;
    const ps = await roomRepo.getPlayerState(roomId, userId);
    if (!ps) return null;
    const hand = ps.hands[ps.currentHandIndex];
    if (!hand || !canSplit(hand)) return null;
    const sessionToken = await userSessionRepo.getSessionByUserId(userId);
    if (!sessionToken) return null;
    const session = await userSessionRepo.getUserSessionByToken(sessionToken);
    if (!session || session.balance < hand.bet) return null;
    // Списываем ставку для второй руки
    const newBalance = session.balance - hand.bet;
    await userSessionRepo.setSession({ ...session, balance: newBalance }, sessionToken, 60 *
    60 * 24);
    const deck = game.deck;
    // Создаём две руки: каждая получает исходную карту + новую
    const hand1: Hand = { bet: hand.bet, cards: [hand.cards[0]!, await this.popCard(roomId,
    deck)], status: hand_status.ACTIVE };
    const hand2: Hand = { bet: hand.bet, cards: [hand.cards[1]!, await this.popCard(roomId,
    deck)], status: hand_status.ACTIVE };
    await roomRepo.setRoomGameFields(roomId, { deck: JSON.stringify(deck) });
    // Заменяем текущую руку двумя новыми
    ps.hands.splice(ps.currentHandIndex, 1, hand1, hand2);
    await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
    return {
      hands: ps.hands.map(toHandState),
      currentHandIndex: ps.currentHandIndex,
      balanceDeducted: hand.bet,
    };
  };
  /**
   * SURRENDER — игрок сдаётся, получает обратно 50% ставки.
   * Доступно только при 2 картах на руке.
   */
  playerSurrender = async (roomId: string, userId: number): Promise<{ hands: HandState[];
  currentHandIndex: number; returned: number } | null> => {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game || game.status !== room_status.PLAYING || game.currentPlayer !== userId) return
    null;
    const ps = await roomRepo.getPlayerState(roomId, userId);
    if (!ps) return null;
    const hand = ps.hands[ps.currentHandIndex];
    if (!hand || hand.cards.length !== 2 || isHandDone(hand)) return null;
    hand.status = hand_status.SURRENDER;
    const returned = Math.floor(hand.bet / 2); // возврат 50%
    // Возвращаем половину ставки в баланс
    const sessionToken = await userSessionRepo.getSessionByUserId(userId);
    if (sessionToken) {
      const session = await userSessionRepo.getUserSessionByToken(sessionToken);
      if (session) {
        await userSessionRepo.setSession({ ...session, balance: session.balance +
        returned }, sessionToken, 60 * 60 * 24);
      }
    }
    await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
    return { hands: ps.hands.map(toHandState), currentHandIndex: ps.currentHandIndex,
    returned };
  };
  /**
   * Определяет, кто ходит следующим после завершения действия.
   *
   * Логика:
   * 1. Если у текущего игрока есть ещё незавершённые руки (после split) → тот же игрок
   * 2. Следующий игрок с незавершённой рукой
   * 3. Если таких нет → { dealerPlaying: true } — ход переходит к дилеру

```

```

    */
    nextTurn = async (roomId: string, currentUserId: number): Promise<
    | { nextUserId: number; currentHandIndex: number }
    | { dealerPlaying: true }
    | { roundOver: true; result: Awaited<ReturnType<BlackjackEngine["settleRound"]>> }
    > => {
        const game = await roomRepo.getRoomGame(roomId);
        if (!game) throw new Error("Game not found");
        // Проверяем следующую незавершённую руку у текущего игрока (split)
        const ps = await roomRepo.getPlayerState(roomId, currentUserId);
        if (ps) {
            const nextHandIdx = ps.hands.findIndex((h, i) => i > ps.currentHandIndex &&
!isHandDone(h));
            if (nextHandIdx !== -1) {
                await roomRepo.setPlayerState(roomId, currentUserId, ps.hands, nextHandIdx);
                await roomRepo.setRoomGameFields(roomId, { current_player: String(currentUserId)
});
                return { nextUserId: currentUserId, currentHandIndex: nextHandIdx };
            }
            // Ищем следующего игрока по порядку
            const order = game.playerOrder;
            const currIdx = order.indexOf(currentUserId);
            for (let i = currIdx + 1; i < order.length; i++) {
                const nextId = order[i]!;
                const nextPs = await roomRepo.getPlayerState(roomId, nextId);
                if (nextPs && nextPs.hands.some(h => !isHandDone(h))) {
                    await roomRepo.setRoomGameFields(roomId, { current_player: String(nextId) });
                    return { nextUserId: nextId, currentHandIndex: nextPs.currentHandIndex };
                }
            }
            // Все игроки завершили ходы → дилер играет
            return { dealerPlaying: true };
        }
    };
    /**
     * Проверяет, завершили ли все игроки свои ходы.
     * Используется после startGame для автоматического пропуска фазы игроков
     * (например, если у всех блекджек или дилер открылся с блекджеком).
     */
    isPlayerPhaseComplete = async (roomId: string): Promise<{ fromUserId: number } | null> => {
        const game = await roomRepo.getRoomGame(roomId);
        if (!game || game.status !== room_status.PLAYING) return null;
        for (const uid of game.playerOrder) {
            const ps = await roomRepo.getPlayerState(roomId, uid);
            if (ps?.hands.some(h => h.status === hand_status.ACTIVE)) return null;
        }
        if (game.playerOrder.length === 0) return null;
        const fromUserId = game.currentPlayer ?? game.playerOrder[game.playerOrder.length - 1]!;
        return { fromUserId };
    };
    /**
     * Дилер подбирает карты по правилу "стоит на 17+".
     * Возвращает финальные карты и счёт.
     */
    playDealer = async (roomId: string): Promise<{ cards: string[]; score: number }> => {
        const game = await roomRepo.getRoomGame(roomId);
        if (!game) throw new Error("Game not found");
        const deck = game.deck;
        const dealer = game.dealer.filter((c): c is string => !!c && c !== "??"); // убираем
рубашки
        while (countHand(dealer) < 17) {
            dealer.push(await this.popCard(roomId, deck));
        }
        await roomRepo.setRoomGameFields(roomId, {
            deck: JSON.stringify(deck),
            dealer: JSON.stringify(dealer),
        });
        return { cards: dealer, score: countHand(dealer) };
    };
    /**
     * Подводит итоги раунда для всех игроков.
     *
     * Для каждой руки вызывает resolveHand, суммирует выплаты,
     * обновляет баланс в Redis (и опционально в PostgreSQL).
     * Если был установлен reshufflePending — перемешивает колоду.
     */
    settleRound = async (roomId: string): Promise<{
        results: RoundResultEntry[];
        dealerCards: string[];
        dealerScore: number;
    }>

```

```

    reshuffled:    boolean;
  }> => {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game) throw new Error("Game not found");
    const dealerCards = game.dealer;
    const dealerScore = countHand(dealerCards);
    const dealerBust = dealerScore > 21;
    const results: RoundResultEntry[] = [];
    const allStates = await roomRepo.getAllPlayersStates(roomId);
    for (const [userId, ps] of allStates) {
      const sessionToken = await userSessionRepo.getSessionByUserId(userId);
      const session = sessionToken ? await
userSessionRepo.getUserSessionByToken(sessionToken) : null;
      const name = session?.name ?? String(userId);
      let totalPayout = 0;
      let totalWagered = 0;
      const handResults: RoundResult[] = [];
      for (const hand of ps.hands) {
        totalWagered += hand.bet;
        const { result, payout } = resolveHand(hand, dealerCards, dealerScore,
dealerBust);
        handResults.push(result);
        totalPayout += payout;
      }
      let newBalance = session?.balance ?? 0;
      if (totalPayout > 0 && session && sessionToken) {
        newBalance = session.balance + totalPayout;
        await userSessionRepo.setSession({ ...session, balance: newBalance },
sessionToken, 60 * 60 * 24);
        // PG-запись опциональна - не критична если упадёт
        try { await userPGRepo.updateBalance(userId, newBalance); } catch { /* PG
опционально */ }
      }
      // Учитываем возврат 50% при surrender в netProfit
      let netProfit = totalPayout - totalWagered;
      for (const hand of ps.hands) {
        if (hand.status === hand.status.SURRENDER) {
          netProfit += Math.floor(hand.bet / 2);
        }
      }
      results.push({
        userId,
        name,
        result:    summarizePlayerResult(handResults, totalPayout, totalWagered),
        payout:    totalPayout,
        netProfit,
        newBalance,
        hands:     ps.hands.map(toHandState),
      });
    }
    // Перемешиваем колоду если было установлено reshufflePending
    let reshuffled = false;
    if (game.reshufflePending) {
      const deckCount = await roomRepo.getDeckCount(roomId);
      const newDeck = generateDeck(deckCount);
      shuffleDeck(newDeck);
      await roomRepo.setRoomGameFields(roomId, {
        deck:    JSON.stringify(newDeck),
        reshuffle_pending: "false",
      });
      reshuffled = true;
    }
    return { results, dealerCards: game.dealer, dealerScore, reshuffled };
  };
  /**
   * Сбрасывает состояние раунда для следующей фазы ставок.
   * Очищает руки всех игроков, карты дилера и сменяет статус на BETTING.
   */
  resetRound = async (roomId: string) => {
    const userIds = await roomRepo.getAllUsersFromRoom(roomId);
    const multi = RedisClient.multi();
    const { PREFIXES } = await import("../express/repositories/room.redis.repository.js");
    for (const uid of userIds) {
      multi.hSet(PREFIXES.roomUser(roomId, Number(uid)), {
        current_hand_index: "0",
        hands:    "[]",
      });
    }
    multi.hSet(PREFIXES.roomGame(roomId), {
      dealer:    "[]",

```

```

        current_player: "",
        status:      room_status.BETTING,
        player_order:  "[]",
    });
    await multi.exec();
};
/**
 * Форсирует завершение всех активных рук игрока (если он отключился).
 * Статус → STOOD, чтобы игра могла продолжиться без него.
 */
forfeitPlayer = async (roomId: string, userId: number) => {
    const ps = await roomRepo.getPlayerState(roomId, userId);
    if (!ps) return;
    for (const hand of ps.hands) {
        if (!isHandDone(hand)) hand.status = hand_status.STOOD;
    }
    await roomRepo.setPlayerState(roomId, userId, ps.hands, ps.currentHandIndex);
};
}
export default new BlackjackEngine();
=====
FILE: src/index.ts
=====
import { server }      from "../server/server.js";
import redisClient     from "../database/redis.js";
import "../socket.io/socket.io.js"; // инициализирует все обработчики WebSocket
import roomRedisRepository from "../express/repositories/room.redis.repository.js";
/**
 * Точка входа приложения.
 *
 * Порядок запуска:
 * 1. Подключение к Redis (обязательно — без него нет сессий и комнат)
 * 2. Очистка комнат из Redis (старые данные невалидны после перезапуска)
 * 3. HTTP-сервер начинает слушать порт
 */
const bootstrap = async () => {
    // 1. Redis
    try {
        await redisClient.connect();
    } catch (err) {
        console.error(`failed to connect to redis on startup: ${err}`);
        process.exit(1); // без Redis приложение не может работать
    }
    // 2. Очистка старых комнат
    try {
        const removedRoomsCount = await roomRedisRepository.deleteAllRooms();
        if (removedRoomsCount > 0) {
            console.log(`deleted ${removedRoomsCount} rooms from Redis on startup`);
        }
    } catch (err) {
        console.error(`failed to clear redis rooms on startup: ${err}`);
        // Не критично — продолжаем запуск
    }
    // 3. HTTP-сервер
    server.listen(process.env.PORT, () => {
        console.log(`server starts listening on http://127.0.0.1:${process.env.PORT}`);
    });
};
void bootstrap();

=====
FILE: src/server/express.ts
=====
import express from "express";
import hbs     from "express-hbs";
import path    from "path";
import cp      from "cookie-parser";
// View-роутеры (SSR)
import authorizationViewRouter from "../express/views/routes/authorization.view.router.js";
import MainViewRouter         from "../express/views/routes/main.view.router.js";
import roomViewRouter         from "../express/views/routes/room.view.router.js";
// API-роутеры
import authorizationApiRouter from "../express/api/routers/authorization.api.router.js";
import roomsApiRouter         from "../express/api/routers/rooms.api.router.js";
// Middleware
import AuthorizationMiddleware from
"../express/middlewares/authentication.middleware.js";

```

```

import refreshCookieMiddleware      from
"./express/middlewares/refreshCookie.middleware.js";
import checkForAuthenticationMiddleware from
"./express/middlewares/checkForAuthentication.middleware.js";
import setCookie                    from "../express/support/setCookie.support.js";
import checkForAuthAPIMiddleware   from
"./express/middlewares/checkForAuthAPI.middleware.js";
import HandlebarsHelpers from "../handlebars_helpers/registerHelpers.js";
// -----
// Конфигурация Express + Handlebars
// -----
const app      = express();
const __dirname = import.meta.dirname;
// Статика: JS, CSS, изображения из /public
app.use(express.static(path.join(__dirname, "../public")));
// Шаблонизатор Handlebars
app.engine("hbs", hbs.express4({
  partialsDir: path.join(__dirname, "../views/partials"),
  layoutsDir: path.join(__dirname, "../views/layouts"),
}));
app.set("view engine", "hbs");
app.set("views", path.join(__dirname, "../views"));
HandlebarsHelpers(hbs); // регистрируем кастомные хелперы
// -----
// Специальный роут: установка cookie после логина/регистрации
// Должен быть ДО cookie-parser middleware (не нуждается в парсинге cookie)
// -----
app.get("/setcookie", setCookie);
// -----
// Глобальные middleware
// -----
app.use(cp()); // парсинг cookie
app.use(AuthorizationMiddleware.checkUserByCookie); // req.userSession из cookie
app.use(refreshCookieMiddleware.refreshSession); // продление TTL сессии
// -----
// View-роуты (SSR)
// -----
app.use("/authorization", authorizationViewRouter);
app.use("/lobby", checkForAuthenticationMiddleware.checkUser, roomViewRouter); // только для
авторизованных
app.use("/main", MainViewRouter);

// -----
// API-роуты (JSON)
// -----
app.use(express.json()); // парсинг JSON-тела запроса (только для API)
app.use("/api/lobby", checkForAuthAPIMiddleware.checkUser, roomsApiRouter);
app.use("/api/authorization", authorizationApiRouter);
// -----
// Фолбэк: любой неизвестный путь → /main
// -----
app.use("/", (req, res) => {
  res.redirect("/main");
});
export default app;
=====
FILE: src/server/server.ts
=====
import app from "../express.js";
import http from "node:http";
import { Server, Socket } from "socket.io";
import cookie from "cookie";
// Создаём HTTP-сервер поверх Express (нужен для Socket.IO)
const server = http.createServer(app);
/**
 * Socket.IO-сервер.
 * CORS разрешён для всех origin (в production стоит ограничить).
 *
 * Основная логика сокетов вынесена в src/socket.io/socket.io.ts.
 * Здесь только минимальная проверка cookie при подключении.
 */
const socketio = new Server(server, {
  cors: {
    origin: ["http://127.0.0.1:8888", "*"],
    methods: ["GET", "POST"],
  },
});

```

```

// Базовая проверка: отключаем соединения без sessionToken в cookie
socketio.on("connection", async (socket: Socket) => {
  const cookies = cookie.parse(socket.handshake.headers.cookie || "");
  const sessionToken = cookies["sessionToken"];
  if (!sessionToken) {
    return; // закрываем без уведомления (authenticateSocket middleware отработает позже)
  }
});
export { server, socketio };
=====
FILE: src/socket.io/socket.io.ts
=====
import type { Server, Socket } from "socket.io";
import { socketio } from "../server/server.js";
import { authenticateSocket, guardSession } from "../socket.middleware.js";
import engine from "../game/BlackjackEngine.js";
import roomService from "../express/services/room.service.js";
import userSessionRepo from "../express/repositories/userSession.redis.repository.js";
import roomRepo from "../express/repositories/room.redis.repository.js";
import { room_status } from "../express/models/room.dto.js";
import type {
  ClientToServerEvents,
  ServerToClientEvents,
  SocketData,
} from "../socket.types.js";
// Типизированный alias для io (для автодополнения событий)
const io = socketio as unknown as Server<ClientToServerEvents, ServerToClientEvents, {}, SocketData>;
// Таймеры "отложенного выхода": при disconnect даём 15 сек на переподключение
const disconnectTimers = new Map<number, NodeJS.Timeout>();
// Аутентификация при каждом соединении (до io.on("connection"))
io.use(authenticateSocket as any);

// -----
// Вспомогательные функции
// -----
/**
 * Возвращает актуальный баланс игрока из Redis-сессии.
 */
async function getPlayerBalance(userId: number): Promise<number> {
  const sessionToken = await userSessionRepo.getSessionByUserId(userId);
  if (!sessionToken) return 0;
  const session = await userSessionRepo.getUserSessionByToken(sessionToken);
  return session?.balance ?? 0;
}
/**
 * Определяет следующий ход и рассылает соответствующие события.
 *
 * Возможные исходы:
 * - Следующая рука/игрок → TURN_CHANGED
 * - Все завершили → дилер играет → DEALER_PLAY → ROUND_RESULT → BETTING_PHASE (через 3 сек)
 */
async function handleNextTurn(roomId: string, userId: number) {
  const turn = await engine.nextTurn(roomId, userId);
  if ("nextUserId" in turn) {
    // Передаём ход следующему игроку/руке
    io.to(roomId).emit("TURN_CHANGED", { userId: turn.nextUserId, currentHandIndex: turn.currentHandIndex });
    return;
  }
  if ("dealerPlaying" in turn) {
    // Дилер подбирает карты
    const dealerResult = await engine.playDealer(roomId);
    io.to(roomId).emit("DEALER_PLAY", dealerResult);
    // Подводим итоги раунда
    const settle = await engine.settleRound(roomId);
    io.to(roomId).emit("ROUND_RESULT", {
      results: settle.results,
      dealer: { cards: settle.dealerCards, score: settle.dealerScore },
    });
    // Рассылаем обновлённые балансы
    for (const r of settle.results) {
      io.to(roomId).emit("PLAYER_BALANCE", { userId: r.userId, balance: r.newBalance });
    }
    // Если колода была перемешана — уведомляем
    if (settle.reshuffled) {

```

```

        const g = await (await
import("../express/repositories/room.redis.repository.js")).default.getRoomGame(roomId);
        io.to(roomId).emit("DECK_SHUFFLED", { remainingCards: g?.deck.length ?? 0 });
    }
    // Сбрасываем раунд и через 3 секунды начинаем следующую фазу ставок
    await engine.resetRound(roomId);
    setTimeout(async () => {
        io.to(roomId).emit("BETTING_PHASE", {});
        const state = await engine.buildRoomState(roomId);
        io.to(roomId).emit("ROOM_STATE", state);
    }, 3000);
}
}
/**
 * Запускает игру: раздаёт карты, рассылает GAME_STARTED.
 * Если первый игрок уже завершён (блекджек/BJ дилера) – сразу переходим к следующему.
 */
async function emitGameStarted(roomId: string): Promise<void> {
    const gameState = await engine.startGame(roomId);
    io.to(roomId).emit("GAME_STARTED", gameState);
    const firstPlayer = gameState.currentPlayerId;
    if (firstPlayer !== null) {
        const fp = gameState.players.find(p => p.userId === firstPlayer);
        // Если первый игрок не имеет активных рук – пропускаем сразу
        if (fp?.hands.every(h => h.status !== "ACTIVE")) {
            await handleNextTurn(roomId, firstPlayer);
        }
        return;
    }
    // Если currentPlayerId null – возможно все завершены (все BJ или BJ у дилера)
    const advance = await engine.isPlayerPhaseComplete(roomId);
    if (advance) {
        await handleNextTurn(roomId, advance.fromUserId);
    }
}
}
/**
 * Проверяет, все ли игроки поставили ставки, и если да – запускает игру.
 * @returns true если игра запущена
 */
async function startGameIfAllBetsPlaced(roomId: string): Promise<boolean> {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game || game.status !== room_status.BETTING) return false;
    const userIds = await roomRepo.getAllUsersFromRoom(roomId);
    if (userIds.length === 0) return false;
    const allStates = await roomRepo.getAllPlayersStates(roomId);
    const allBetsPlaced = userIds.every(uid => {
        const state = allStates.get(Number(uid));
        return Boolean(state?.hands[0]?.bet && state.hands[0].bet > 0);
    });
    if (!allBetsPlaced) return false;
    await emitGameStarted(roomId);
    return true;
}
}
// -----
// Основной обработчик подключения
// -----
io.on("connection", async (socket: Socket<ClientToServerEvents, ServerToClientEvents, {},
SocketData>) => {
    const { userSession } = socket.data;
    if (!userSession) return;
    // Per-event middleware: проверяем валидность сессии перед каждым событием
    socket.use(async ([event, ...args], next) => {
        await guardSession(socket as any, next);
    });
    console.log(`[socket] connected: userId=${userSession.userId}, socketId=${socket.id}`);
    // -----
    // ROOM_JOIN – вход в комнату по WebSocket
    // -----
    socket.on("ROOM_JOIN", async ({ roomId }) => {
        const { userSession } = socket.data;
        if (!userSession) return;
        // Отменяем таймер "отложенного выхода" если он был запущен
        if (disconnectTimers.has(userSession.userId)) {
            clearTimeout(disconnectTimers.get(userSession.userId));
            disconnectTimers.delete(userSession.userId);
        }
        // Проверяем, что пользователь действительно в этой комнате (через Redis)
        const users = await roomService.getAllUsersFromRoom(roomId);
        if (!users.includes(String(userSession.userId))) {
            socket.emit("ERROR", { code: "NOT_IN_ROOM", message: "Вы не в этой комнате" });

```

```

        return;
    }
    // Покидаем предыдущую Socket.IO-комнату если нужно
    if (socket.data.currentRoomId && socket.data.currentRoomId !== roomId) {
        socket.leave(socket.data.currentRoomId);
    }
    await socket.join(roomId);
    socket.data.currentRoomId = roomId;
    // Отправляем текущее состояние новому игроку
    const state = await engine.buildRoomState(roomId);
    socket.emit("ROOM_STATE", state);
    // Уведомляем остальных о подключении
    socket.to(roomId).emit("PLAYER_JOINED", {
        userId: userSession.userId,
        name: userSession.name,
    });
});
// -----
// ROOM_LEAVE - добровольный выход из комнаты
// -----
socket.on("ROOM_LEAVE", async () => {
    await handleLeave(socket);
});
// -----
// disconnect - обрыв соединения (не всегда намеренный)
// Даём 15 секунд на переподключение перед тем как убрать игрока из комнаты
// -----
socket.on("disconnect", async () => {
    await handleLeave(socket, true);
    const { userSession, currentRoomId } = socket.data;
    if (!userSession) return;
    const timer = setTimeout(async () => {
        try {
            if (currentRoomId) {
                // Форсируем завершение ходов отключившегося игрока
                await engine.forfeitPlayer(currentRoomId, userSession.userId);
                const game = await (await
import("../express/repositories/room.redis.repository.js"))
                .default.getRoomGame(currentRoomId);
                // Если игрок был текущим ходящим - передаём ход дальше
                if (game?.currentPlayer === userSession.userId) {
                    await handleNextTurn(currentRoomId, userSession.userId);
                }
                io.to(currentRoomId).emit("PLAYER_LEFT", { userId: userSession.userId });
            }
            await roomService.removeUserFromCurrentRoom(userSession.userId);
            if (currentRoomId) {
                // Проверяем, не нужно ли запустить игру (все оставшиеся поставили)
                const gameStarted = await startGameIfAllBetsPlaced(currentRoomId);
                if (!gameStarted) {
                    const state = await engine.buildRoomState(currentRoomId);
                    io.to(currentRoomId).emit("ROOM_STATE", state);
                }
            }
        } catch (err) {
            console.error("[disconnect timeout error]", err);
        }
        disconnectTimers.delete(userSession.userId);
    }, 15000); // 15 секунд на переподключение
    disconnectTimers.set(userSession.userId, timer);
});
// -----
// GET_ROOM_STATE - ручной запрос текущего состояния комнаты
// Поддерживает acknowledgement (callback с результатом)
// -----
socket.on("GET_ROOM_STATE", async (payload, ack) => {
    const { userSession } = socket.data;
    if (!userSession) {
        ack?.({ ok: false, message: "Не авторизован" });
        return;
    }
    const roomId = payload?.roomId ?? socket.data.currentRoomId;
    if (!roomId) {
        ack?.({ ok: false, message: "Комната не указана" });
        return;
    }
    try {
        const game = await roomRepo.getRoomGame(roomId);
        if (!game) {
            ack?.({ ok: false, message: "Игра не найдена" });
        }
    }

```

```

        return;
    }
    // Скрываем карту дилера только во время активной игры
    const revealDealer = game.status !== room_status.PLAYING;
    const state = await engine.buildRoomState(roomId, revealDealer);
    socket.emit("ROOM_STATE", state);
    ack?.({ ok: true });
  } catch (err) {
    console.error("[GET_ROOM_STATE]", err);
    socket.emit("ERROR", { code: "STATE_FAILED", message: "Не удалось обновить состояние"
  });
  }
  ack?.({ ok: false, message: "Не удалось обновить состояние" });
});
// -----
// PLACE_BET — игрок делает ставку
// -----
socket.on("PLACE_BET", async ({ roomId, amount }) => {
  const { userSession } = socket.data;
  if (!userSession) return;
  const result = await engine.placeBet(roomId, userSession.userId, amount);
  if (!result) {
    socket.emit("ERROR", { code: "BET_FAILED", message: "Невозможно сделать ставку" });
    return;
  }
  // Подтверждаем ставку только этому игроку
  socket.emit("BET_CONFIRMED", { balance: result.newBalance });
  // Остальным сообщаем о новом балансе
  socket.to(roomId).emit("PLAYER_BALANCE", {
    userId: userSession.userId,
    balance: result.newBalance,
  });
  // Если все поставили — запускаем игру
  if (result.allBetsPlaced) {
    await emitGameStarted(roomId);
  }
});
// -----
// HIT — взять карту
// -----
socket.on("HIT", async ({ roomId }) => {
  const { userSession } = socket.data;
  if (!userSession) return;
  const result = await engine.playerHit(roomId, userSession.userId);
  if (!result) {
    socket.emit("ERROR", { code: "ACTION_FAILED", message: "Нельзя взять карту" });
    return;
  }
  io.to(roomId).emit("PLAYER_ACTION", {
    userId: userSession.userId,
    action: "HIT",
    hands: result.hands,
    currentHandIndex: result.currentHandIndex,
  });
  // Если рука bust — автоматически передаём ход
  const currentHand = result.hands[result.currentHandIndex];
  if (currentHand?.status === "BUST") {
    await handleNextTurn(roomId, userSession.userId);
  }
});

// -----
// STAND — остановиться
// -----
socket.on("STAND", async ({ roomId }) => {
  const { userSession } = socket.data;
  if (!userSession) return;
  const result = await engine.playerStand(roomId, userSession.userId);
  if (!result) {
    socket.emit("ERROR", { code: "ACTION_FAILED", message: "Нельзя остановиться" });
    return;
  }
  io.to(roomId).emit("PLAYER_ACTION", {
    userId: userSession.userId,
    action: "STAND",
    hands: result.hands,
    currentHandIndex: result.currentHandIndex,
  });
});

```

```

    });
    await handleNextTurn(roomId, userSession.userId);
  });
  // -----
  // DOUBLE – удвоить ставку
  // -----
  socket.on("DOUBLE", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;
    const result = await engine.playerDouble(roomId, userSession.userId);
    if (!result) {
      socket.emit("ERROR", { code: "ACTION_FAILED", message: "Нельзя удвоить" });
      return;
    }
    io.to(roomId).emit("PLAYER_ACTION", {
      userId:      userSession.userId,
      action:      "DOUBLE",
      hands:       result.hands,
      currentHandIndex: result.currentHandIndex,
      balance:     await getPlayerBalance(userSession.userId),
    });
    await handleNextTurn(roomId, userSession.userId);
  });
  // -----
  // SPLIT – разделить руку
  // -----
  socket.on("SPLIT", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;
    const result = await engine.playerSplit(roomId, userSession.userId);
    if (!result) {
      socket.emit("ERROR", { code: "ACTION_FAILED", message: "Нельзя сделать сплит" });
      return;
    }
    io.to(roomId).emit("PLAYER_ACTION", {
      userId:      userSession.userId,
      action:      "SPLIT",
      hands:       result.hands,
      currentHandIndex: result.currentHandIndex,
      balance:     await getPlayerBalance(userSession.userId),
    });
    // После split – передаём ход на первую из новых рук (того же игрока)
    io.to(roomId).emit("TURN_CHANGED", { userId: userSession.userId, currentHandIndex:
result.currentHandIndex });
  });
  // -----
  // SURRENDER – сдаться (возврат 50% ставки)
  // -----
  socket.on("SURRENDER", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;
    const result = await engine.playerSurrender(roomId, userSession.userId);
    if (!result) {
      socket.emit("ERROR", { code: "ACTION_FAILED", message: "Нельзя сдаться" });
      return;
    }
    io.to(roomId).emit("PLAYER_ACTION", {
      userId:      userSession.userId,
      action:      "SURRENDER",
      hands:       result.hands,
      currentHandIndex: result.currentHandIndex,
      balance:     await getPlayerBalance(userSession.userId),
    });
    await handleNextTurn(roomId, userSession.userId);
  });

  // -----
  // CHAT_MESSAGE – сообщение в чат комнаты
  // Ограничение: 300 символов, пустые сообщения игнорируются
  // -----
  socket.on("CHAT_MESSAGE", ({ roomId, text }) => {
    const { userSession } = socket.data;
    if (!userSession || !text.trim()) return;
    io.to(roomId).emit("CHAT_MESSAGE", {
      userId: userSession.userId,
      name:   userSession.name,
      text:   text.slice(0, 300),
    });
  });

```

```

    });
  });
});

// -----
// Вспомогательная функция: обработка выхода из комнаты
// -----
/**
 * Обрабатывает уход игрока из комнаты (добровольный или при disconnect).
 *
 * При добровольном выходе (isDisconnect=false):
 * - Форсируем завершение его рук (forfeit)
 * - Уведомляем комнату о PLAYER_LEFT
 * - Удаляем из Redis
 *
 * При disconnect (isDisconnect=true):
 * - Только помечаем currentRoomId = null в socket.data
 * - Реальный выход происходит в таймере (15 сек на переподключение)
 */
async function handleLeave(
  socket: Socket<ClientToServerEvents, ServerToClientEvents, {}, SocketData>,
  isDisconnect = false
) {
  const { userSession, currentRoomId } = socket.data;
  if (!userSession || !currentRoomId) return;
  if (!isDisconnect) {
    try {
      // Форсируем ходы игрока
      await engine.forfeitPlayer(currentRoomId, userSession.userId);
      const game = await (await import("../express/repositories/room.redis.repository.js"))
        .default.getRoomGame(currentRoomId);
      // Если игрок был текущим – передаём ход дальше
      if (game?.currentPlayer === userSession.userId) {
        await handleNextTurn(currentRoomId, userSession.userId);
      }
    } catch { /* Игнорируем ошибки при выходе */ }
    socket.to(currentRoomId).emit("PLAYER_LEFT", { userId: userSession.userId });
    await roomService.removeUserFromRoom(currentRoomId, userSession.userId);
    socket.leave(currentRoomId);
    socket.data.currentRoomId = null;
    // Проверяем не нужно ли запустить/обновить игру
    const gameStarted = await startGameIfAllBetsPlaced(currentRoomId);
    if (!gameStarted) {
      const state = await engine.buildRoomState(currentRoomId);
      io.to(currentRoomId).emit("ROOM_STATE", state);
    }
  }
}
export { io };

=====
FILE: src/socket.io/socket.middleware.ts
=====
import type { Socket } from "socket.io";
import cookie           from "cookie";
import userSessionService from "../express/services/userSession.service.js";
import type { SocketData } from "../socket.types.js";
type SocketWithData = Socket & { data: SocketData };
/**
 * Middleware аутентификации для Socket.IO.
 * Выполняется один раз при установке соединения (до io.on("connection")).
 *
 * Читает sessionToken из cookie, проверяет сессию в Redis.
 * При ошибке – эмитирует SESSION_INVALID и блокирует соединение.
 */
export const authenticateSocket = async (
  socket: SocketWithData,
  next: (err?: Error) => void
) => {
  const cookies = cookie.parse(socket.handshake.headers.cookie || "");
  const sessionToken = cookies["sessionToken"];

```

```

    if (!sessionToken) {
      socket.emit("SESSION_INVALID");
      return next(new Error("Not authenticated"));
    }
    const session = await userSessionService.getUserSessionByToken(sessionToken);
    if (!session) {
      socket.emit("SESSION_INVALID");
      return next(new Error("Session invalid or expired"));
    }
    // Сохраняем данные сессии в socket.data (доступны во всех обработчиках)
    socket.data.userSession = session;
    socket.data.sessionToken = sessionToken;
    socket.data.currentRoomId = null;
    next();
  };
  /**
   * Per-event middleware: проверяет валидность сессии перед каждым событием.
   * Используется через socket.use() внутри io.on("connection").
   *
   * Необходим для защиты от использования истёкшей сессии после подключения.
   */
  export const guardSession = async (
    socket: SocketWithData,
    next: (err?: Error) => void
  ) => {
    const token = socket.data.sessionToken;
    if (!token) {
      socket.emit("SESSION_INVALID");
      socket.disconnect();
      return;
    }
    const session = await userSessionService.getUserSessionByToken(token);
    if (!session) {
      socket.emit("SESSION_INVALID");
      socket.disconnect();
      return;
    }
    // Обновляем данные сессии (баланс мог измениться)
    socket.data.userSession = session;
    next();
  };
};

```

```

=====
FILE: src/socket.io/socket.types.ts
=====
import type UserSession from "../express/models/userSession.dto.js";
// -----
// Перечисления
// -----
/** Статус руки (см. hand.dto.ts) */
export type HandStatus = "ACTIVE" | "STOOD" | "DOUBLE-HIT" | "SURRENDER" | "BUST" | "BLACKJACK";
/** Статус комнаты */
export type RoomStatus = "PLAYING" | "BETTING";
/** Итог руки/паунда */
export type RoundResult = "blackjack" | "win" | "lose" | "push";
// -----
// Состояния (передаются клиенту)
// -----
/** Состояние одной руки для клиента (включает вычисленный score) */
export interface HandState {
  bet: number;
  cards: string[]; // компактный формат: ["AH", "KS"]
  status: HandStatus;
  score: number;
}
/** Состояние игрока (включает все его руки) */
export interface PlayerState {
  userId: number;
  name: string;
  balance: number;
  currentHandIndex: number;
  hands: HandState[];
}
/**
 * Полное состояние комнаты (рассылается при ROOM_STATE / GAME_STARTED).
 * deckRemaining — количество оставшихся карт в колоде (для UI-индикатора).

```

```

*/
export interface RoomState {
  roomId: string;
  status: RoomStatus;
  currentPlayerId: number | null;
  dealer: {
    cards: string[]; // вторая карта может быть "?" во время игры
    score: number;
  };
  players: PlayerState[];
  deckRemaining: number;
}

/** Событие об игровом действии игрока (рассылается всем в комнате) */
export interface PlayerActionEvent {
  userId: number;
  action: "HIT" | "STAND" | "DOUBLE" | "SPLIT" | "SURRENDER";
  hands: HandState[];
  currentHandIndex: number;
  balance?: number; // только при double/split/surrender
}

/** Запись результата раунда для одного игрока */
export interface RoundResultEntry {
  userId: number;
  name: string;
  result: RoundResult;
  payout: number; // сумма выплаты (0 при проигрыше)
  netProfit: number; // payout - wagered (может быть отрицательным)
  newBalance: number;
  hands: HandState[];
}

/** Событие ROUND_RESULT */
export interface RoundResultEvent {
  results: RoundResultEntry[];
  dealer: {
    cards: string[];
    score: number;
  };
};

// -----
// Типизация событий Socket.IO
// -----

/** События от клиента к серверу */
export interface ClientToServerEvents {
  ROOM_JOIN: (data: { roomId: string }) => void;
  ROOM_LEAVE: () => void;
  PLACE_BET: (data: { roomId: string; amount: number }) => void;
  HIT: (data: { roomId: string }) => void;
  STAND: (data: { roomId: string }) => void;
  DOUBLE: (data: { roomId: string }) => void;
  SPLIT: (data: { roomId: string }) => void;
  SURRENDER: (data: { roomId: string }) => void;
  CHAT_MESSAGE: (data: { roomId: string; text: string }) => void;
  GET_ROOM_STATE: (data?: { roomId?: string }, ack?: (response: { ok: boolean; message?: string }) => void) => void;
}

/** События от сервера к клиенту */
export interface ServerToClientEvents {
  ROOM_STATE: (data: RoomState) => void;
  PLAYER_JOINED: (data: { userId: number; name: string }) => void;
  PLAYER_LEFT: (data: { userId: number }) => void;
  GAME_STARTED: (data: RoomState) => void;
  PLAYER_ACTION: (data: PlayerActionEvent) => void;
  TURN_CHANGED: (data: { userId: number; currentHandIndex?: number }) => void;
  DEALER_PLAY: (data: { cards: string[]; score: number }) => void;
  ROUND_RESULT: (data: RoundResultEvent) => void;
  BETTING_PHASE: (data: { deckShuffled?: boolean }) => void;
  DECK_SHUFFLED: (data: { remainingCards: number }) => void;
  ERROR: (data: { code: string; message: string }) => void;
  CHAT_MESSAGE: (data: { userId: number; name: string; text: string }) => void;
  FORCE_DISCONNECT: (data: { reason: string }) => void;
  BET_CONFIRMED: (data: { balance: number }) => void;
  PLAYER_BALANCE: (data: { userId: number; balance: number }) => void;
  SESSION_INVALID: () => void;
}

/** Данные, хранимые в socket.data (устанавливаются middleware) */
export interface SocketData {
  userSession: UserSession | null;
  sessionToken: string | null;
  currentRoomId: string | null;
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы